

Chapter 10: Direct Volume Rendering

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

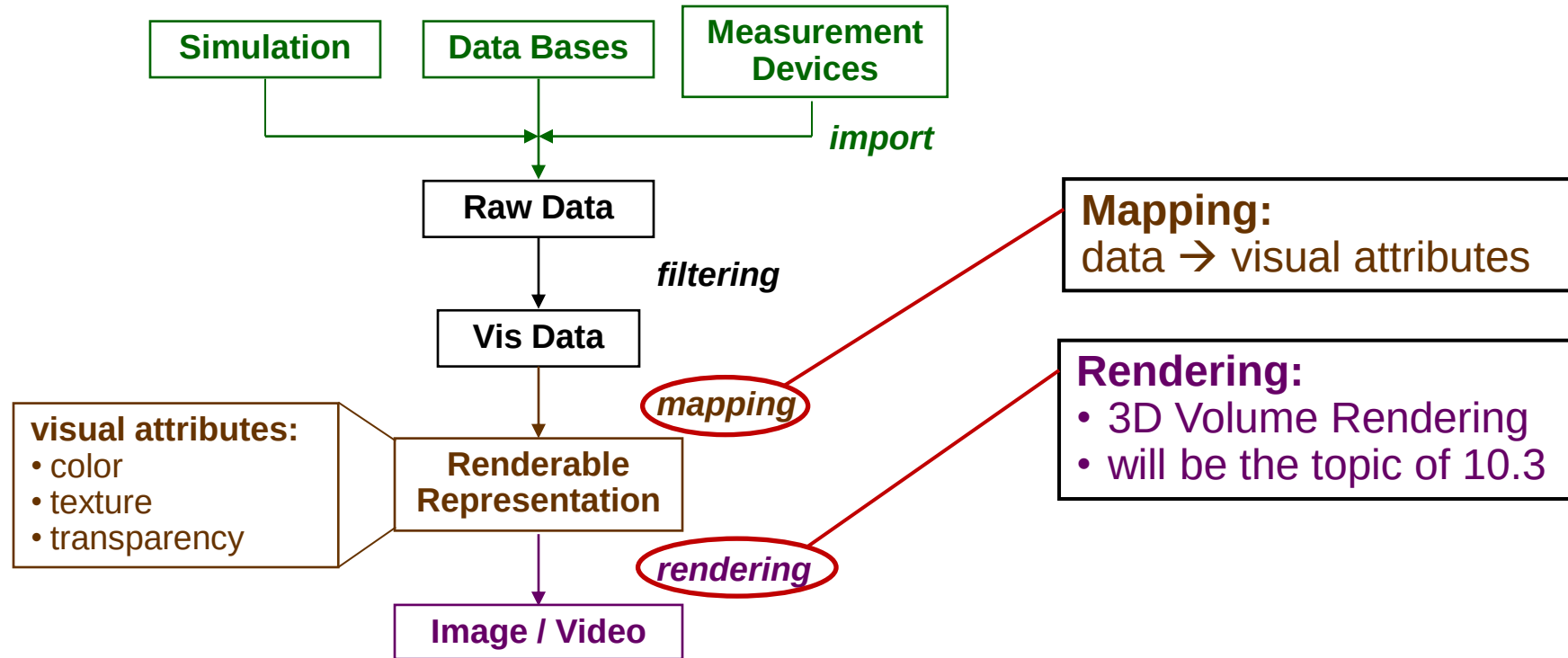
E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

June 17/23, 2026

Section 10.1: Transfer Functions

Where We Are In The Vis Pipeline



Impact of Transfer Function

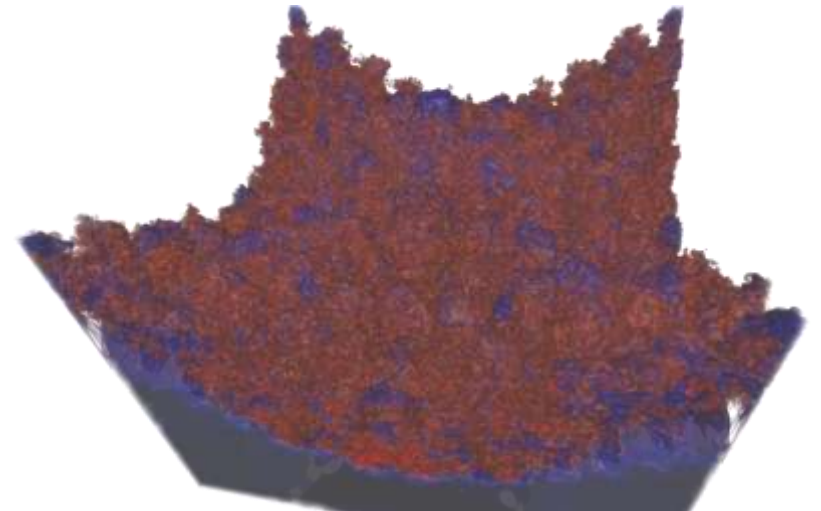
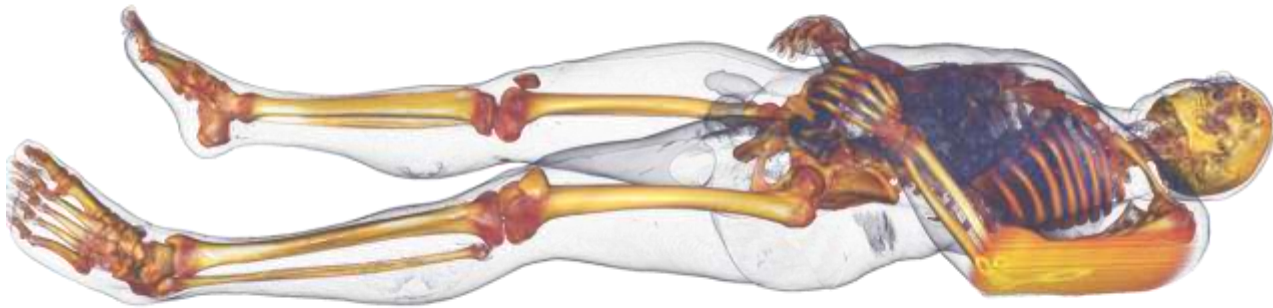


Transfer Functions

A **transfer function** maps data values to attributes that can be used for visualization.

In **direct volume rendering**, transfer functions assign a color and an opacity (RGBA) to each point in the 3D domain

- *Less opacity means more transparent*



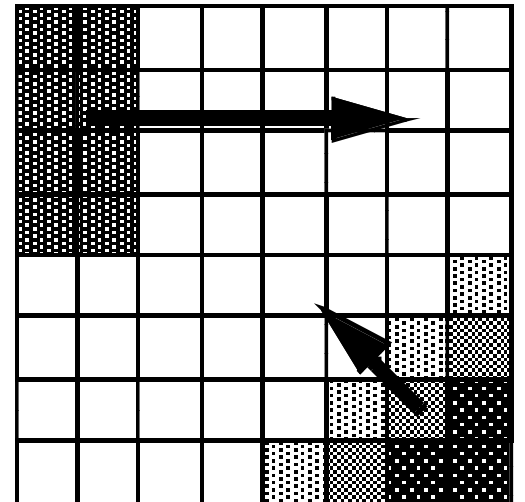
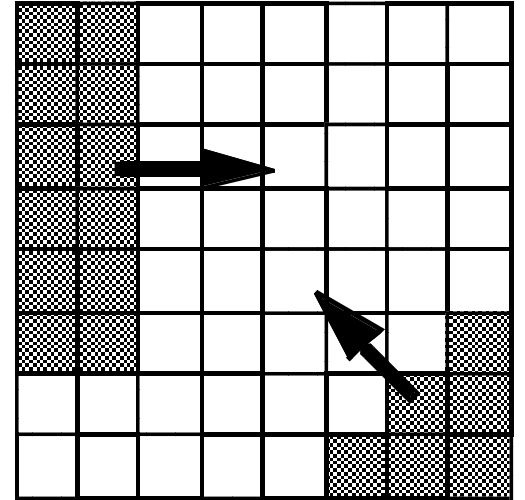
Multi-Dimensional Transfer Functions

- A **1D transfer function** maps scalar values to color and opacity
 - Note similarity to color maps
- In order to **highlight object boundaries** and to **disambiguate materials with similar intensity**, multi-dimensional transfer functions account for additional information, such as
 - Gradient magnitude
 - Second derivatives
 - Output of segmentation algorithms

The Importance of Gradient Information

In many cases, **rapid changes in data values** are important features

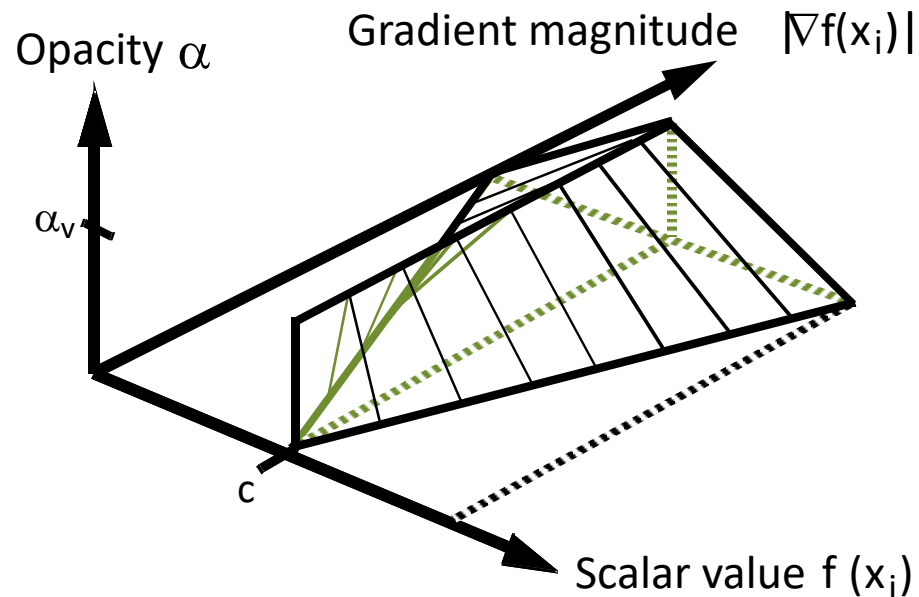
- Boundaries between different objects
- Indicated by a large gradient magnitude
 - see Chapter 9.3 for its computation
- Related to isosurface stability
- May want to assign a high value of opacity in regions of change



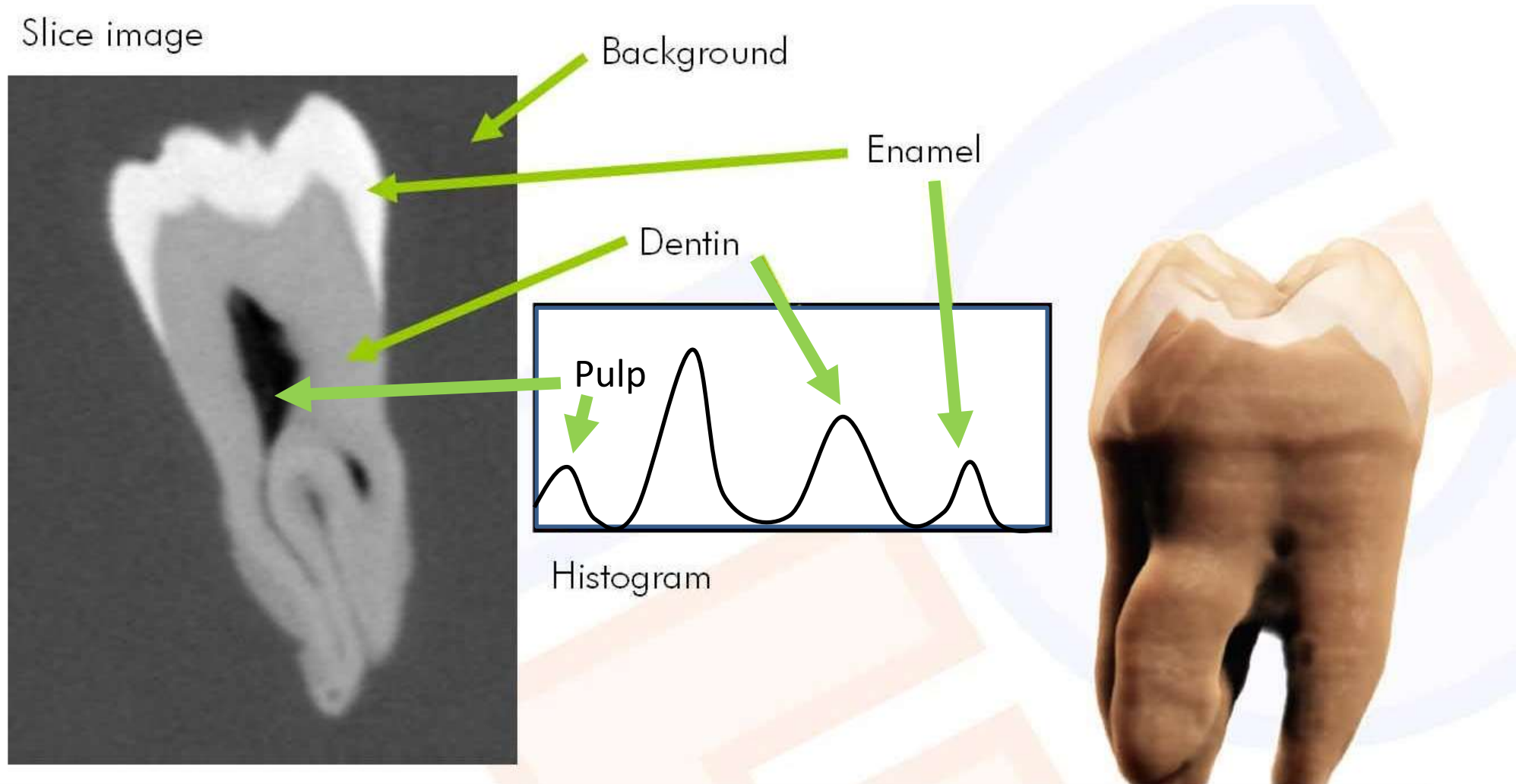
Gradient in Isosurface Rendering

Levoy [1988]: Transfer function to render isosurface at c should account for gradient magnitude $\|\nabla f(\mathbf{x})\|$:

$$\alpha(\mathbf{x}) = \alpha_v \begin{cases} \max\left(0, 1 - \frac{1}{r} \left| \frac{c - f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|} \right| \right) & \text{if } \|\nabla f(\mathbf{x})\| > 0 \\ 1 & \text{if } \|\nabla f(\mathbf{x})\| = 0 \text{ and } f(\mathbf{x}) = c \\ 0 & \text{if } \|\nabla f(\mathbf{x})\| = 0 \text{ and } f(\mathbf{x}) \neq c \end{cases}$$

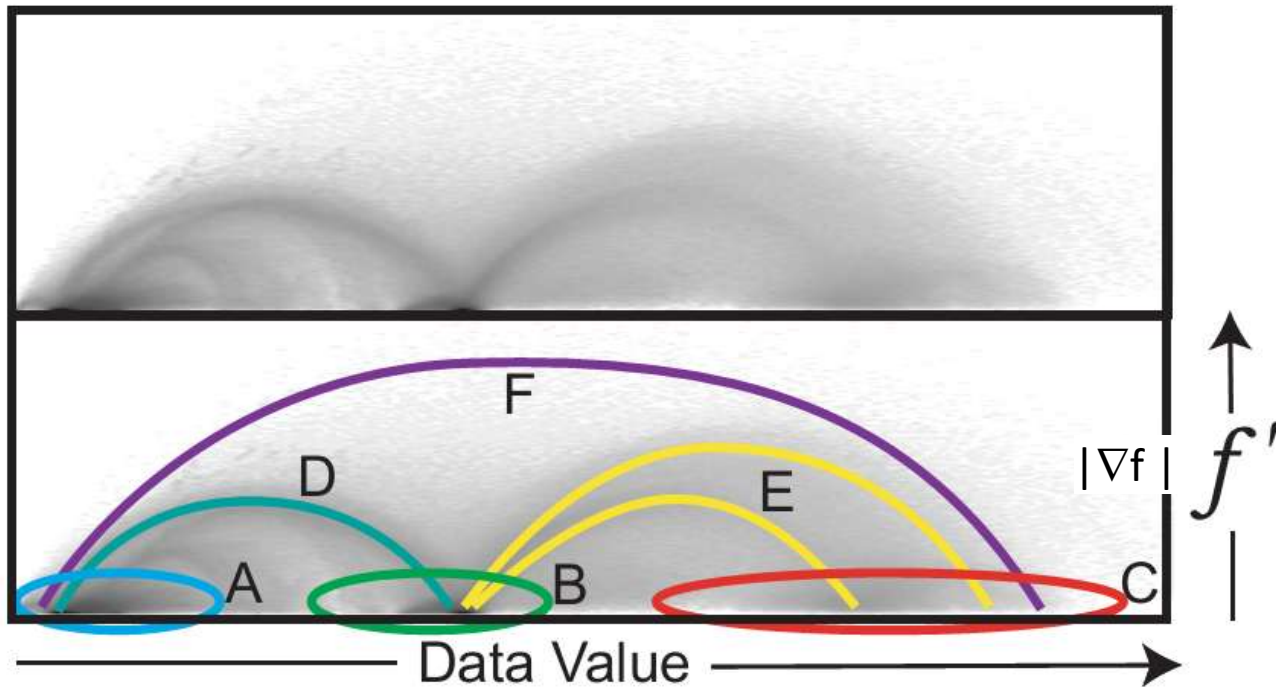


Example: Tooth CT Dataset



2D Transfer Functions

- General 2D TFs used to highlight boundaries
- 2D histogram (x: value, y:gradient magnitude)
 - Material boundaries correspond to arcs

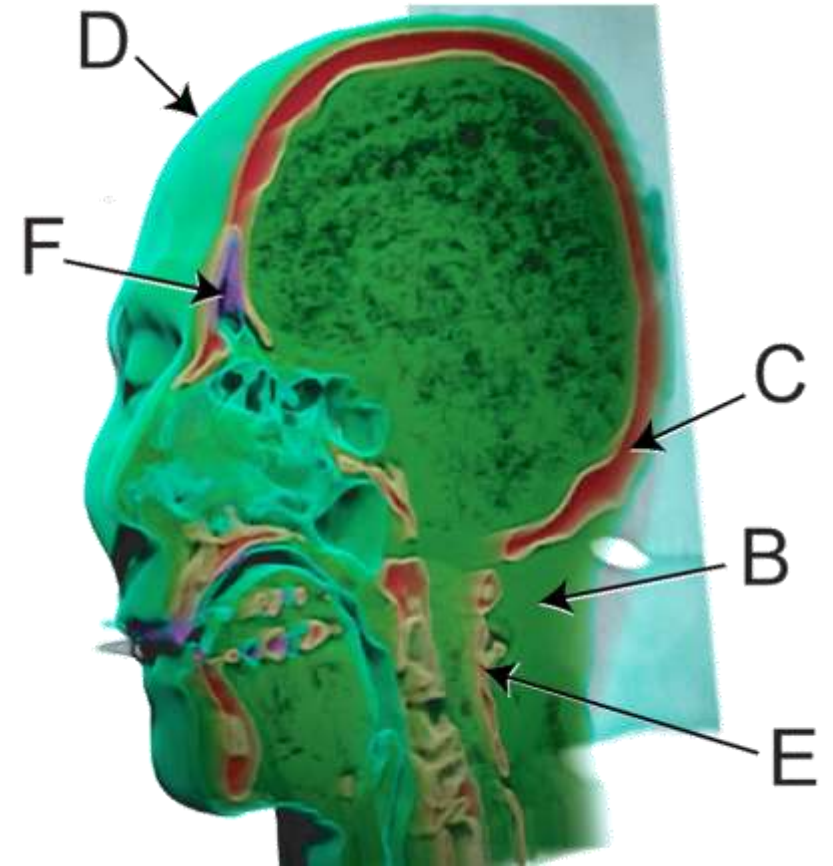


Materials:

A: air
B: soft tissue
C: bone

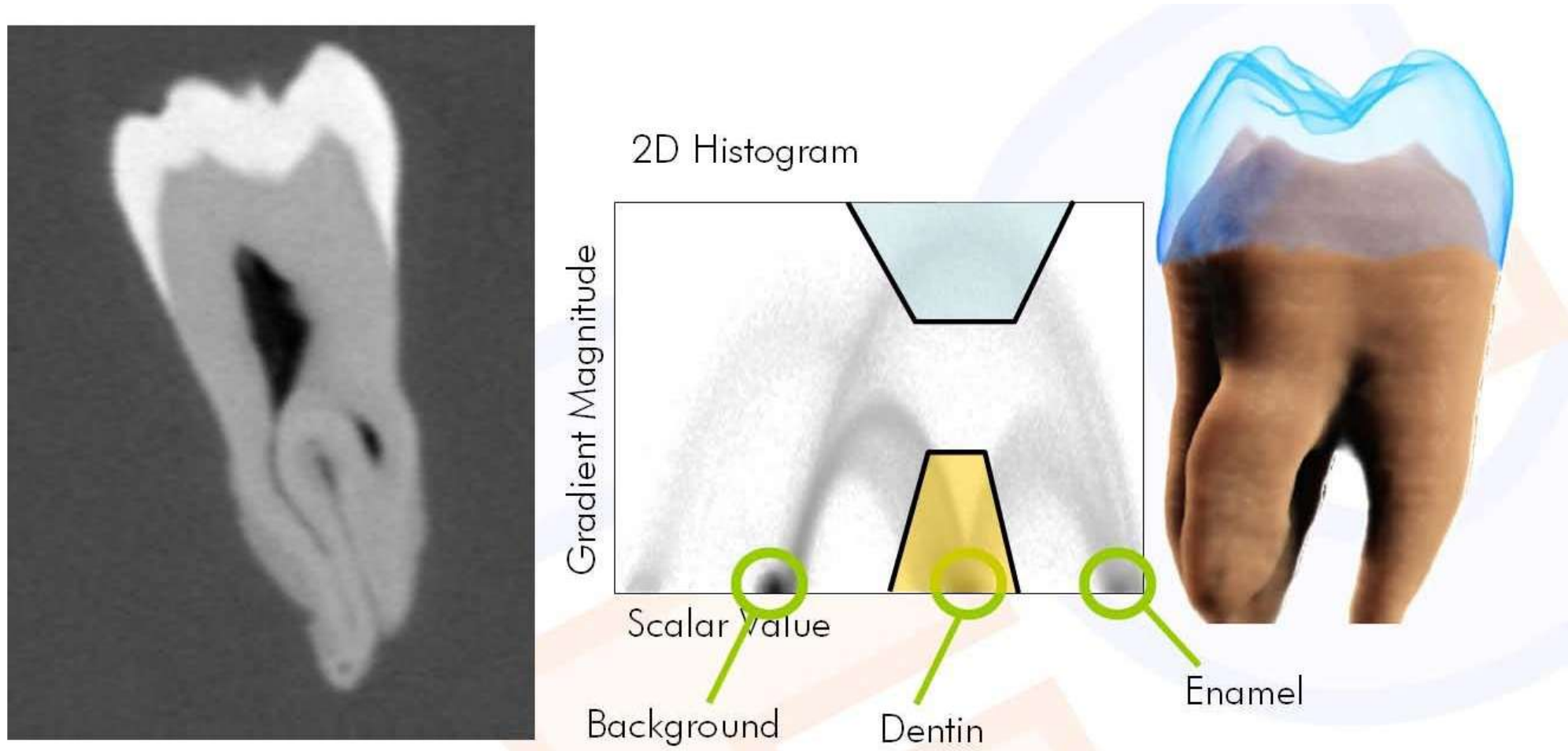
Material boundaries:

D: air – soft tissue
E: soft tissue – bone
F: air – bone



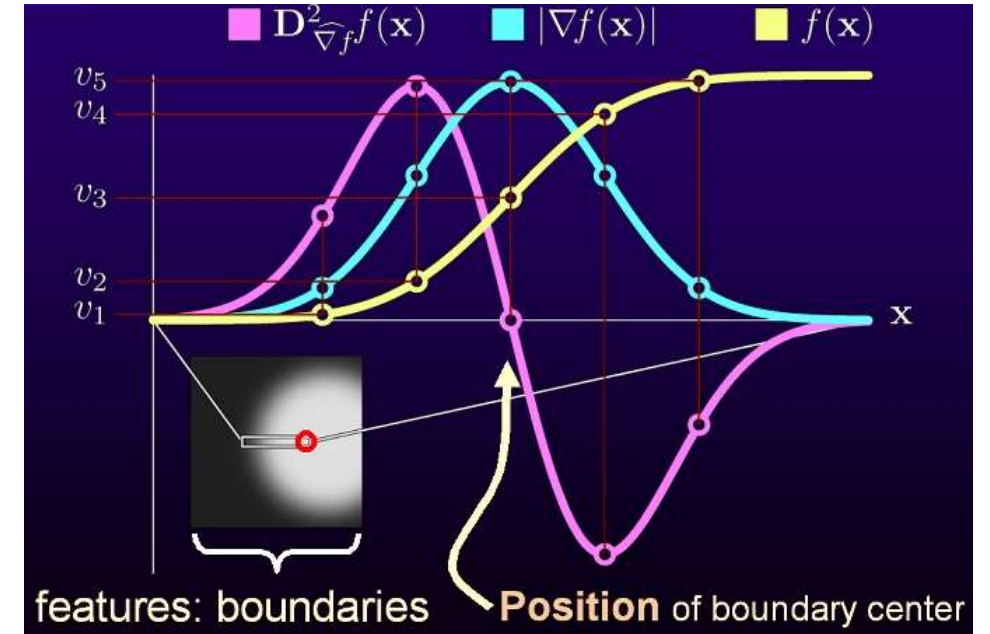
Visualization using 2D transfer function

Example: Tooth CT Dataset 2D



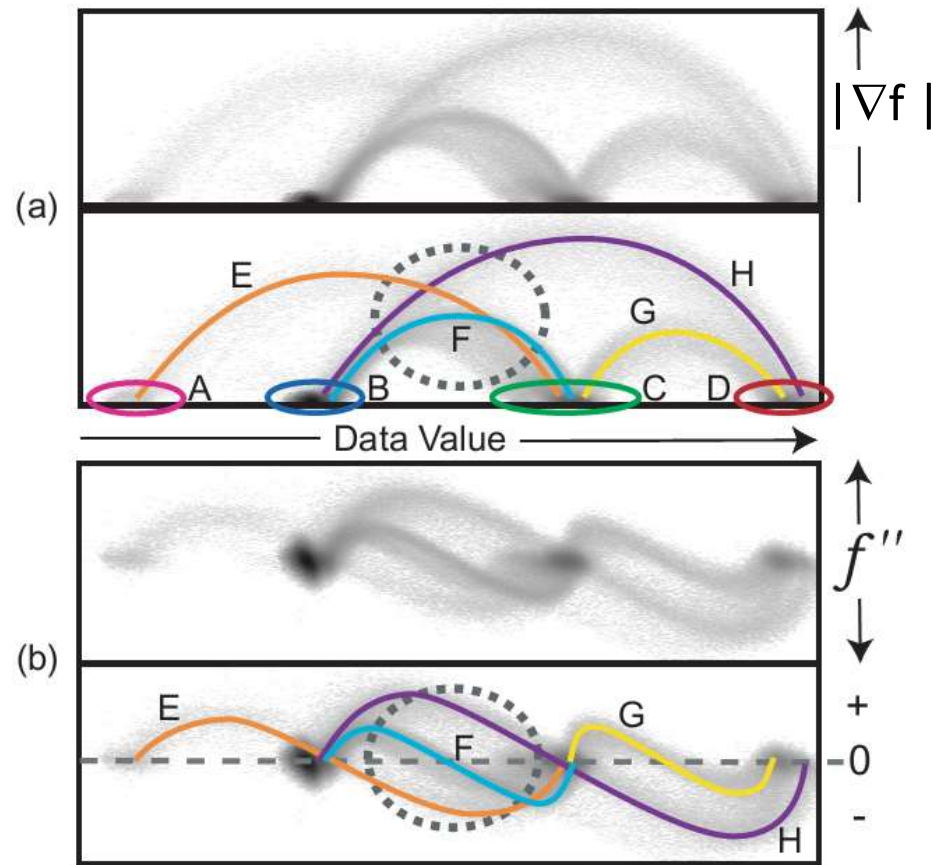
The Value of Second Derivatives

- **Gradient magnitude** can be large in a rather wide region around an edge
 - Arcs from different boundaries can overlap
- Zero crossings in the **second derivative** localize the boundary much more exactly
 - In 3D, we use the directional derivative in gradient direction



3D Transfer Functions

3D transfer functions by [Kniss, Kindlmann, Hansen 01]:

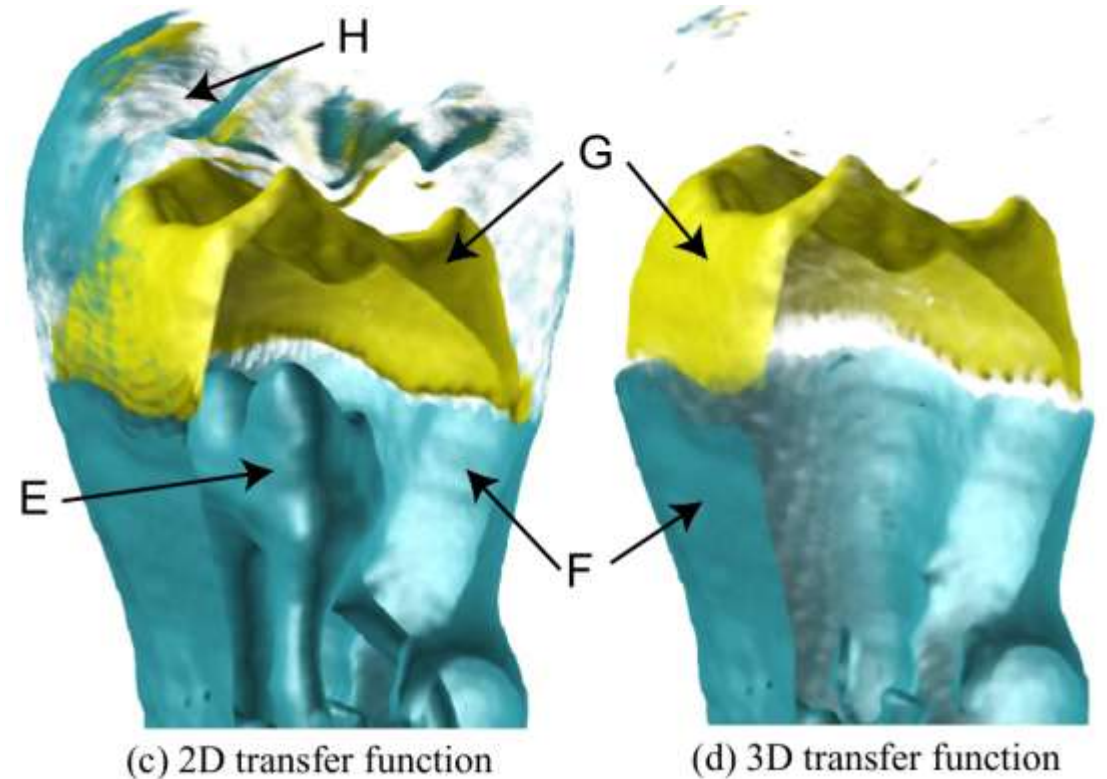


Materials:

A: pulp B: background C: dentin D: enamel

Material boundaries:

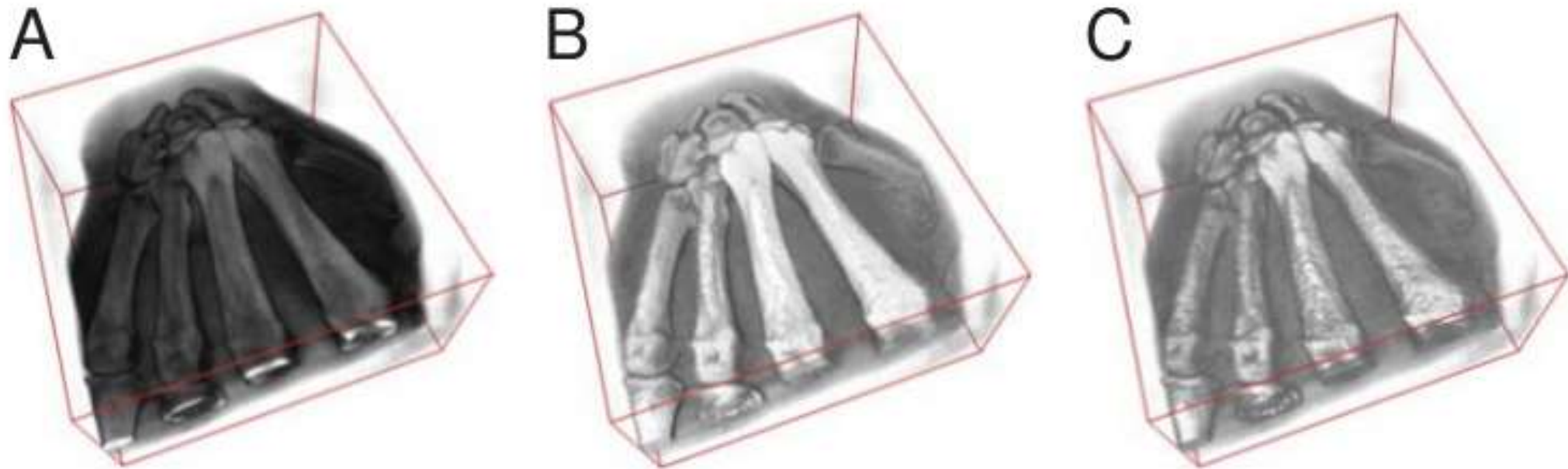
E: dentin – pulp F: background – dentin G: dentin – enamel H: background – enamel



Only using a 3D transfer function, F and G can be separated clearly from other material boundaries like E and H.

Need for Illumination in Volume Rendering

Values from Transfer Function are usually modulated according to a **shading model** such as Phong shading (Chapter 9.5)



A: Pure emission + absorption, no illumination

B: Same with diffuse lighting

C: Same with specular lighting

Transfer Functions: Two-Stage Definition

According to Drebin et al. (1988), we can define transfer functions as the *composition of two functions*:

1. Map data value to probabilities of the presence of any of n materials at this point:

$$p: \mathbb{R}(\times \mathbb{R}(\times \mathbb{R})) \rightarrow [0,1]^n, \text{ often with } \sum_{i=1}^n p_i = 1$$

2. Map the probabilities to color and opacity by assigning a color C_i and an opacity α_i to each material:

$$\tilde{C}: [0,1]^n \rightarrow [0,1]^3$$

$$\alpha: [0,1]^n \rightarrow [0,1]$$

$$\text{Often: } \tilde{C} = \sum_{i=1}^n p_i \tilde{C}_i, \alpha = \sum_{i=1}^n p_i \alpha_i$$

This enables *pre-segmentation* and *visualization of uncertainty*.

Associated / Pre-Multiplied Colors

Averaging uses **associated colors** $\tilde{C}_i := \alpha_i C_i$, which “pre-multiply” straight color C_i with the associated opacity α_i .

- *Benefit:* Avoids leakage of color information from transparent regions during interpolation
- *Example:* What should be the color and opacity halfway between $C_a = [1,0,0]$, $\alpha_a = 1$ and $C_b = [0,1,1]$, $\alpha_b = 0$?

Straight color interpolation:

$$C = \frac{1}{2}(C_a + C_b) = [0.5, 0.5, 0.5]$$
$$\alpha = \frac{1}{2}(\alpha_a + \alpha_b) = 0.5$$

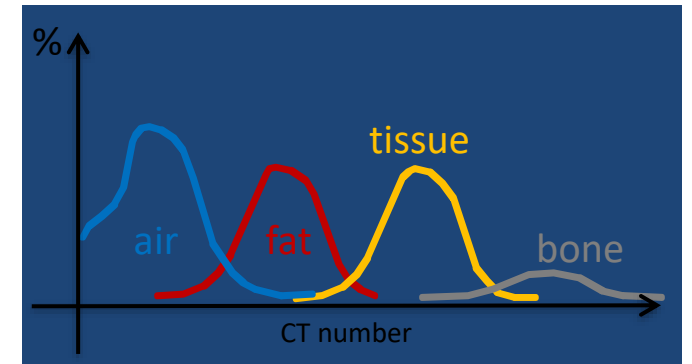
Associated color interpolation:

$$\tilde{C}_a = [1, 0, 0], \tilde{C}_b = [0, 0, 0]$$
$$\tilde{C} = \frac{1}{2}(\tilde{C}_a + \tilde{C}_b) = [0.5, 0, 0]$$
$$\alpha = 0.5, C = \frac{\tilde{C}}{\alpha} = [1, 0, 0]$$

Pre-Segmentation

Problem: Some structures are impossible to distinguish even with high-dimensional transfer functions

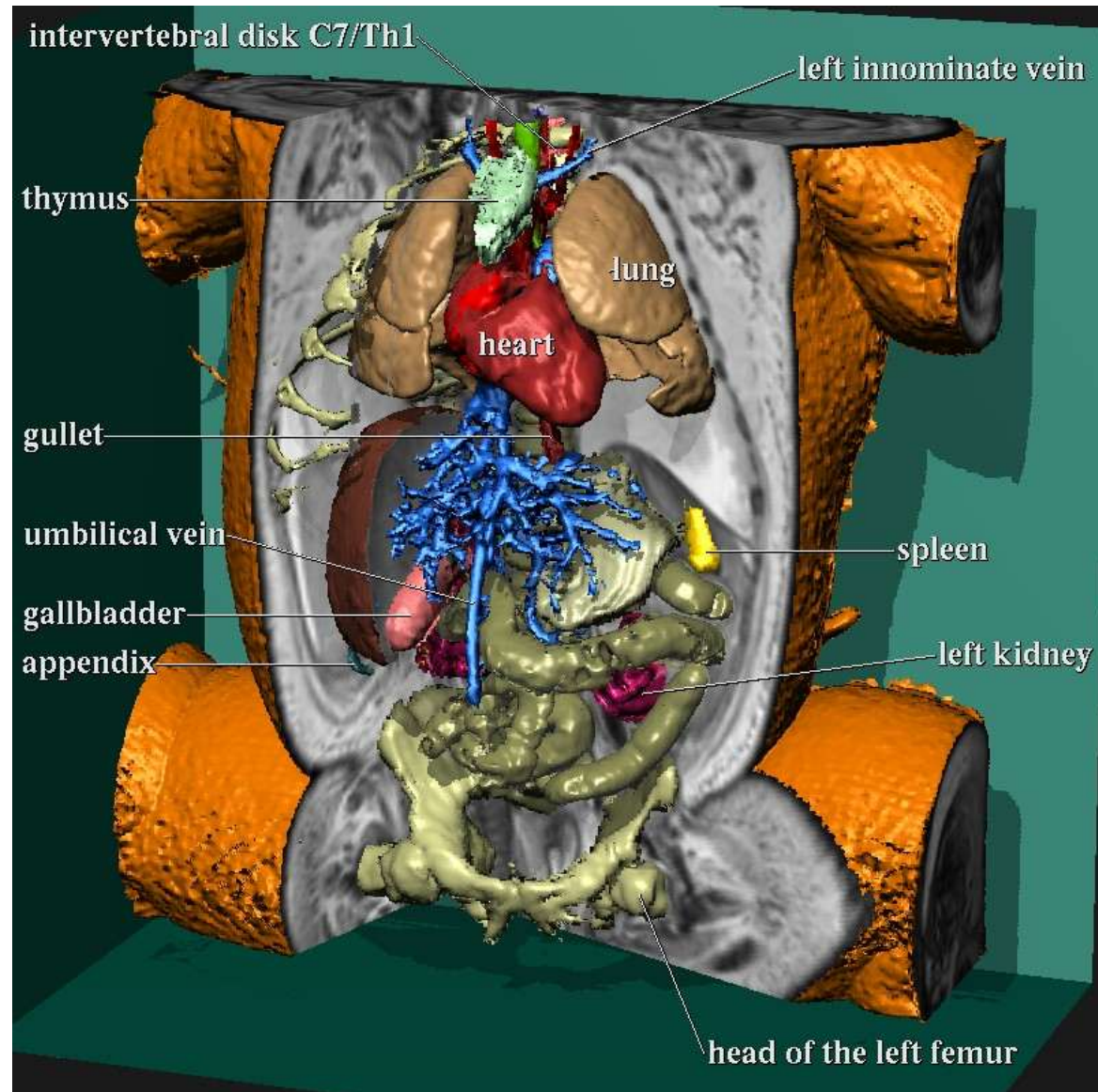
- *Example CT:* different organs have similar Hounsfield values



Approach: In **pre-segmentation**, assignment of voxels to classes (i.e., the first step of the transfer function) is done manually or by a specialized, (semi-) automated segmentation algorithm before visualizing the data

- Can better account for spatial continuity
- Can account for prior knowledge

Example Pre-Segmentation

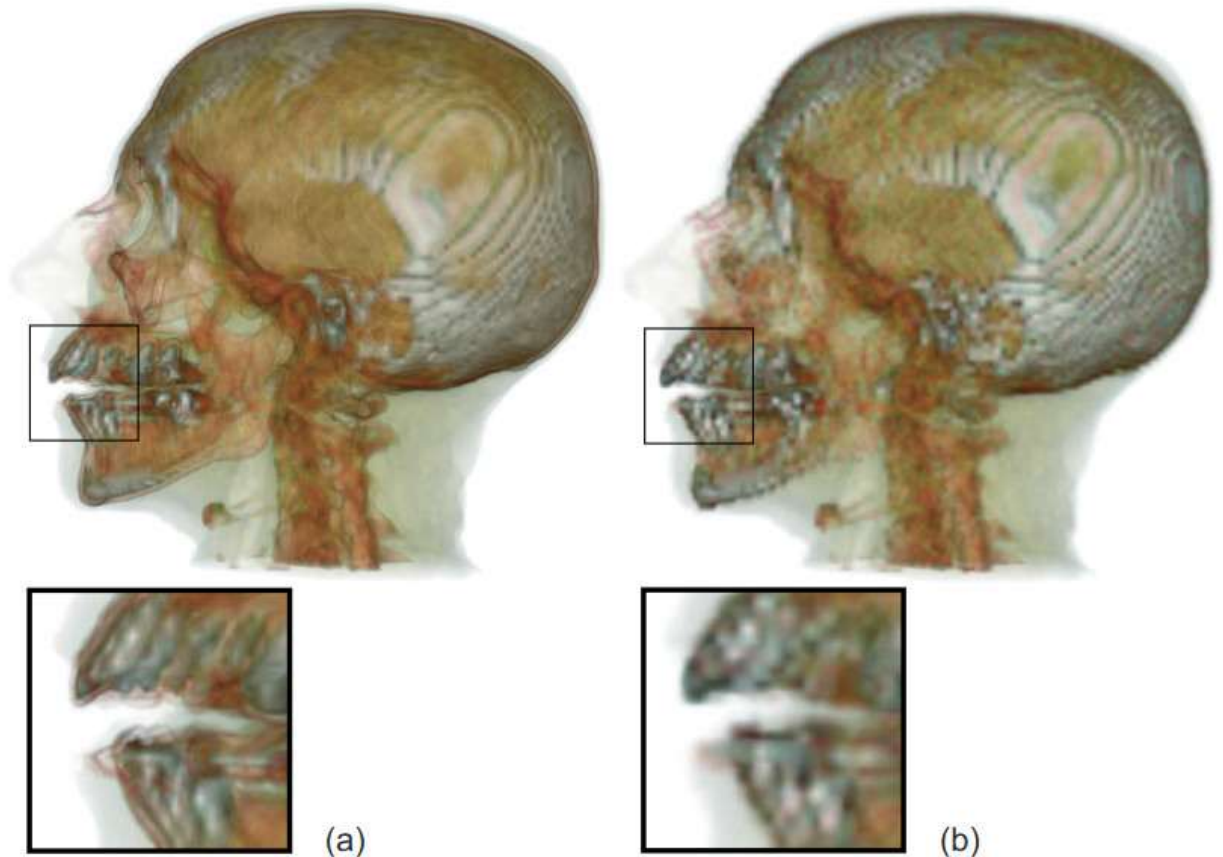


Anatomic atlas

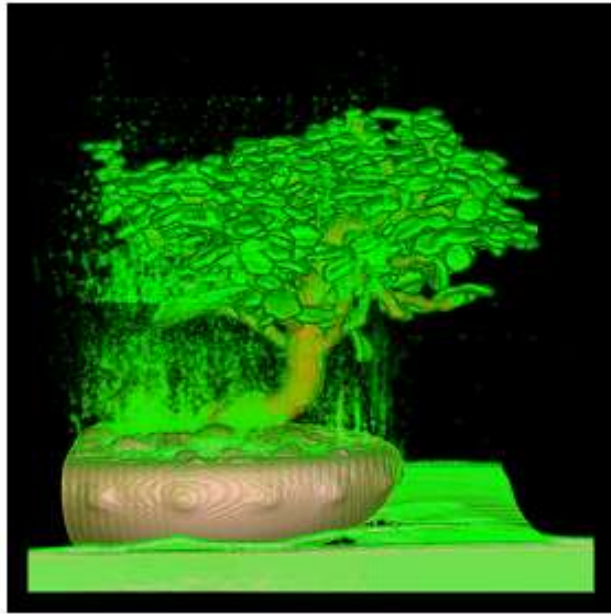
Pre- vs. Post-Classification

Terminology caveat: Pre-segmentation (i.e., running a segmentation algorithm before visualization) is not the same as

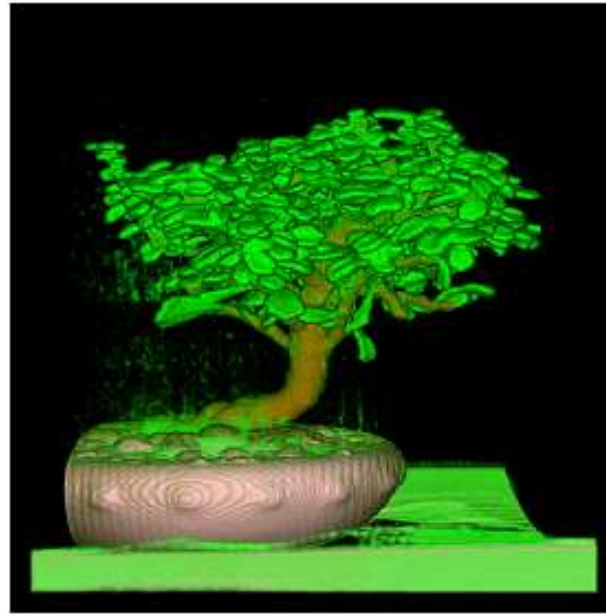
- **Pre-classification (b):** Transfer function is applied to the sampled data values, resulting colors are interpolated
- **Post-classification (a):** Data values and derivatives are interpolated, the transfer function is applied to the resulting value
 - Slower, but can provide finer detail



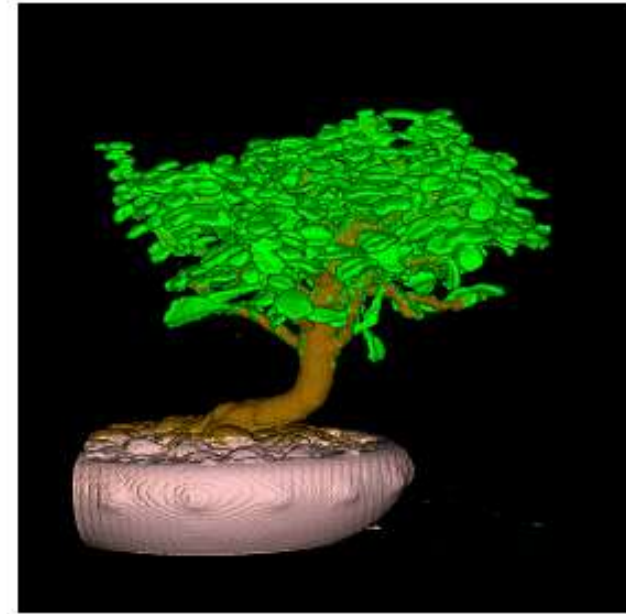
Weighting Different Classes



(a) Probability weighted



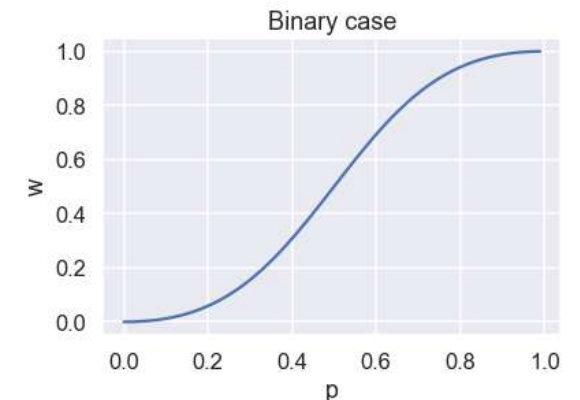
(b) Risk-based with equal weights



(c) Most probable material

Kniss et al. [2005]: Risk-based weights

$$w_i \sim \frac{1}{\lambda_i(x)} \text{ with „risk ratio“ } \lambda_i(x) = \max_{j \neq i} \frac{p_j}{p_i}$$

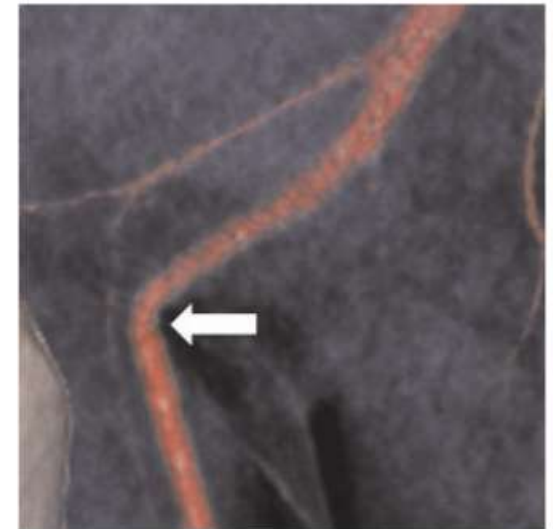
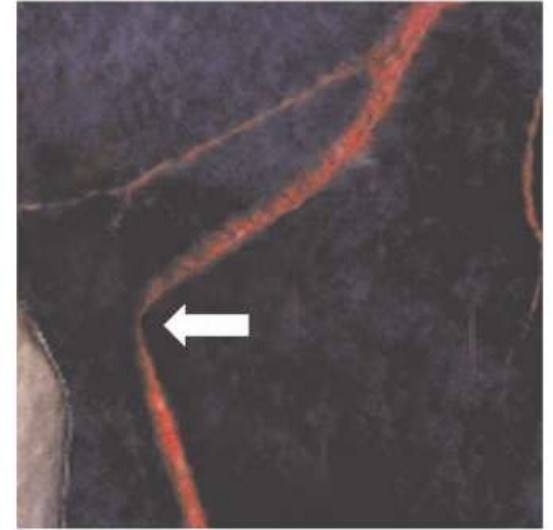


Probabilistic Animation

Problem: If multiple materials have significant probability, mixing the colors can lead to false conclusions.

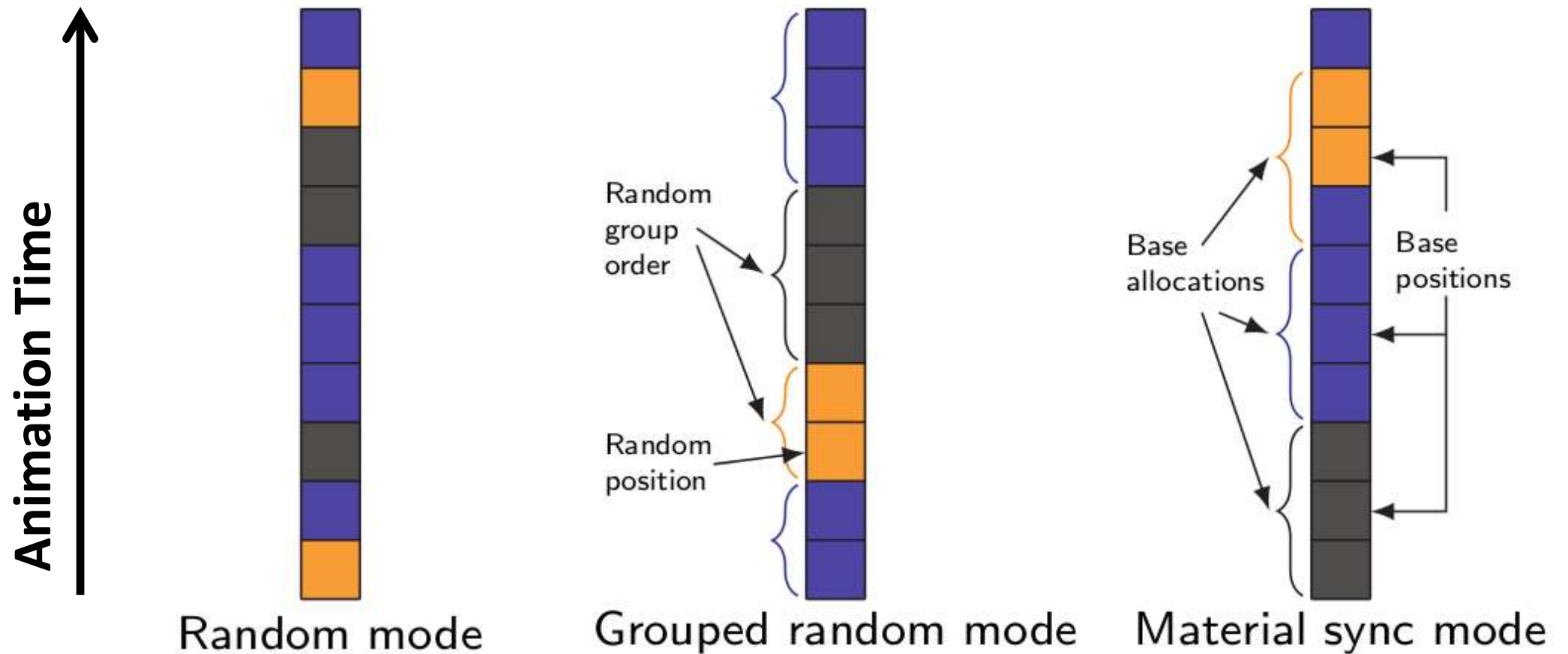
Probabilistic Animation [Lundström et al. 2007]:

- Do not average colors. Instead, produce an animation in which each material is shown for a time proportional to its probability
- Visually emphasizes multiple interpretations of the data



Ordering in Probabilistic Animation

To avoid flickering, put materials into a particular order during the animation:



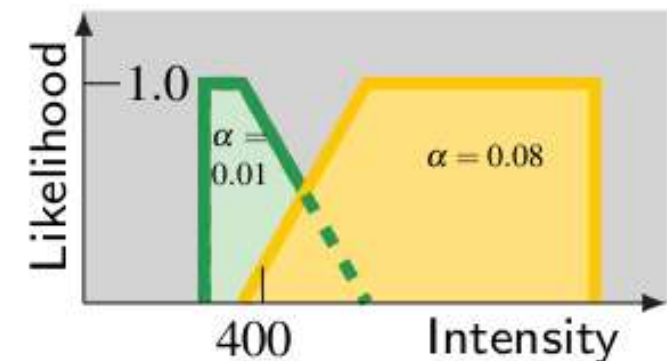
Example: Probabilistic Animation

Orange square acts as a “sensitivity lens”



MR Renal Angiography

Example from [Lundström et al. 2007]



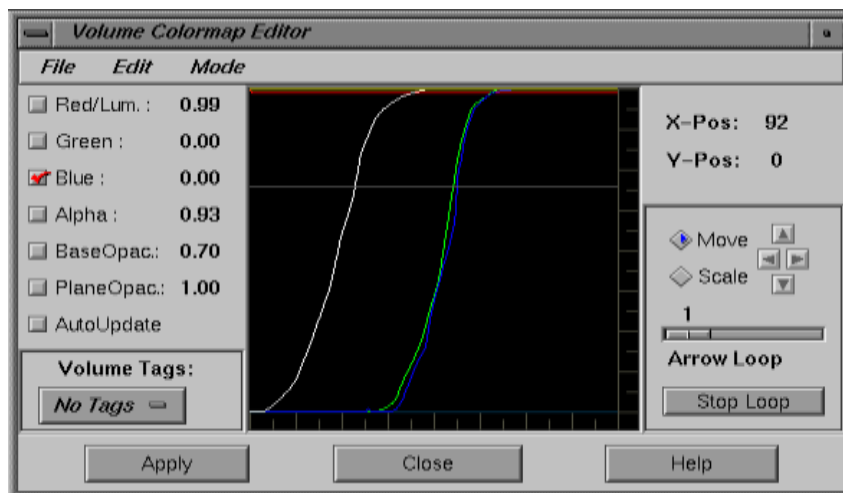
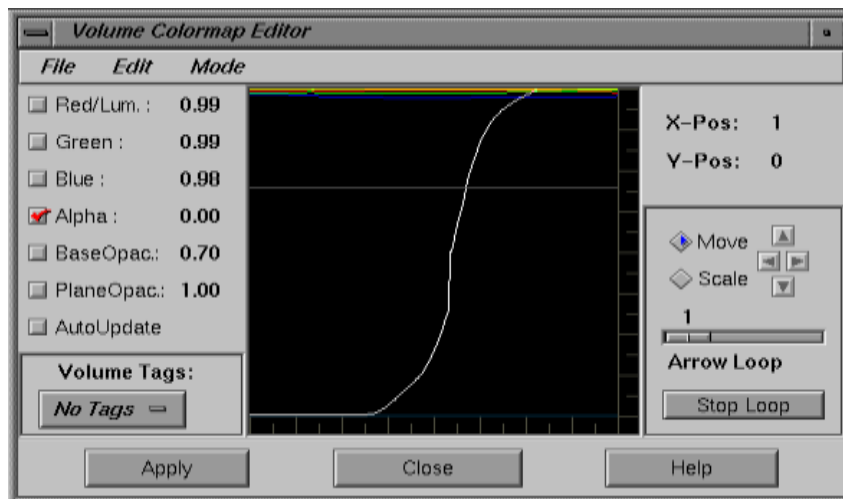
Summary: Transfer Functions

- **Transfer Functions** map data values to optical properties such as color and opacity
 - Commonly used features include
 - Scalar value (1D)
 - Gradient magnitude (2D)
 - Second derivative in gradient direction (3D)
 - Two-step view: Classify materials or objects, then assign color / opacity to each class
 - Can be used to render pre-segmented volumes
 - Different weighting schemes can be applied
 - Can be used for probabilistic animation

Section 10.2:

Transfer Function Design

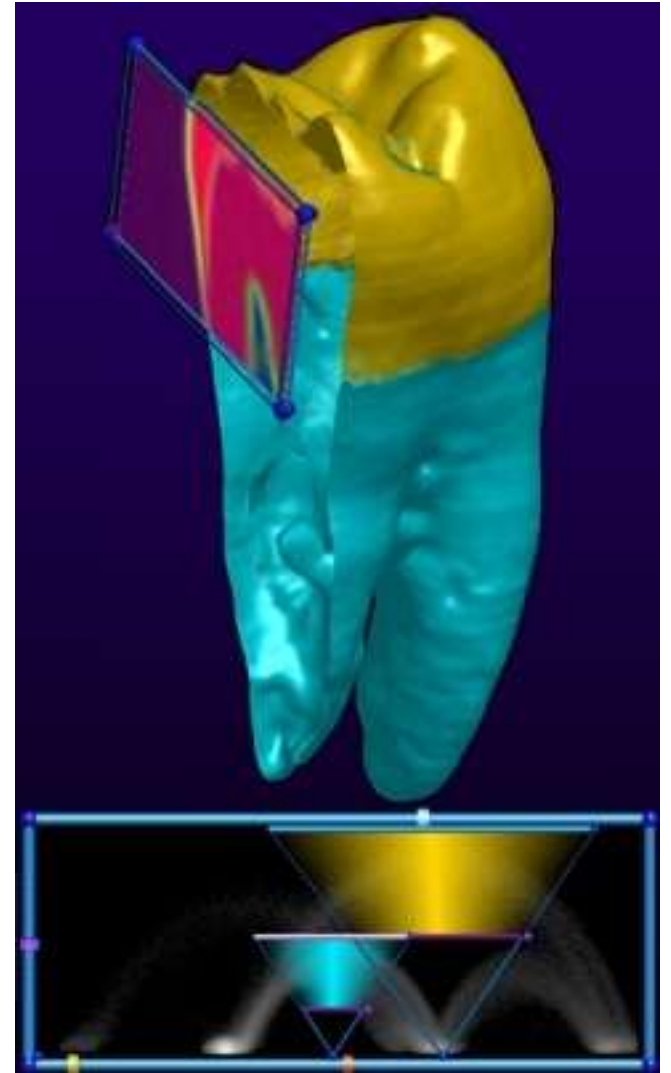
Basic 1D Transfer Function Design



2D Transfer Function Design

Triangular classification widget in 2D histogram:

- Upper and lower threshold for gradient magnitude
- Maximum opacity along center line
- Opacity falling off towards left and right edges of triangle

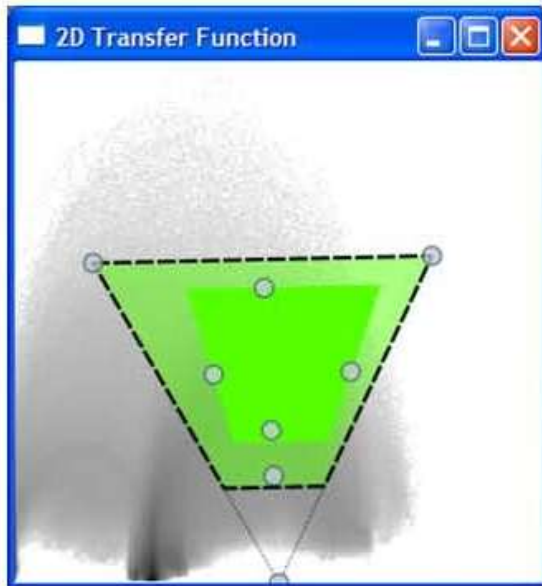


Other 2D Transfer Function Widgets

- Geometric shapes for transfer function design
 - Afford parametric representation of transfer function
 - High-dimensional lookup tables memory inefficient

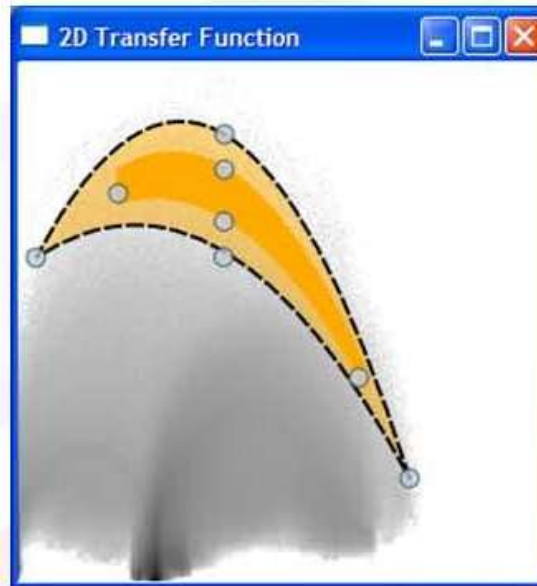
Trapezoids

(Kniss et al. 2001)

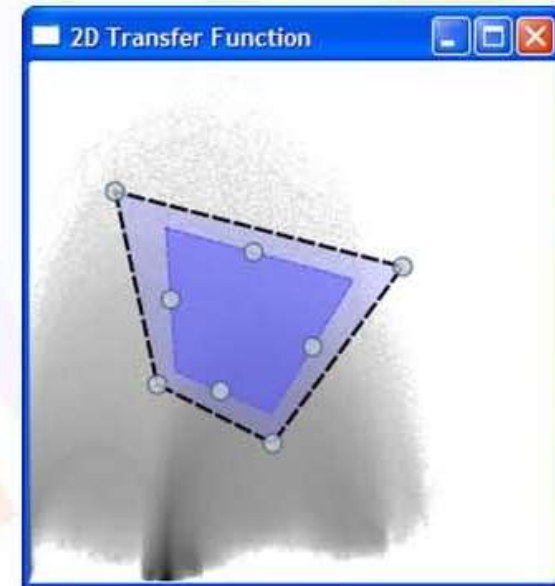


Paraboloids

(Vega et al. 2004)

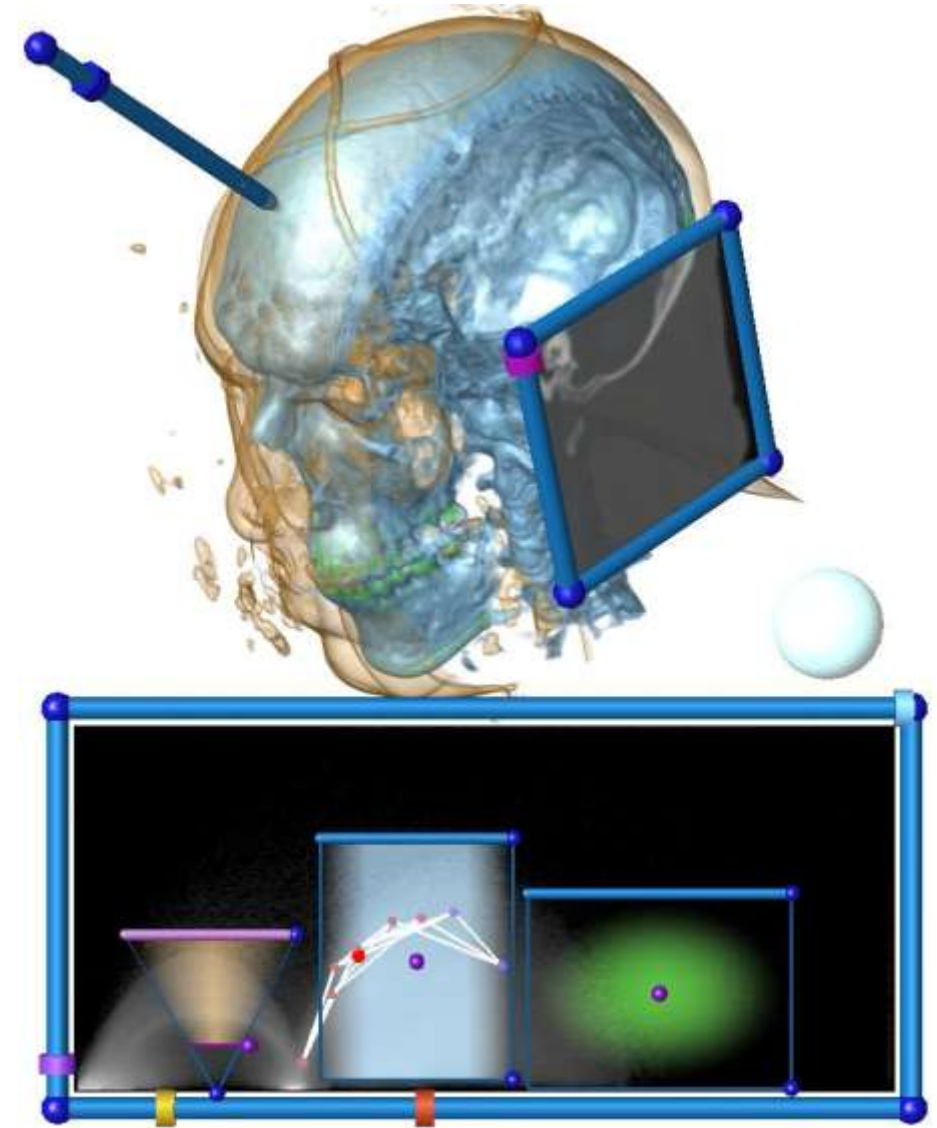


Quadrilaterals



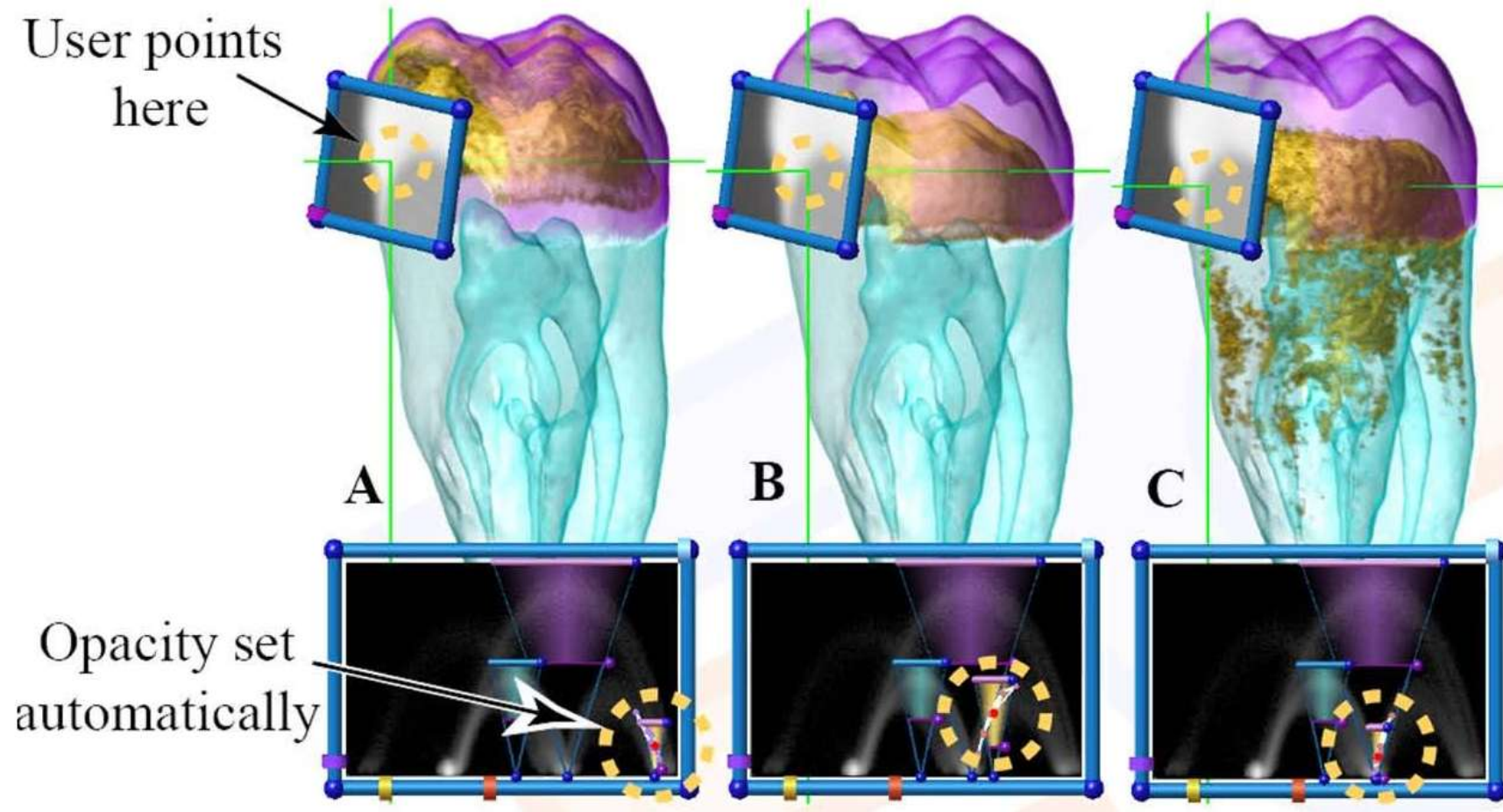
Direct Manipulation Widgets

- **Direct Manipulation Widgets** allow user to better understand the link between
 - Spatial Domain and
 - Transfer Function Domain
- **Examples:**
 - Data Probe
 - Clipping Plane



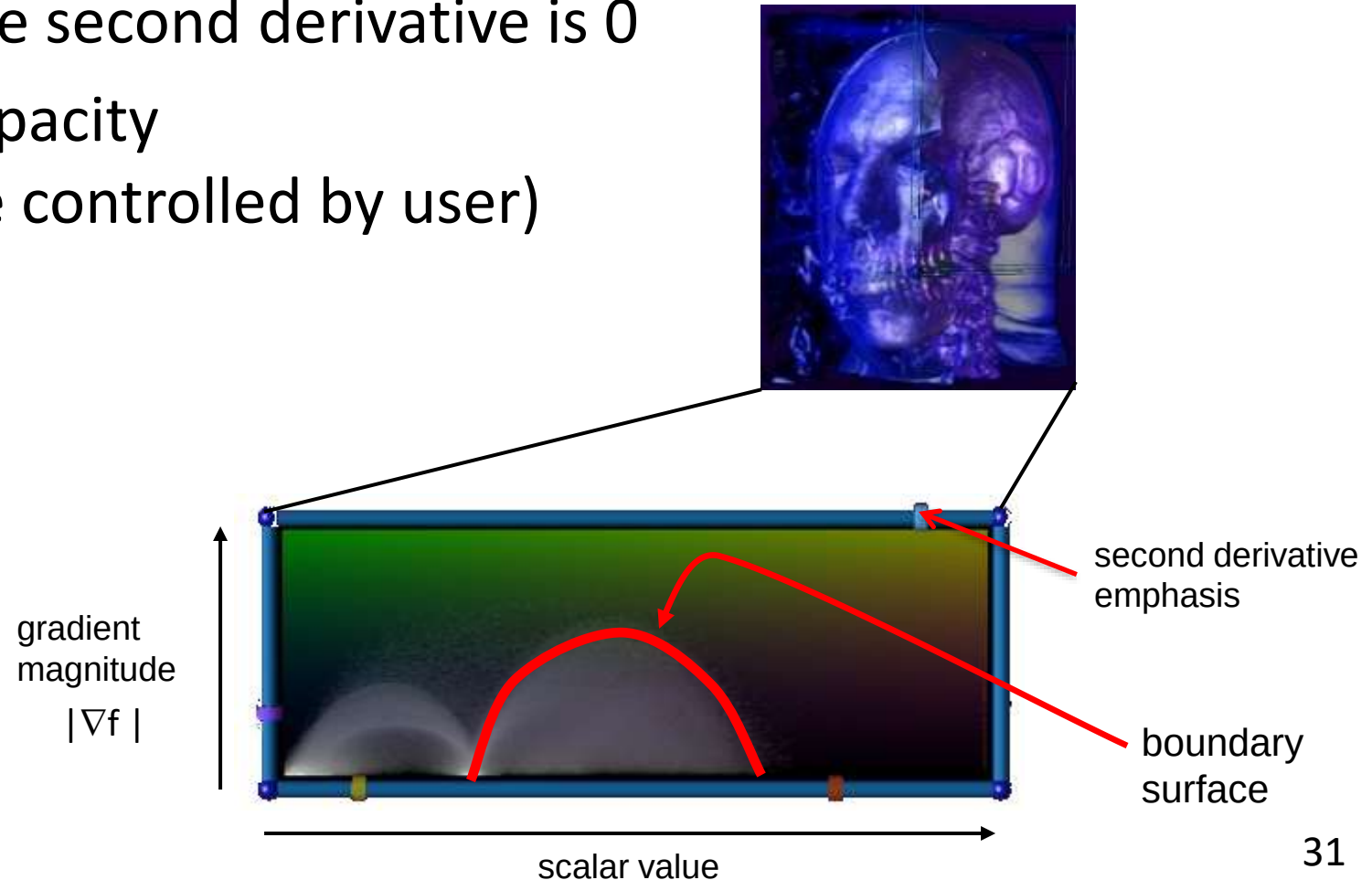
Dual-Domain Interaction

- **Dual-Domain Interaction**
 - directly modifies transfer function by interactions in spatial domain



3D Transfer Function Design

- Primarily based on 2D Histogram (as before)
- Dependence on second derivative:
 - Maximum opacity where second derivative is 0
 - Otherwise decreasing opacity (rate of decrease can be controlled by user)



Default Transfer Function

- Simple **3D Transfer Function** for a quick initial impression of an unknown volume:
 - High opacity at high gradient magnitude and near zero crossings in second derivative
 - Spectral color map to encode values
 - Can serve as a starting point for interaction



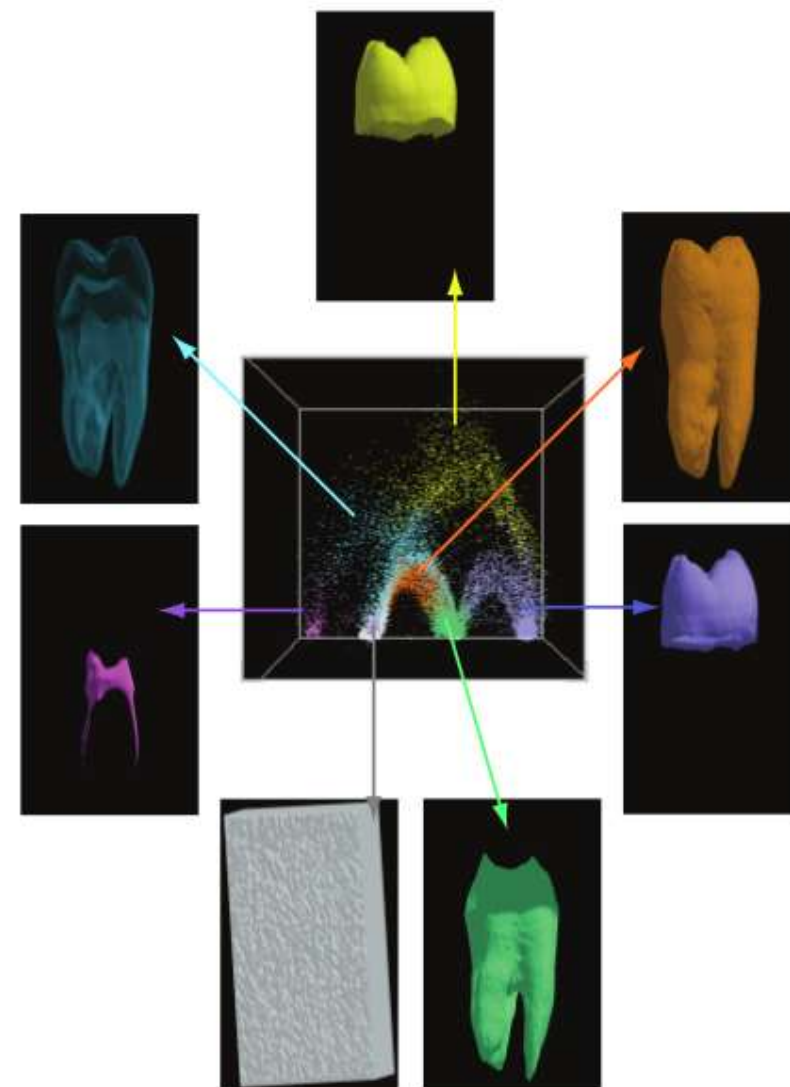
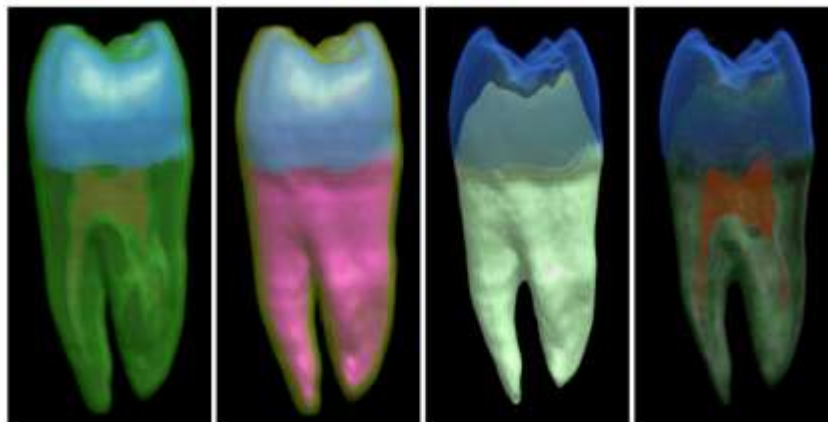
Cluster-based TF Design

Problem: As we include more dimensions (size, texture...), interactive specification of transfer functions becomes challenging

Clustering-based Approach [Tzeng & Ma 04]:

- Classification performed by clustering voxels in Transfer Function Domain
- Color and opacity can be specified per cluster

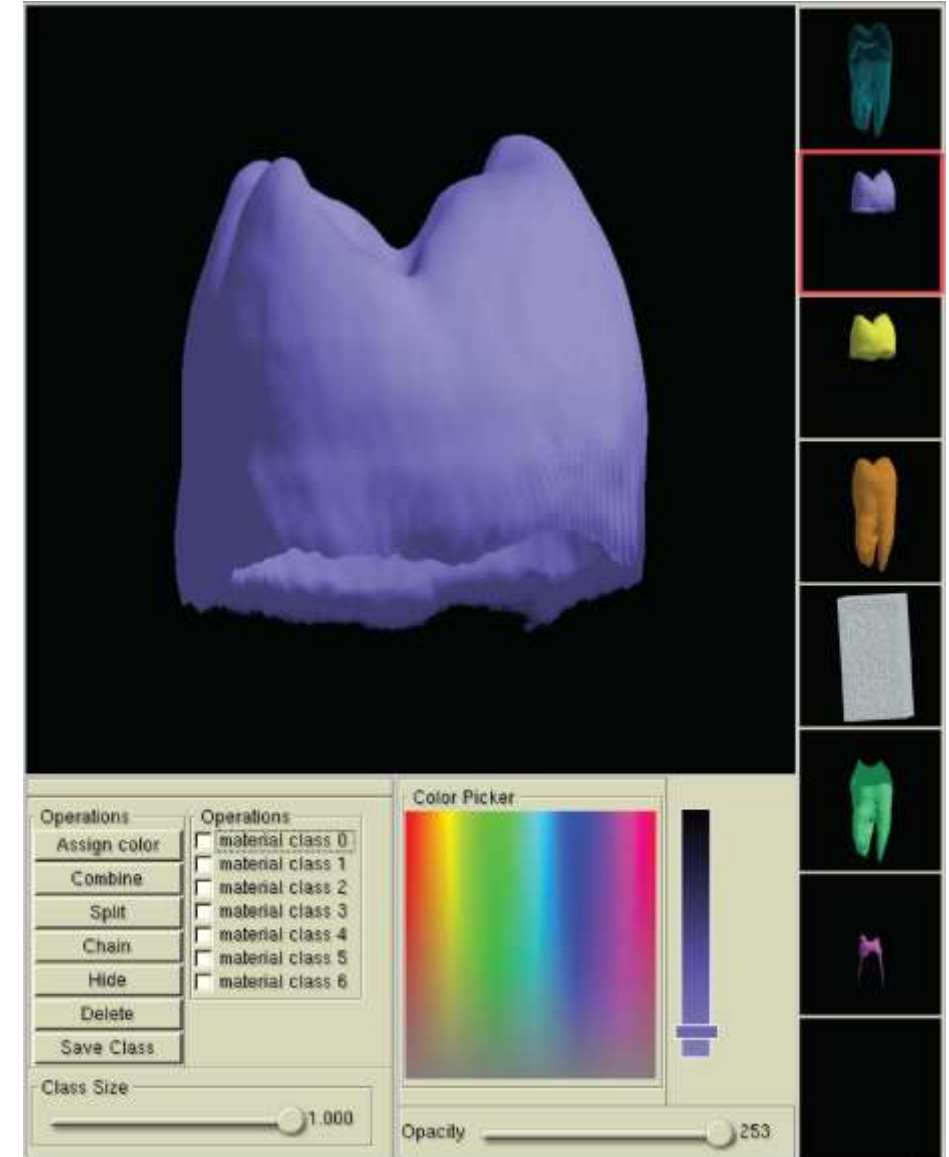
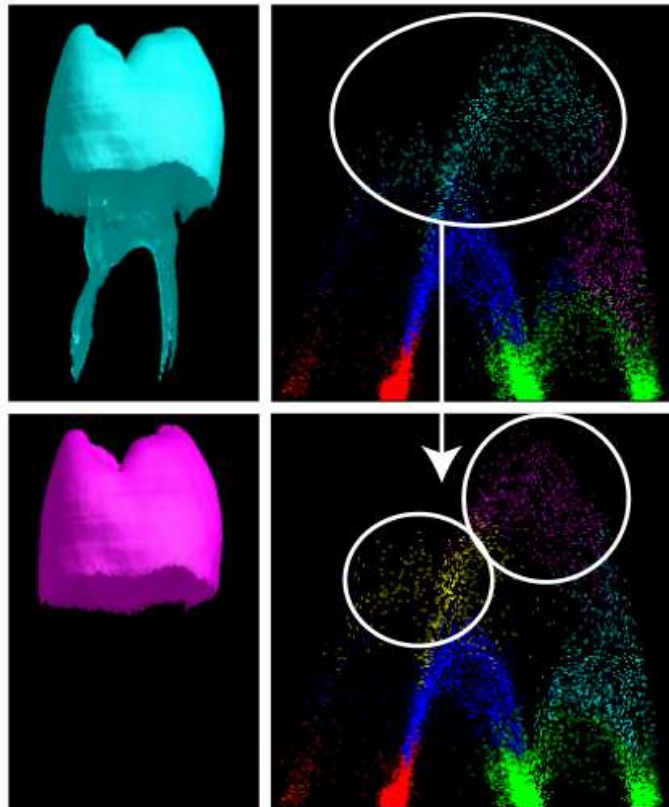
• *Example results:*



Correspondence Cluster - Material

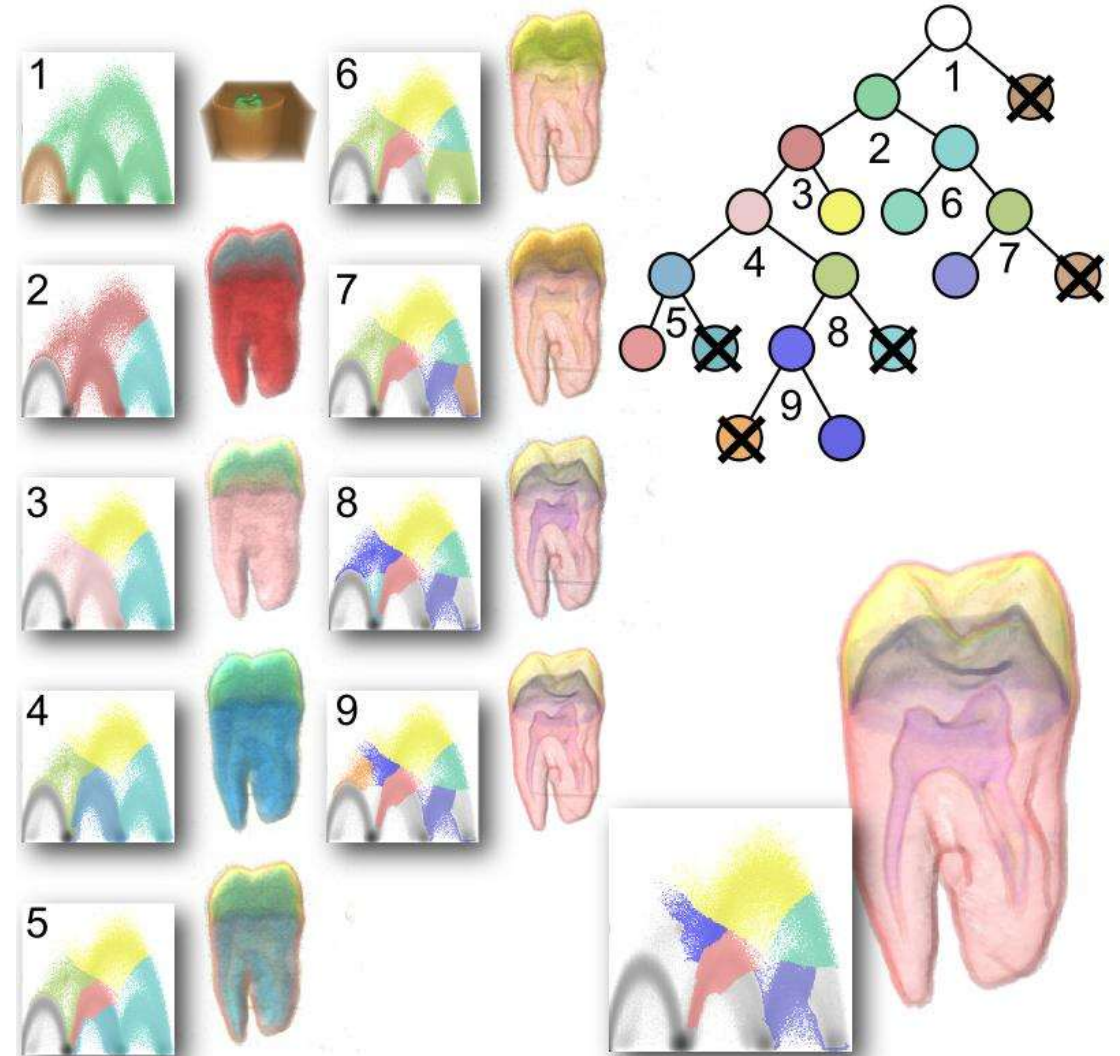
User Interface for Cluster-based TF Design

- In addition to assigning optical properties, the user interface supports merging and splitting



Hierarchical Clustering and Exploration

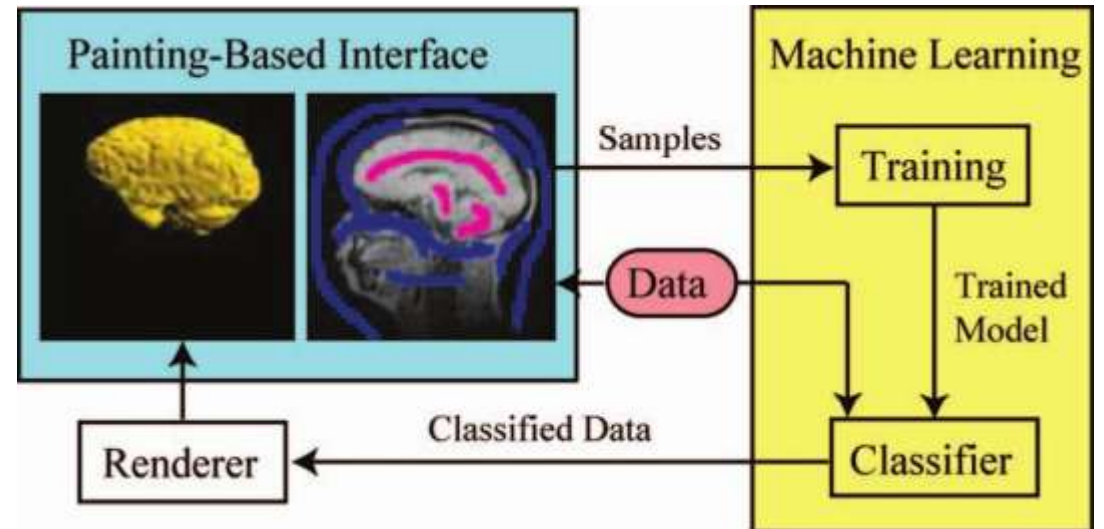
- [Ip et al. 2012] focus on **hierarchical clustering and exploration** of 2D feature space
 - Suggests which nodes might be split based on homogeneity of parent and children (entropy / information gain)



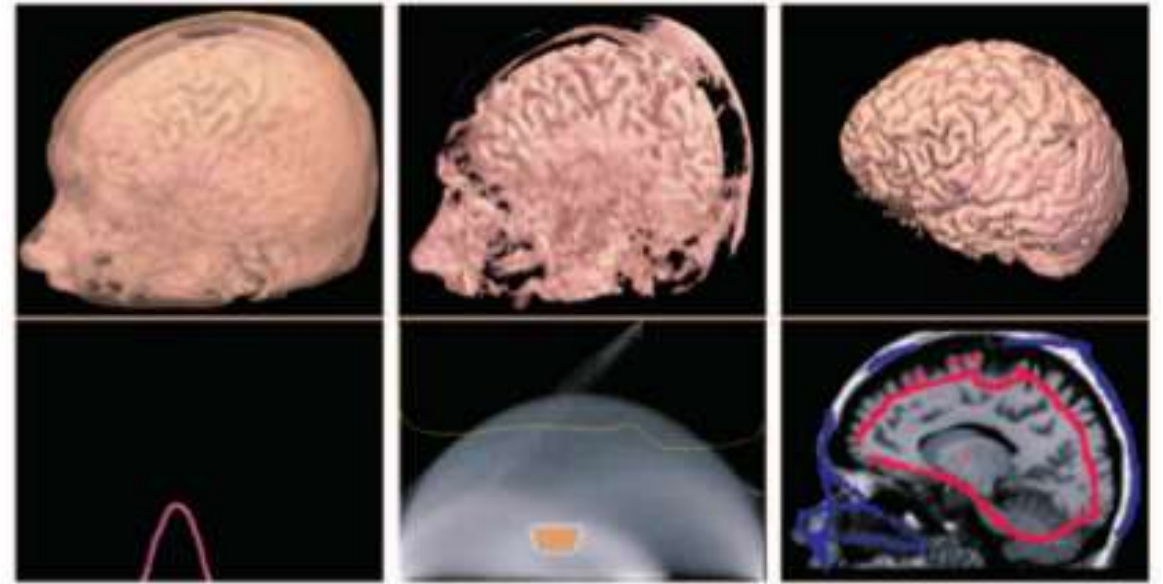
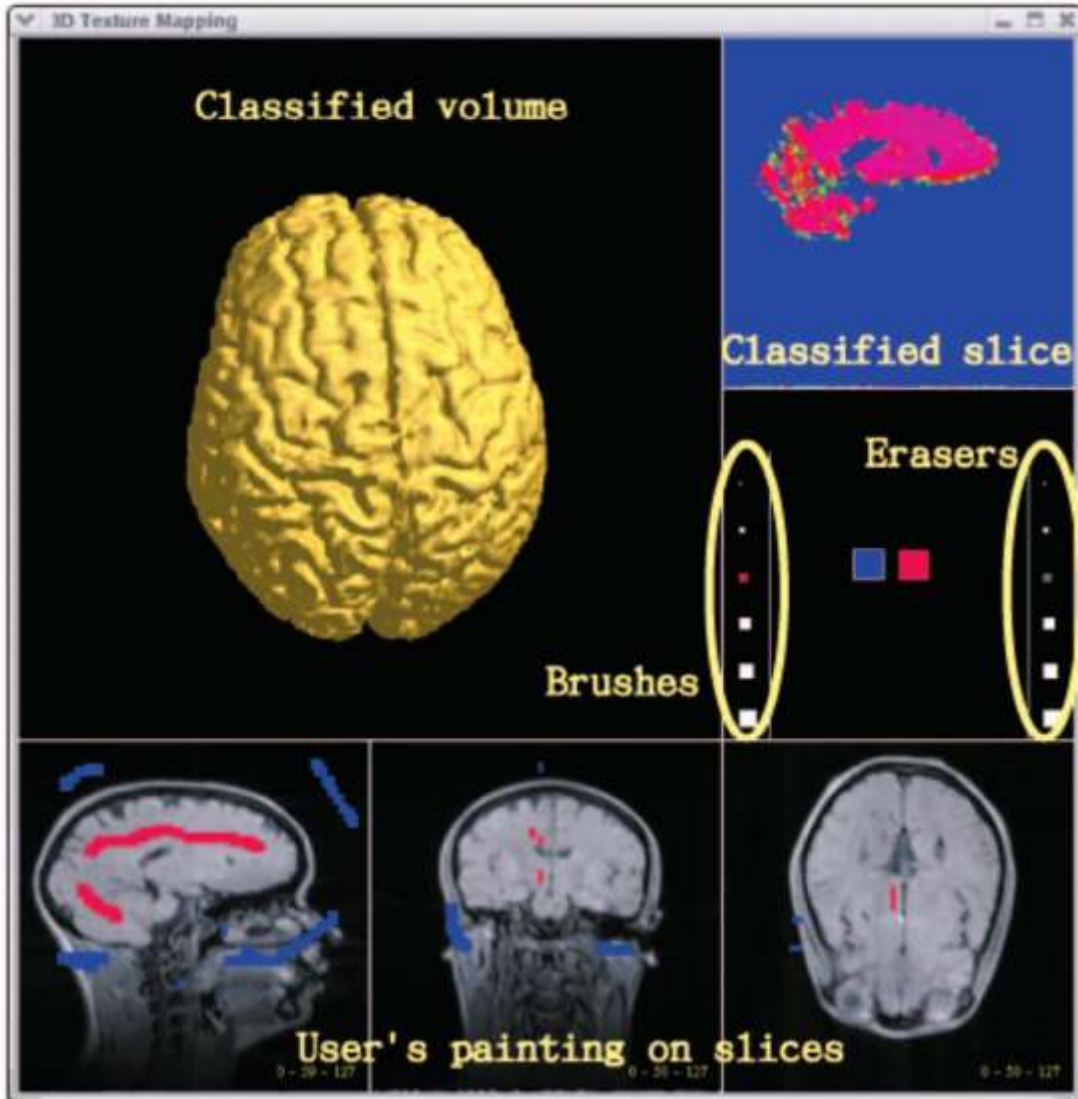
Learning-based TF Design

Supervised learning based approach by [Tzeng et al. 05]:

- User scribbles on a slice visualization
 - Examples of foreground/background voxels
- Transfer function based on a learned classifier
 - Neural Network
 - Support Vector Machine
- Generalizes to high dimension (e.g., 10D)



Learning Approach: UI and Results



1D TF

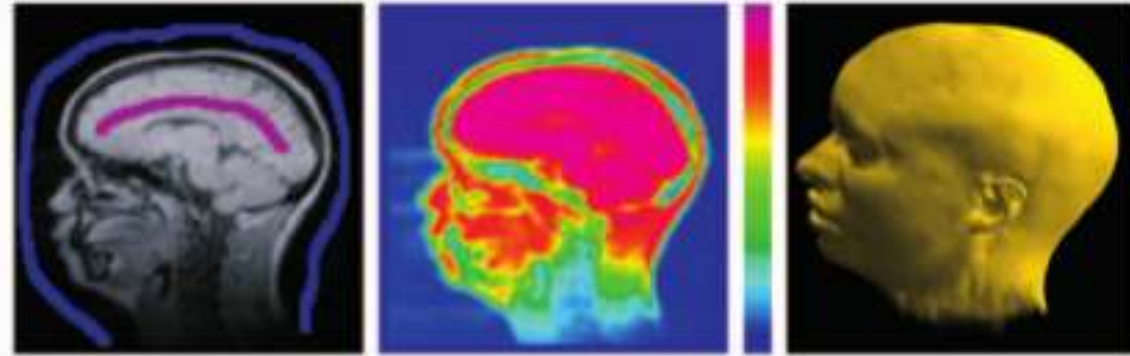
2D TF

10D TF

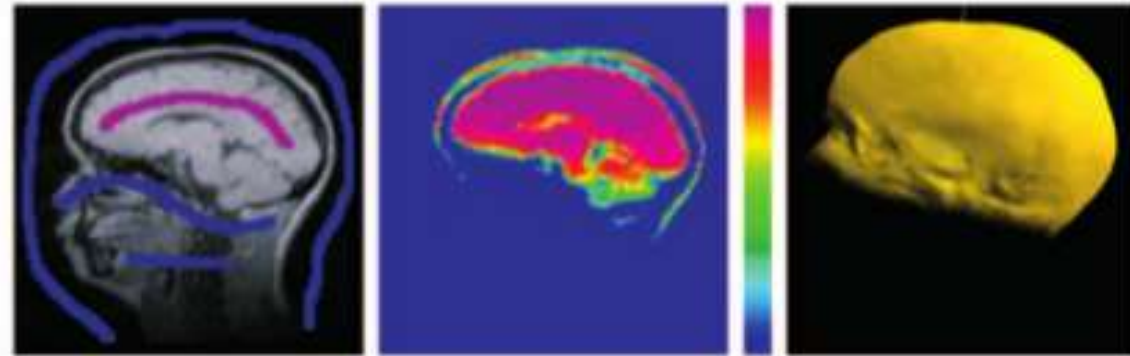
- Features can include:
 - Scalar value
 - Gradient Magnitude
 - Neighboring values
 - Spatial position
 - ...

Iterative Refinement in Learning Approach

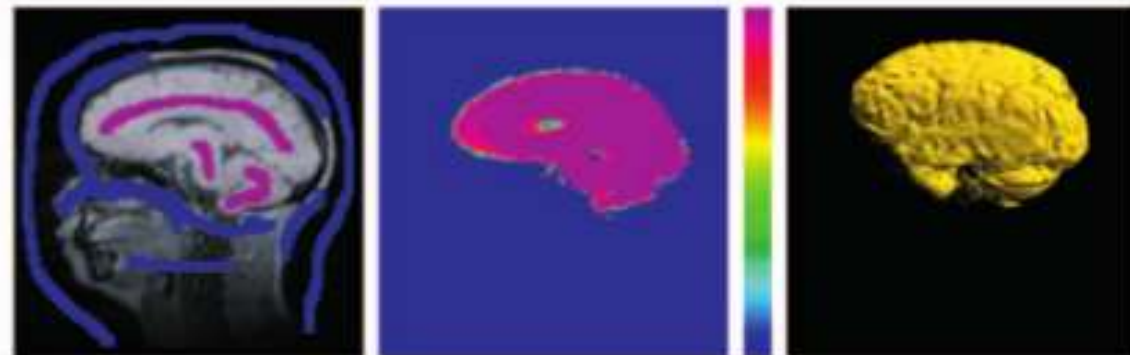
1st Sketch



2nd Sketch



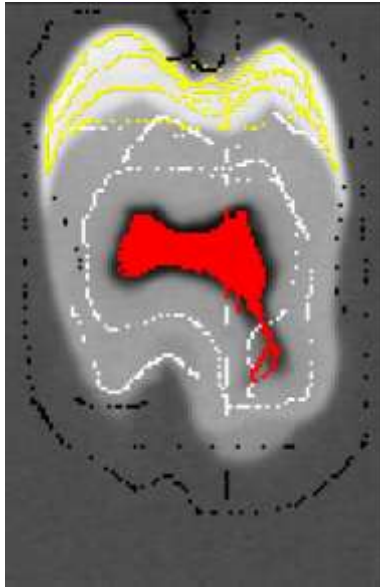
Final Sketch



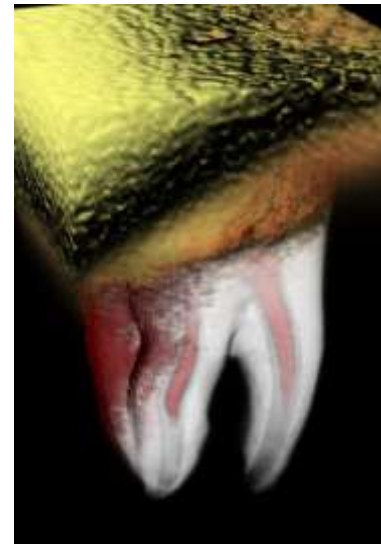
Multiple Classes / Different Classifiers

Soundararajan / Schultz [2015]:

- Generalized to multi-material classification
- Explored different supervised classifiers



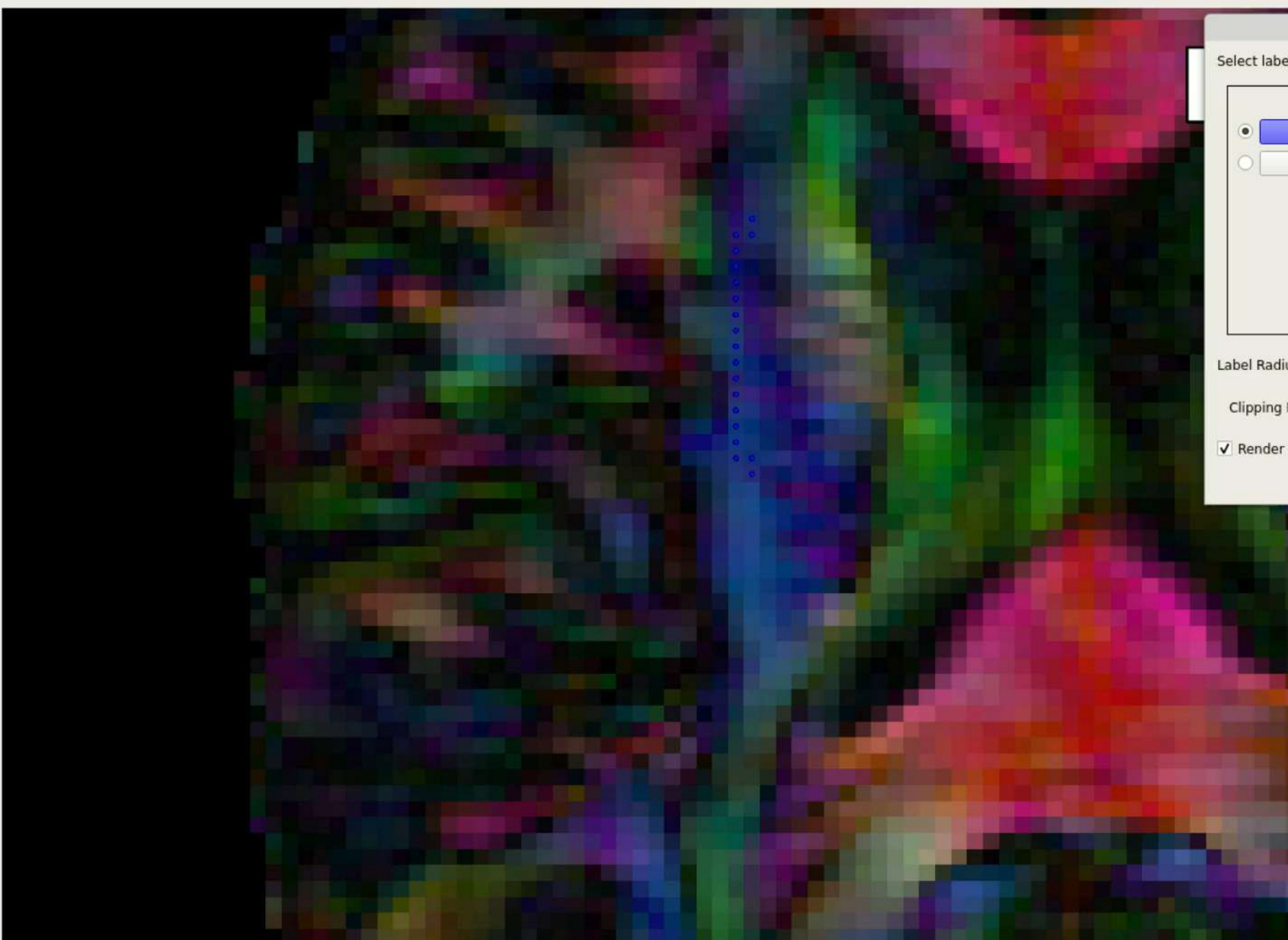
k -NN



SVM



RF



Multi-Label Annotations

Select label to modify, hold Ctrl, add with left button, remove with right button

	Name	Points
☒	CST	19
☐	Other	?

Classify

Select Features

Add Label

Load Labels

Save Labels

Label Radius: 0

Clipping Plane: ☐ None ☐ S+ ☐ S- ☐ C+ ☐ C- ☐ A+ ☒ A-

☒ Render all labels ☐ including background ☐ Show Glyphs

Close

Show P

A 87

3x Speed

Show I

S 71

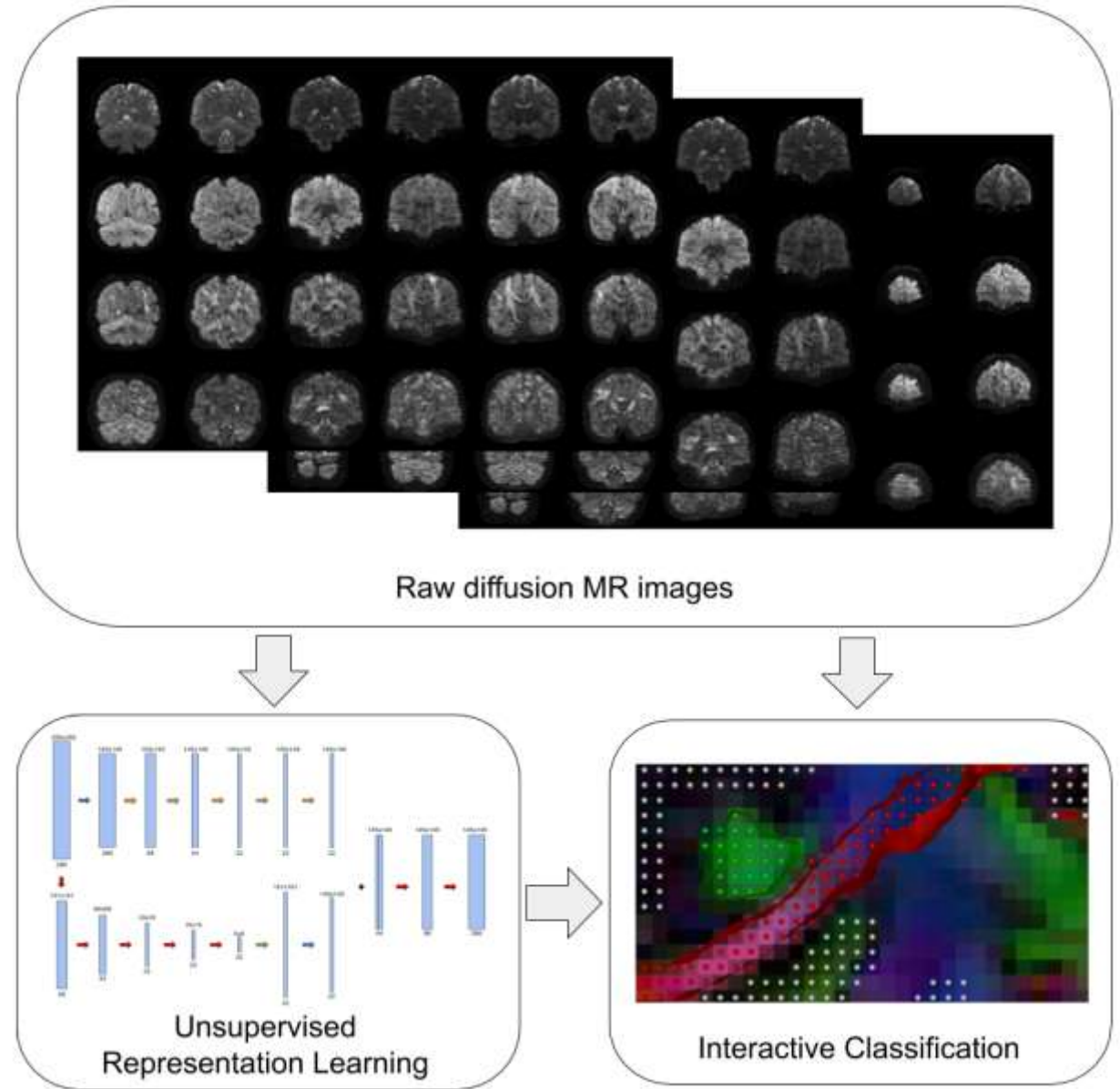
Representation Learning for High-Dimensional Data

[Torayev/Schultz, VCBM 2020]:

Challenge: High-dimensional data (288 values per voxel)

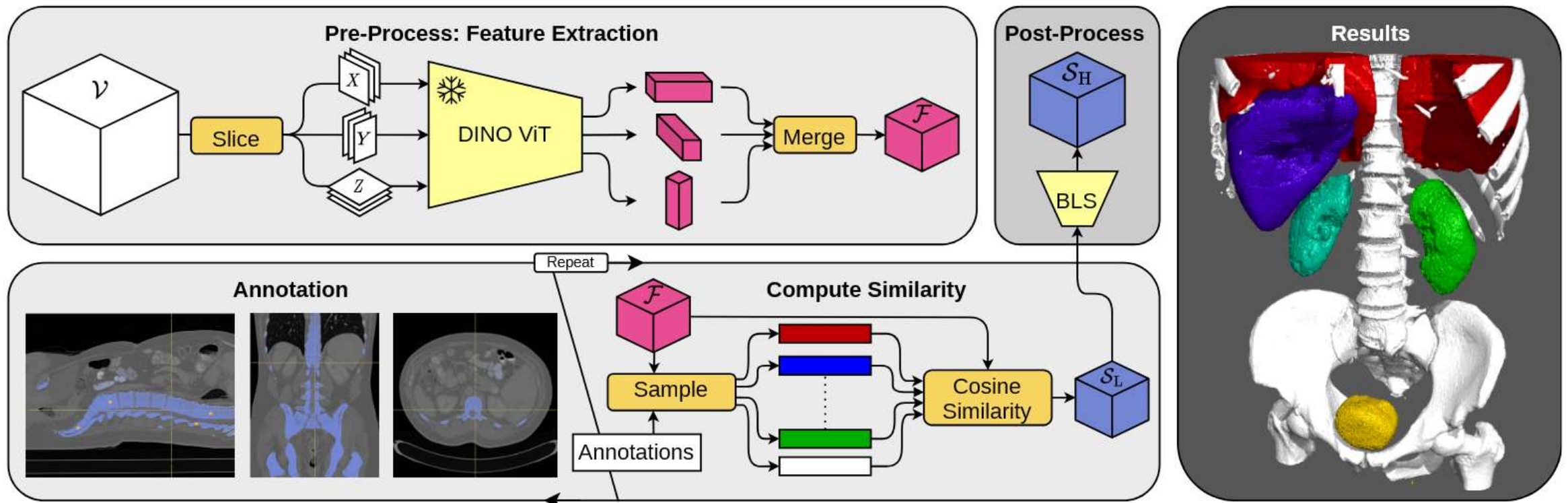
- Dimensionality reduction
- Integrating information from spatial neighborhoods

Proposed solution: Combine a CNN-based autoencoder (offline pre-computation) with a random forest for interactive learning



Foundation Models for Transfer Function Design

- [Engel et al. 2024] learn transfer functions for scalar volumes based on representations from a pre-trained foundation model (DINO)



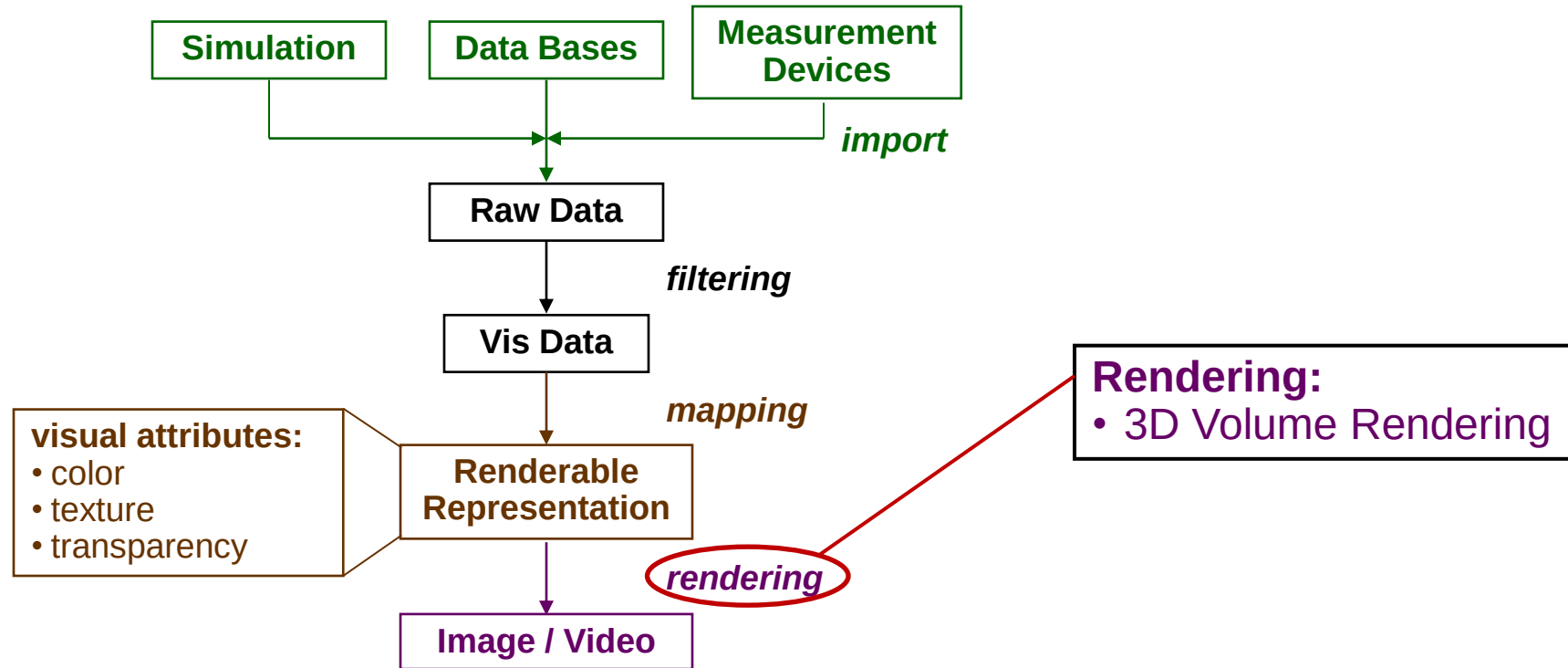
Summary: Transfer Function Design

- **Interactive Transfer Function Design**
 - Most basic: Draw or specify control points
 - Widgets: Trapezoid / Paraboloid / Quadrilateral
 - Boundary enhancement slider
- **Dual-domain interaction** makes feature space easier to understand
- **Clustering** and **supervised learning** provide advanced ways to specify transfer functions while focusing on spatial domain

Section 10.3:

Volume Rendering Integral

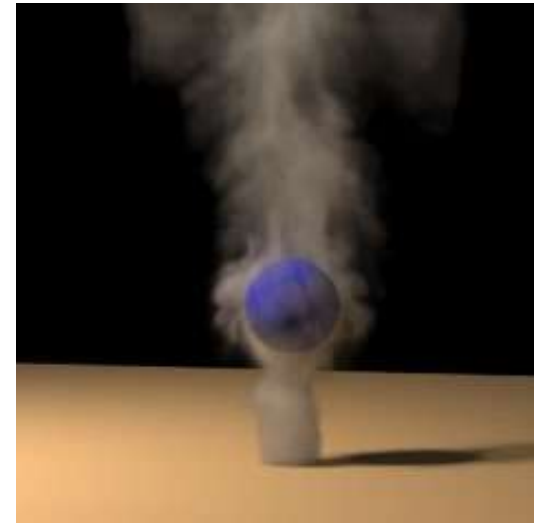
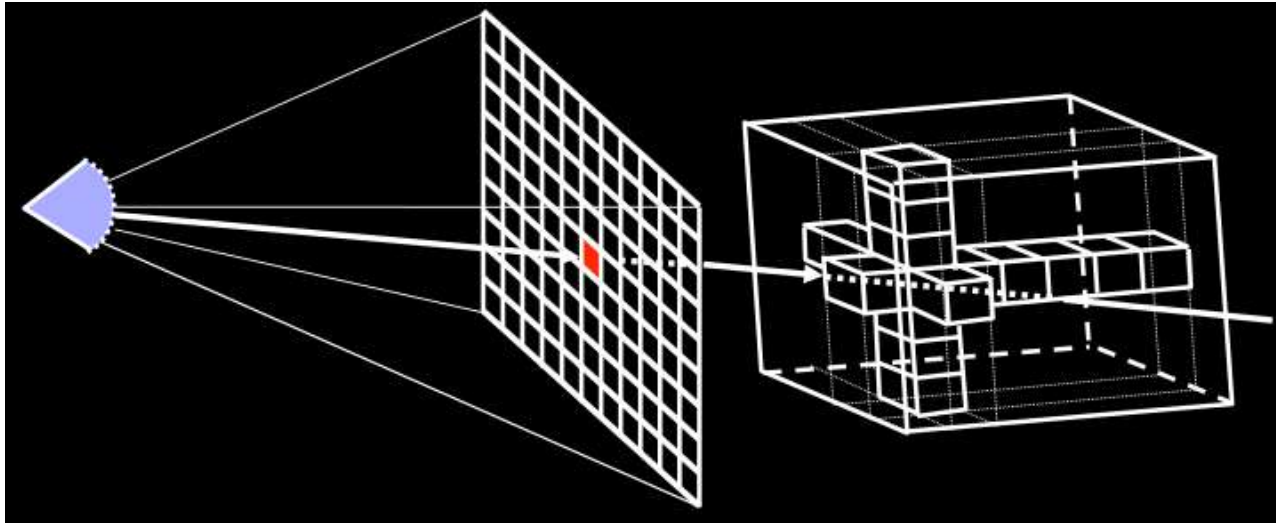
Where We Are In The Vis Pipeline



Volume Rendering: Formalization

Optical model: Each point in the volume *emits and absorbs light*, according to the color and opacity specified by the transfer function.

Those contributions are *integrated along viewing rays* to produce the final image.



Note: Direction of ray *opposite* to direction of light!

Setup: Volume Rendering Integral

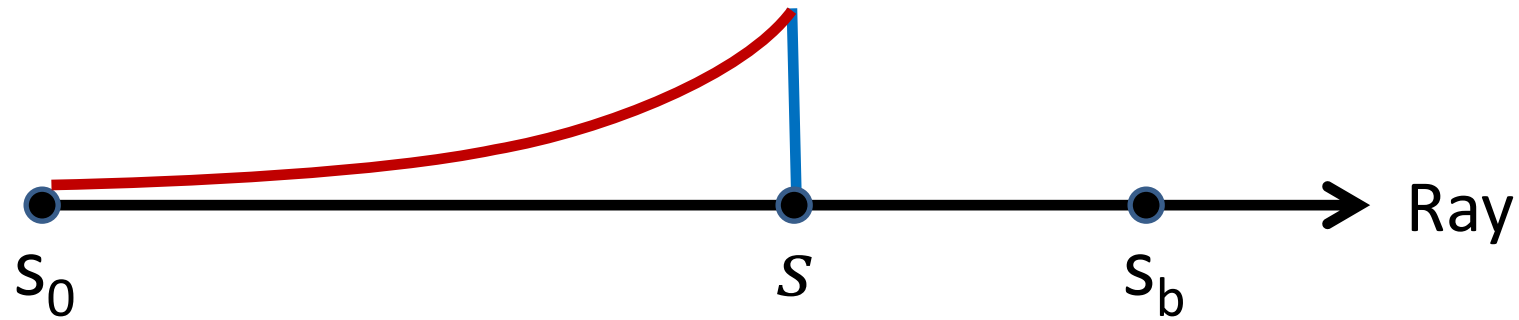


s_0 : Position on image

s_b : Position at back of volume

Task: Compute light intensity $I(s_0)$

Contribution From A Given Point



Light intensity that reaches s_0 from s :

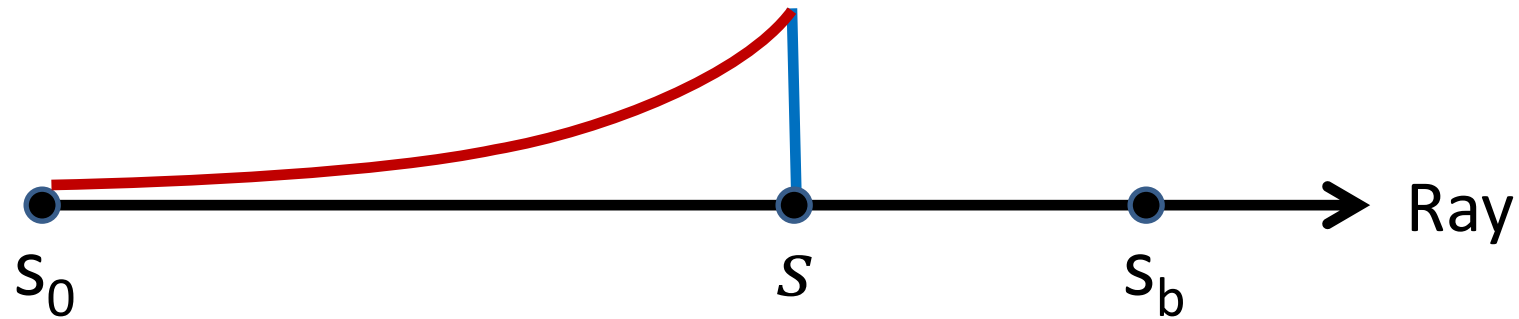
$$I(s_0|s) = \boxed{q(s)} \boxed{e^{-\tau(s_0,s)}}$$

Emission at s Attenuation along $s_0 - s$

Absorption vs. Optical Depth

Optical depth τ
Absorption κ

$$\tau(s_0, s) = \int_{s_0}^s \kappa(s') ds'$$



Light intensity that reaches s_0 from s :

$$I(s_0|s) = \boxed{q(s)} \boxed{e^{-\tau(s_0, s)}}$$

Emission at s Attenuation along $s_0 - s$

The Volume Rendering Integral

Optical depth τ
Absorption κ

$$\tau(s_0, s) = \int_{s_0}^s \kappa(s') ds'$$

Integrate contributions of
all points along the ray



Overall light intensity that reaches s_0 :

$$I(s_0) = \int_{s_0}^{s_b} q(s) e^{-\tau(s_0, s)} ds$$

Computing the Volume Rendering Integral

$$I(s_0) = \int_{s_0}^{s_b} q(s) e^{-\tau(s_0, s)} ds$$

Approximate the integral with a Riemann sum:

- Cast a ray $\mathbf{r}(s)$ from s_0 to s_b , split it into equidistant segments of length Δs , yielding $n = \lfloor (s_b - s_0) / \Delta s \rfloor$ steps
- Each segment i is assumed to have
 - constant opacity $A_i = 1 - e^{-\kappa(i \cdot \Delta s) \Delta s}$
 - constant light emission $\tilde{C}_i = C(i \cdot \Delta s) A_i$
 - *Notation:* $\kappa(i \cdot \Delta s)$ / $C(i \cdot \Delta s)$ are absorption coefficient/color along the ray
 - *Reminder:* Associated colors $\tilde{C}$ are multiplied by corresponding opacity

Absorption vs. Opacity

Optical depth: $\tau(s_o, s) = \int_{s_o}^s \kappa(s') ds'$

Discretize as: $\tau_i = \sum_{j=1}^{i-1} \kappa(j \cdot \Delta s) \Delta s$

Resulting attenuation: $e^{-\tau_i} = e^{-\sum_{j=1}^{i-1} \kappa(j \cdot \Delta s) \Delta s}$

Given opacity $A_j = 1 - e^{-\kappa(j \cdot \Delta s) \Delta s}$, we can re-write it as

$$e^{-\tau_i} = \prod_{j=1}^{i-1} e^{-\kappa(j \cdot \Delta s) \Delta s} = \prod_{j=1}^{i-1} (1 - A_j)$$

This allows for simple and efficient computation, but requires adapting A_j to step size Δs

Front-to-Back Compositing

$$\int_{s_0}^{s_b} q(s) e^{-\tau(s_0, s)} ds \approx \sum_{i=1}^n \tilde{C}_i \prod_{j=1}^{i-1} (1 - A_j)$$

Compute the sum iteratively (starting with $\tilde{C}'_0 = A'_0 = 0$; final result in $\tilde{C}'_n$)

- $\tilde{C}'_i = \tilde{C}'_{i-1} + (1 - A'_{i-1})\tilde{C}_i$
- $A'_i = A'_{i-1} + (1 - A'_{i-1})A_i$
 - follows from $1 - A'_i = (1 - A_i)(1 - A'_{i-1})$

Early ray termination: A'_i is monotonically increasing; we can stop computing once $A'_i \approx 1$

Analogy to Ray Tracing

- Similar to ray tracing in surface-based computer graphics
- In volume rendering,
 - we only deal with primary rays; hence: *ray-casting*
 - we composit in each step, rather than searching a ray/surface intersection
 - we ignore rays that do not intersect the volume

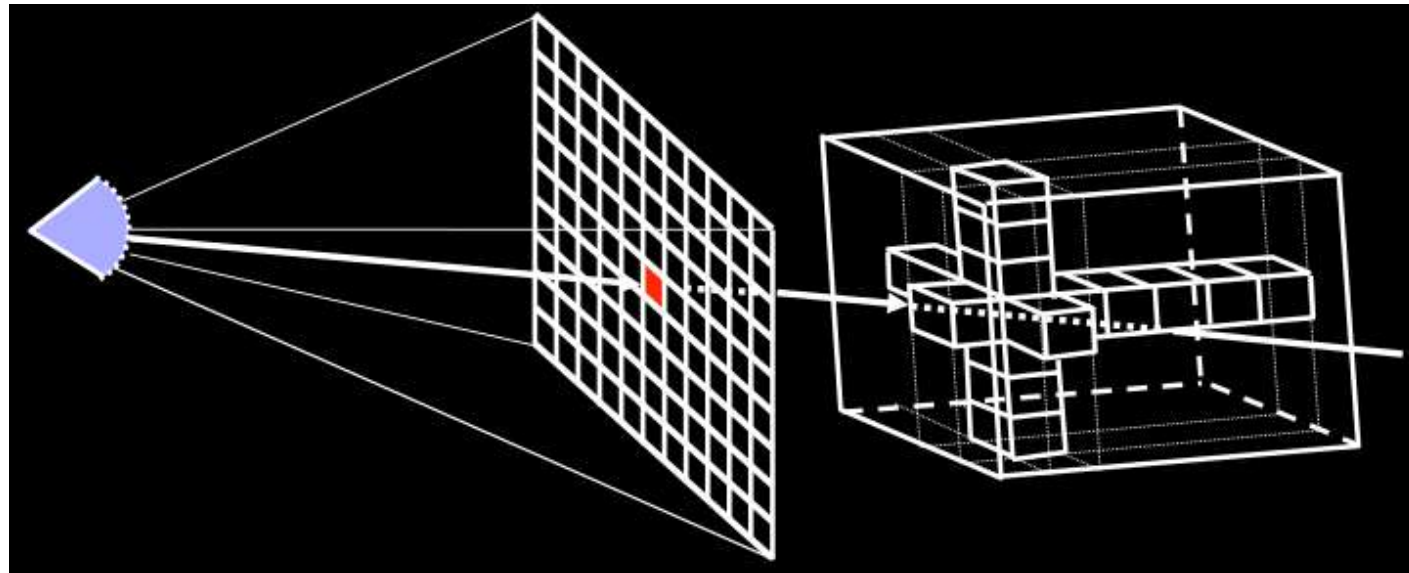


Illustration of Ray Casting

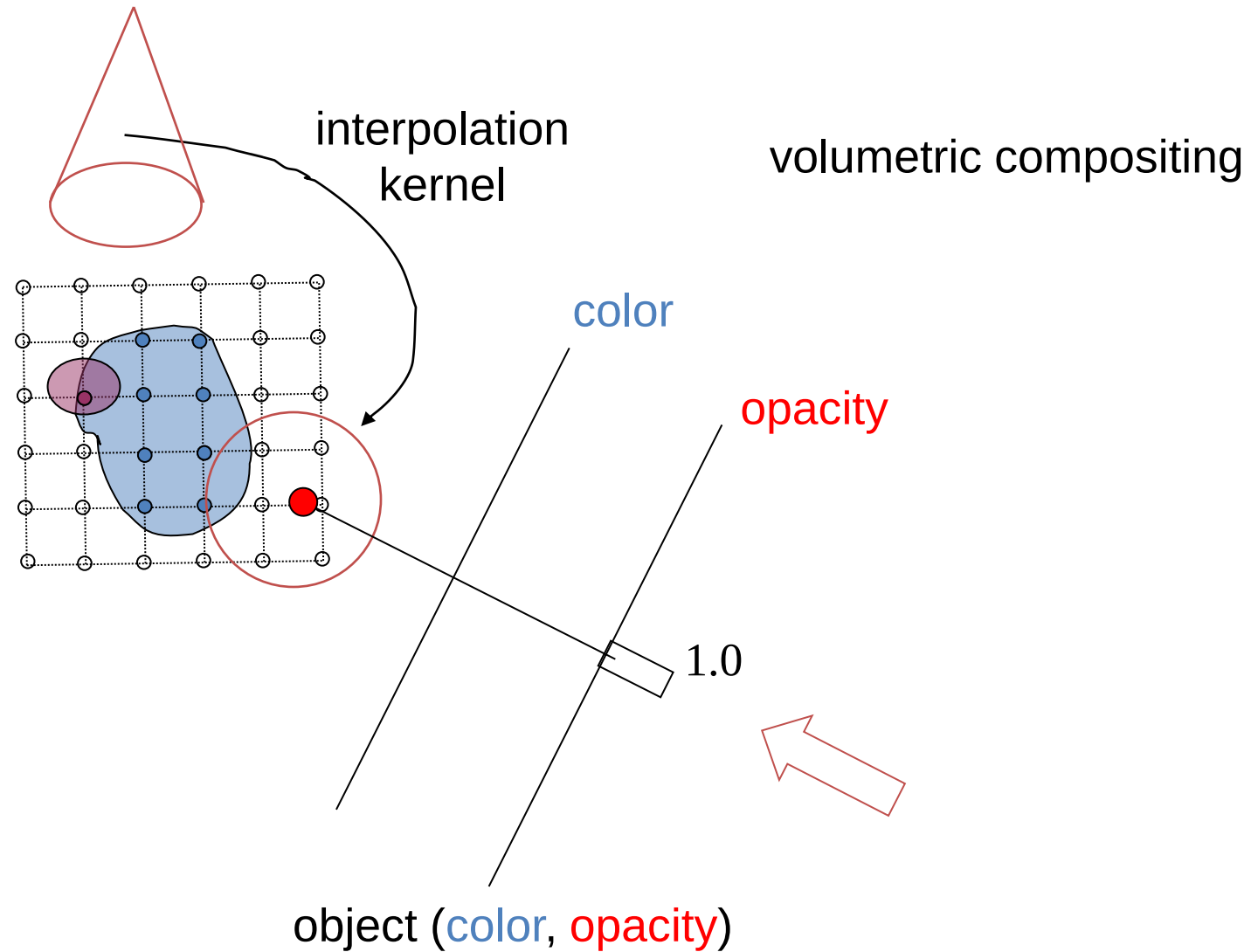


Illustration of Ray Casting

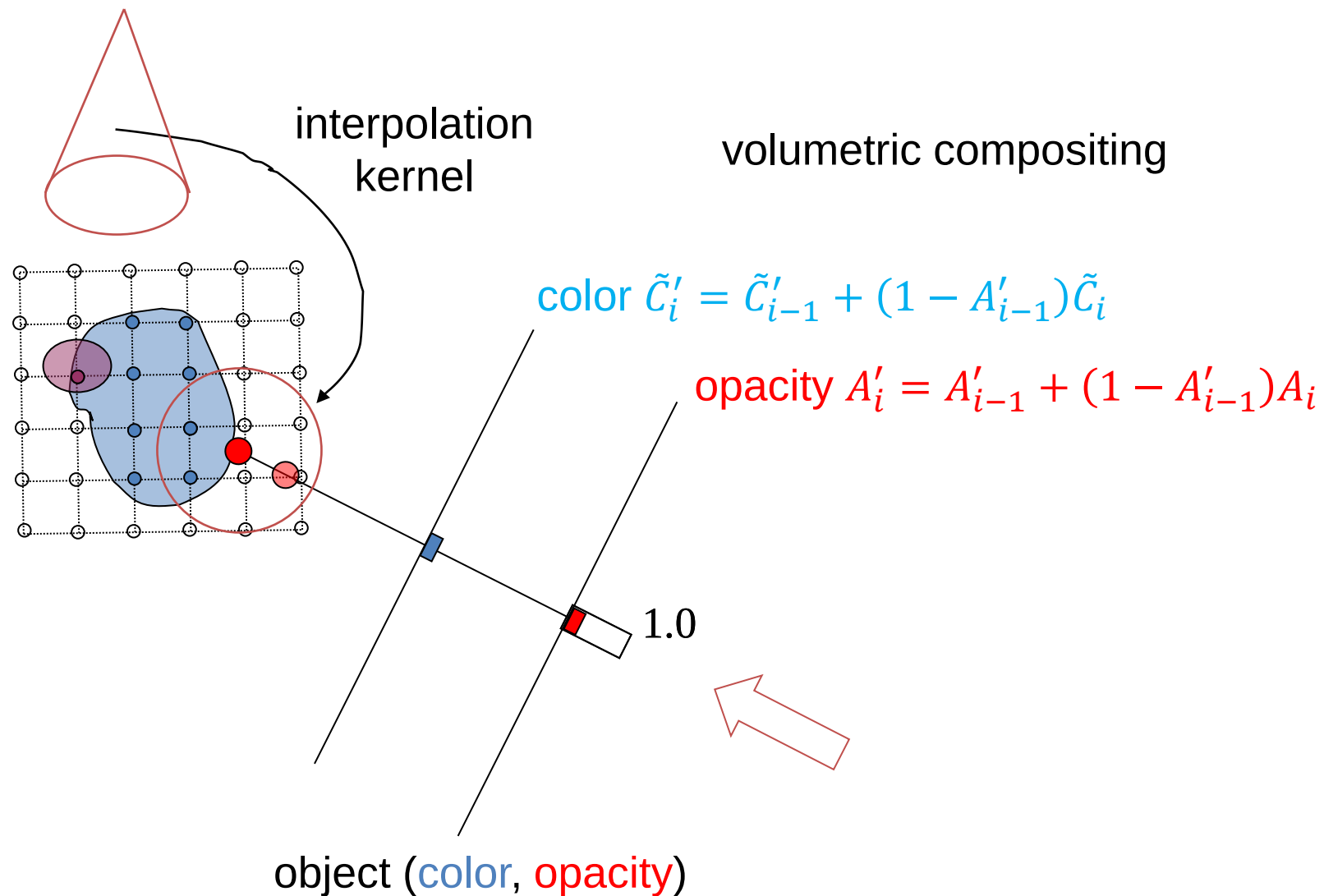


Illustration of Ray Casting

volumetric compositing

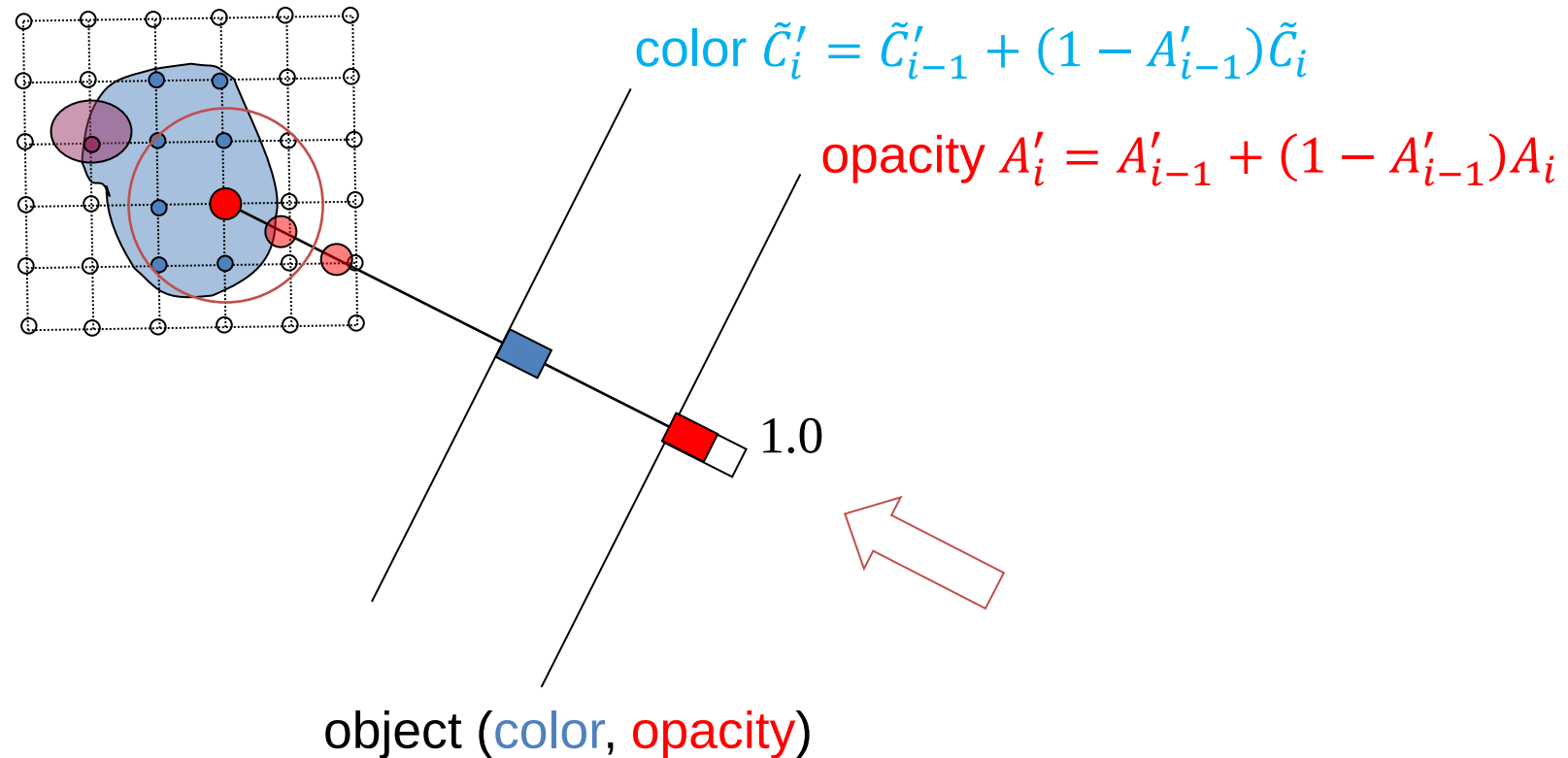


Illustration of Ray Casting

volumetric compositing

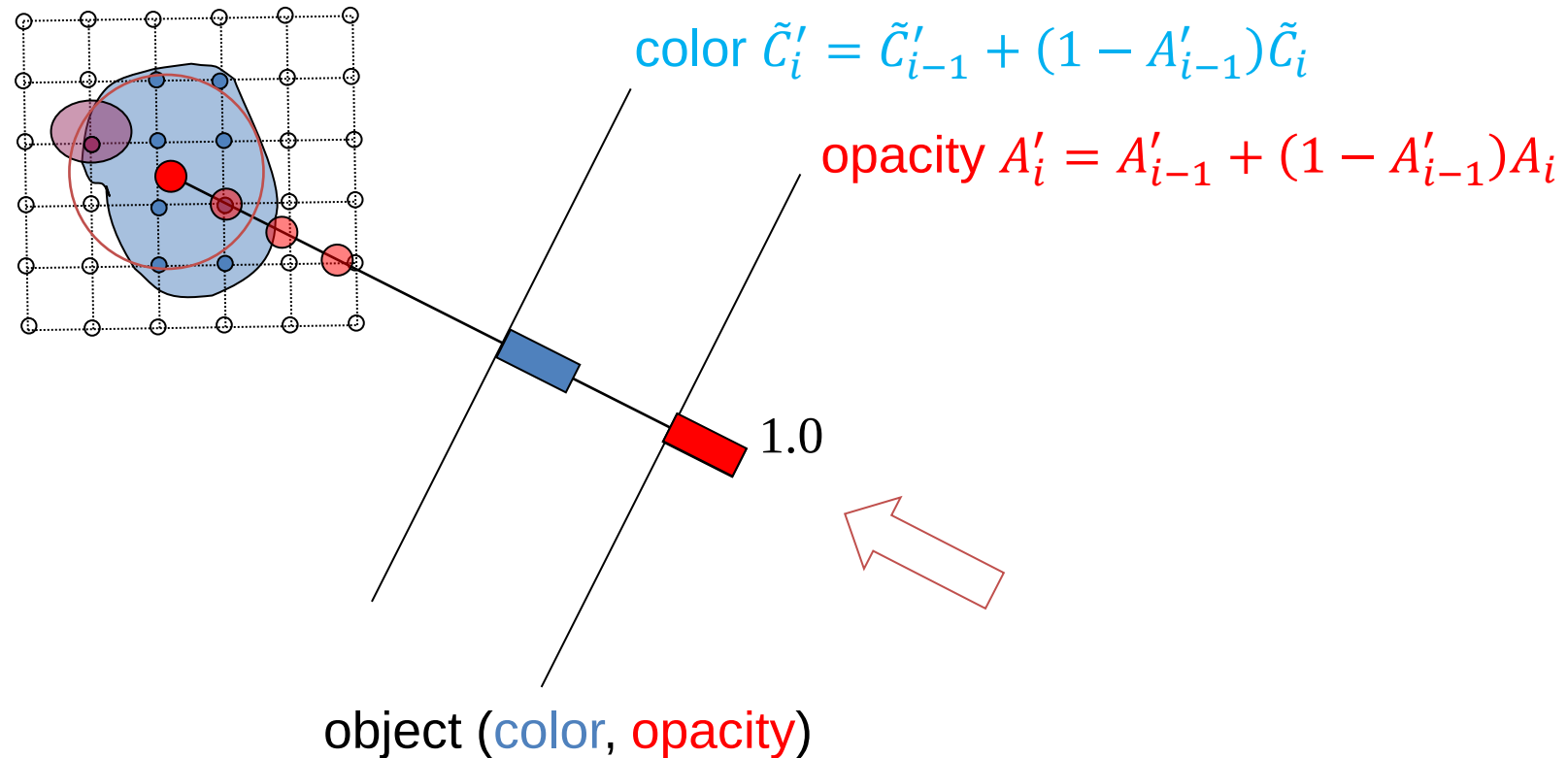


Illustration of Ray Casting

volumetric compositing

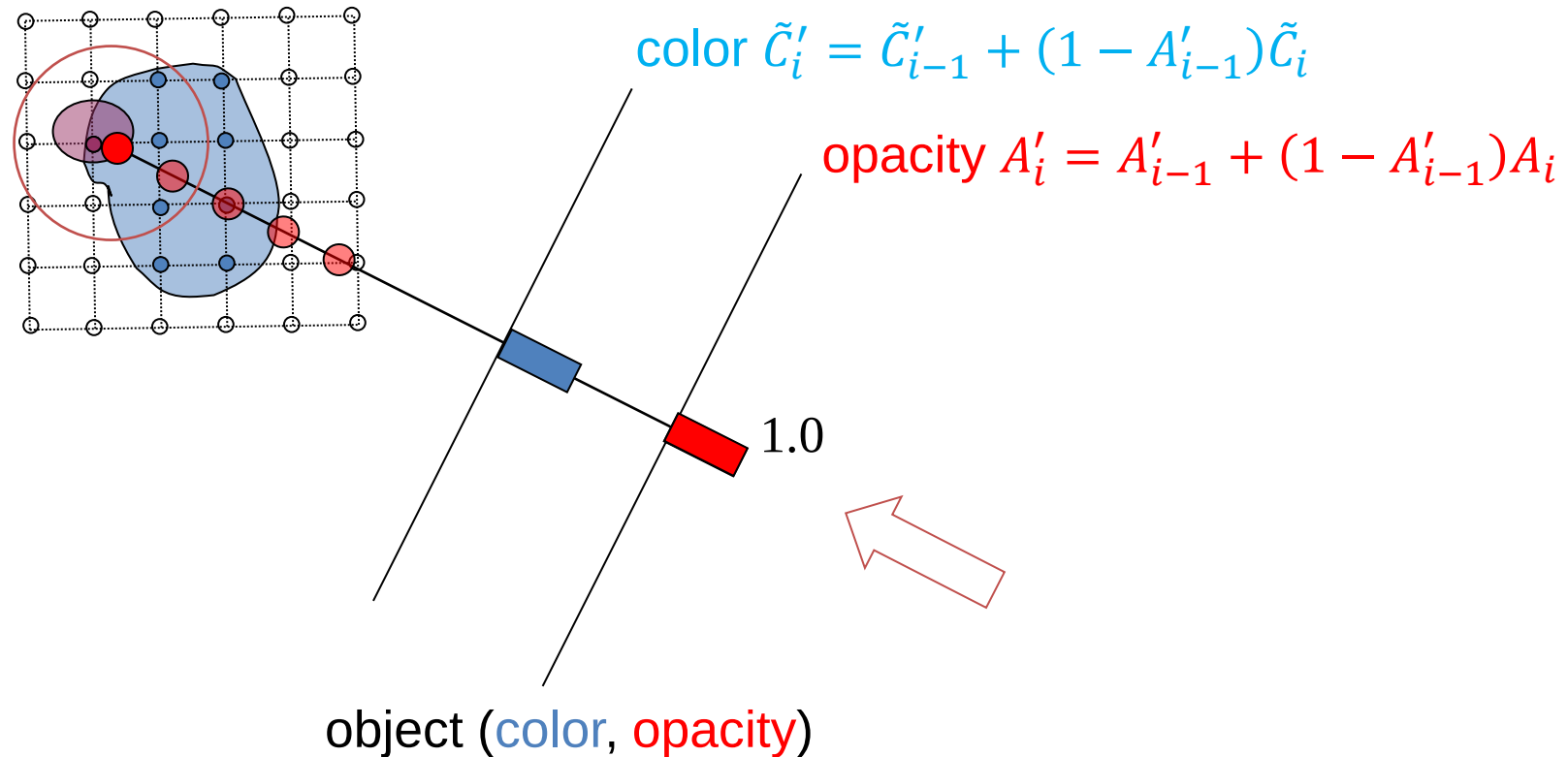


Illustration of Ray Casting

volumetric compositing

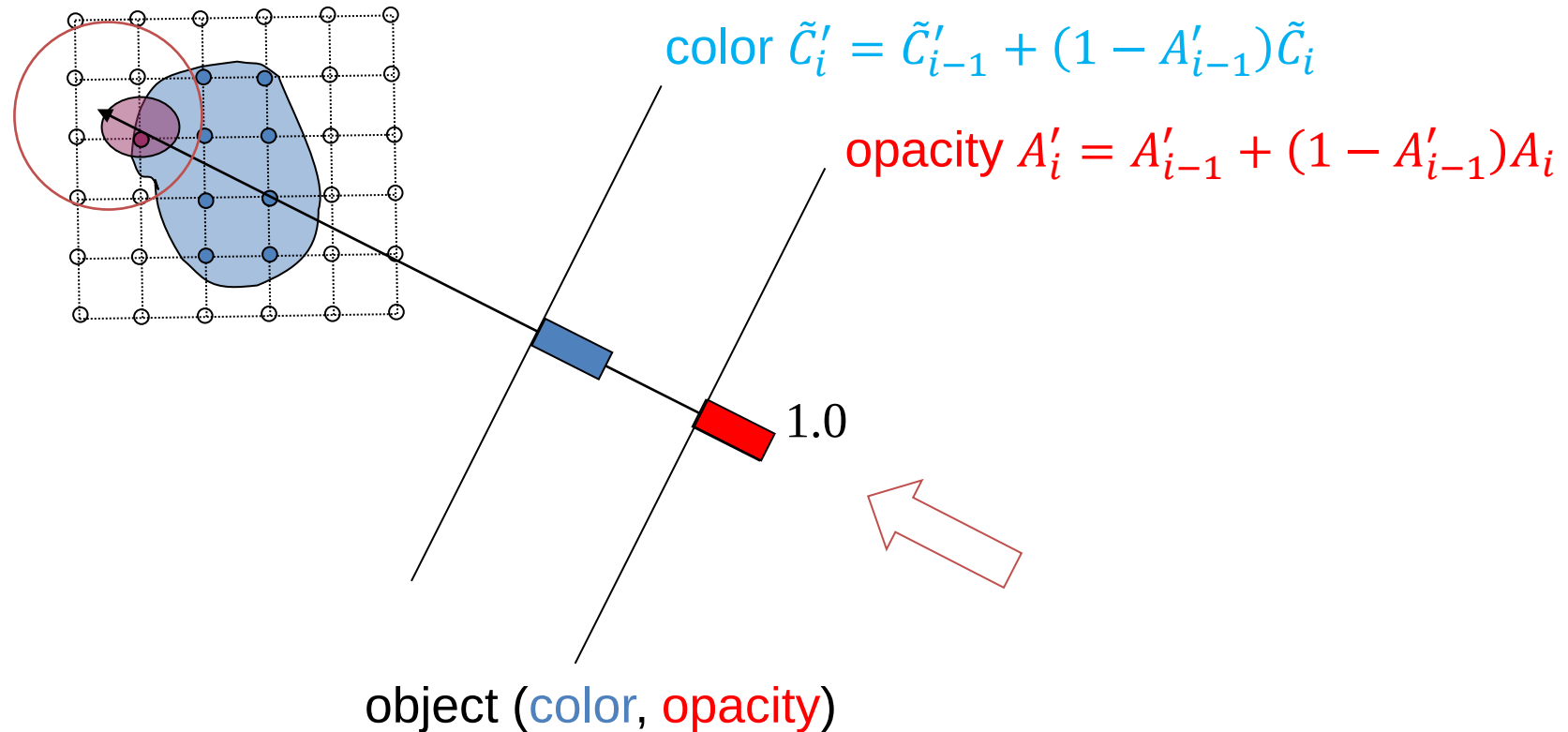
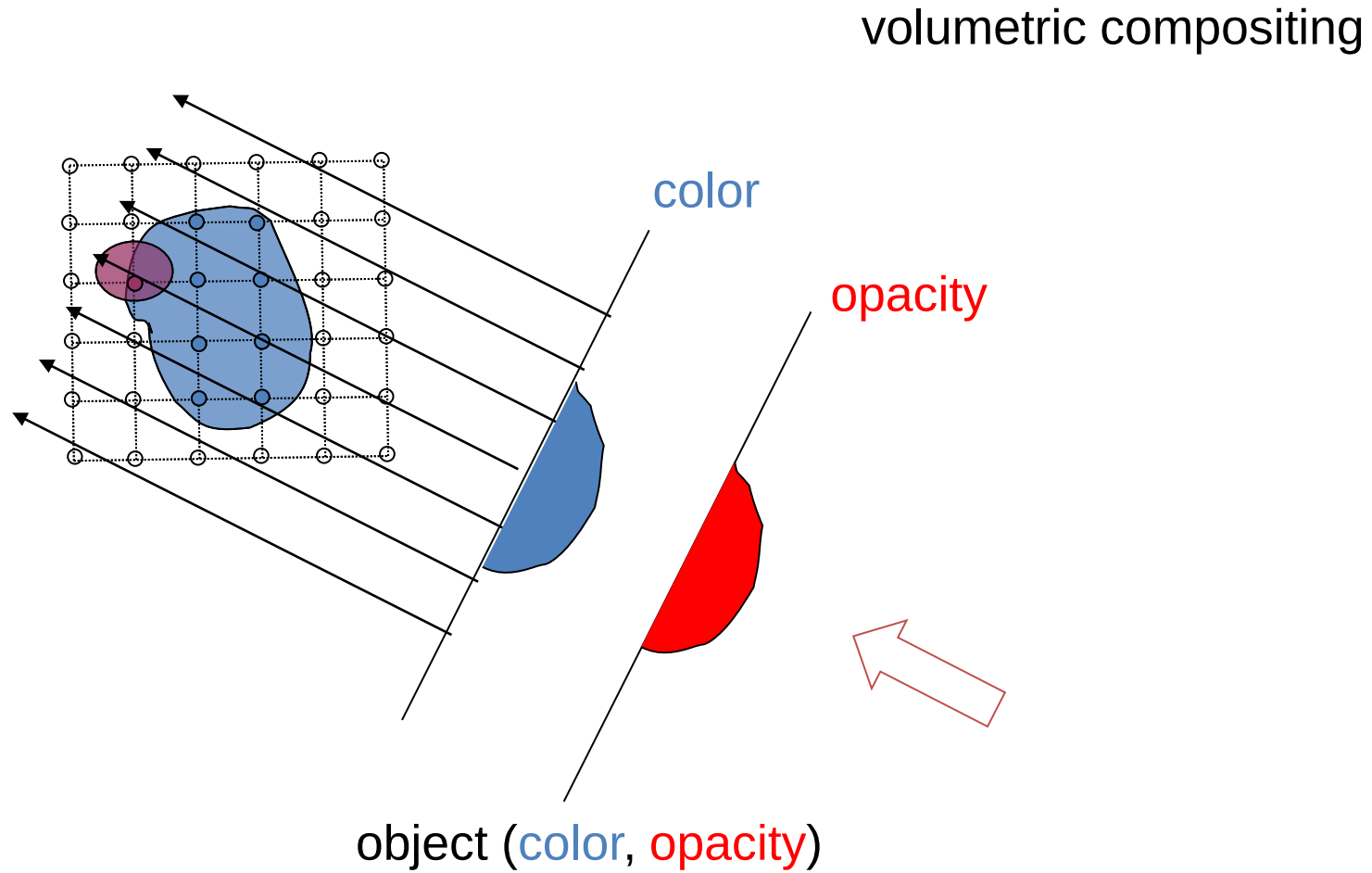


Illustration of Ray Casting



Aside: Opacity Peeling

Problem: Nested structures with similar intensities are notoriously difficult to volume render

- *Example:* Render the brain based on head MRI
- Pre-segmentation is the typical solution, but requires separate pre-process, not always reliable



Simple yet effective trick by [Rezk-Salama/Kolb 2006]:

- Instead of terminating the ray when the accumulated opacity saturates, start rendering a second image
- Second image should reveal structures that are otherwise hidden behind the first

Two Thresholds in Opacity Peeling

Problem: Thresholding accumulated opacity alone leads to incomplete peeling.

Solution: Use a second threshold.
Start new layer if

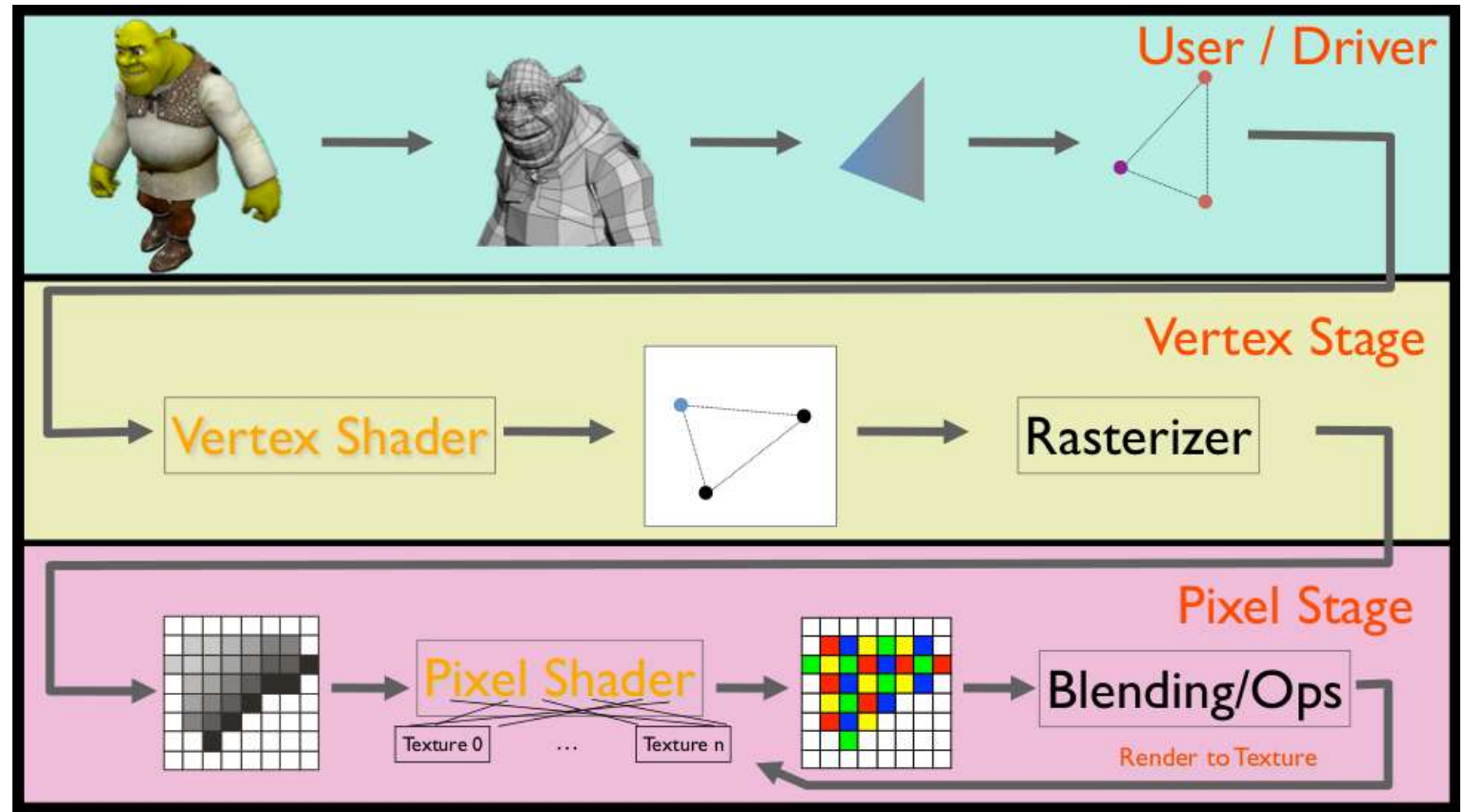
- Accum. alpha $> T_{\text{high}}$ and
- Local alpha $< T_{\text{low}}$



Technical Note: GPU Programming

Graphics Processing Units (GPUs) are highly parallel coprocessors that support rapid image synthesis

- Pipeline programmable since mid-2000s
- Ray casting can be implemented in pixel (fragment) shader and is now widely used
- When only a fixed-function pipeline is available, back-to-front compositing can be used



Back-to-Front Compositing

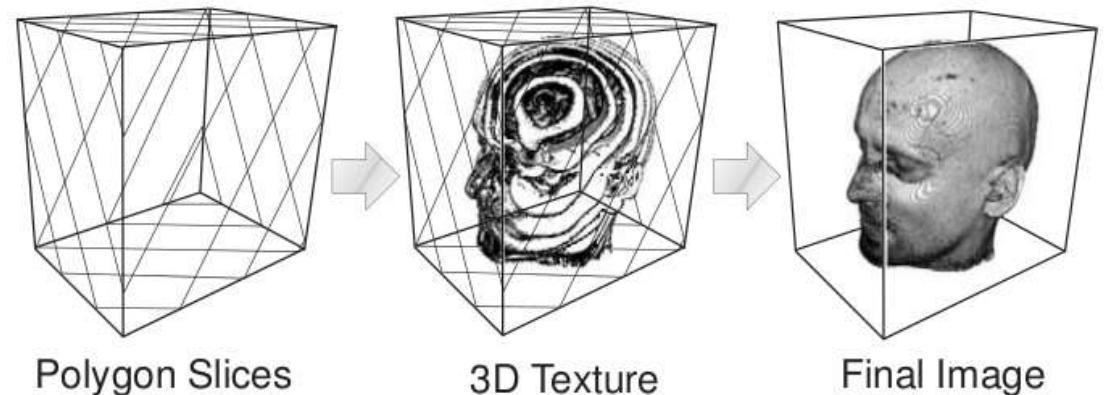
$$\int_{s_0}^{s_b} q(s) e^{-\tau(s_0, s)} ds \approx \sum_{i=1}^n \tilde{C}_i \prod_{j=1}^{i-1} (1 - A_j)$$

Compute the sum recursively

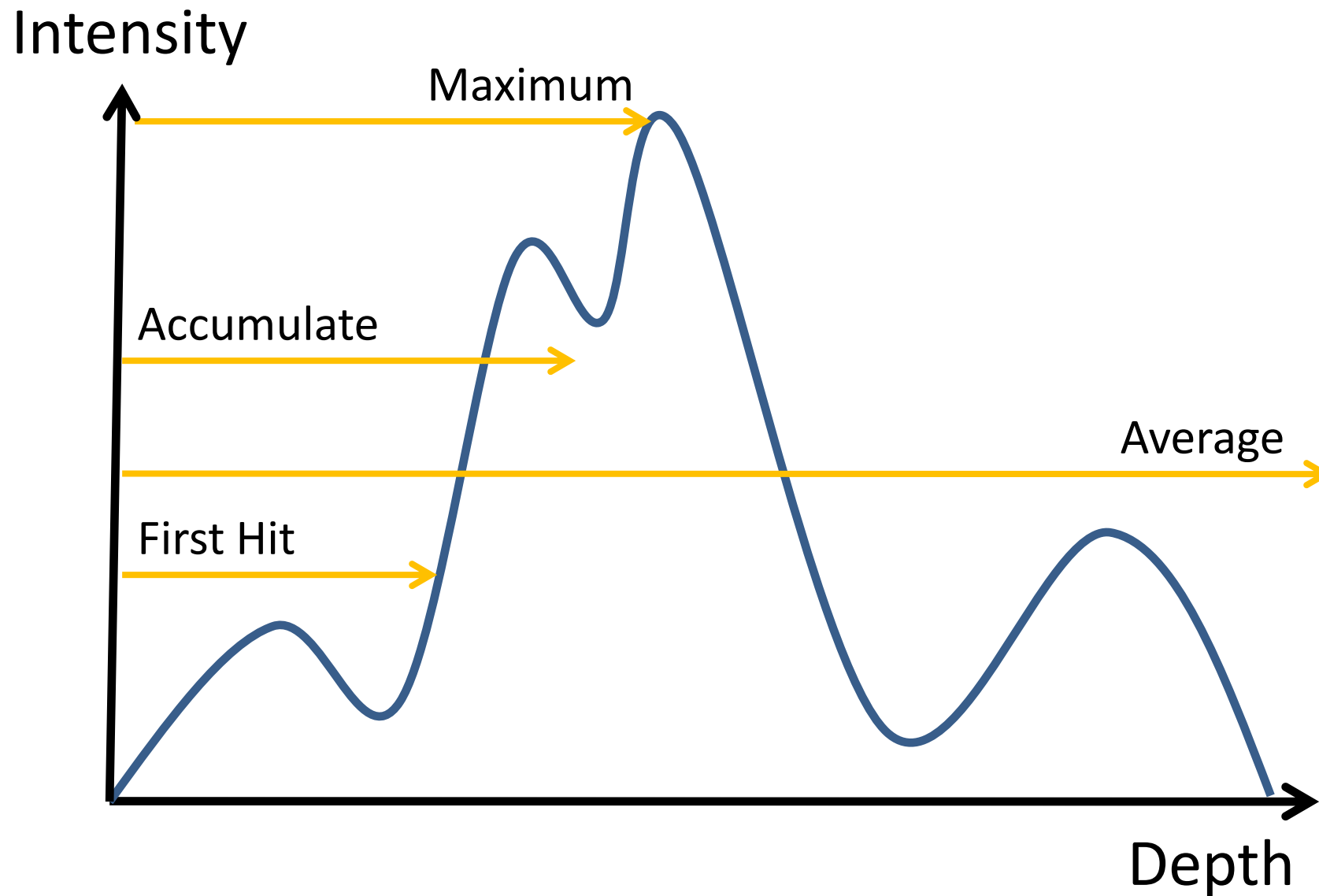
(starting with $\tilde{C}'_{n+1} = 0$; final result in $\tilde{C}'_1$)

- $\tilde{C}'_i = \tilde{C}_i + (1 - A_i) \tilde{C}'_{i+1}$

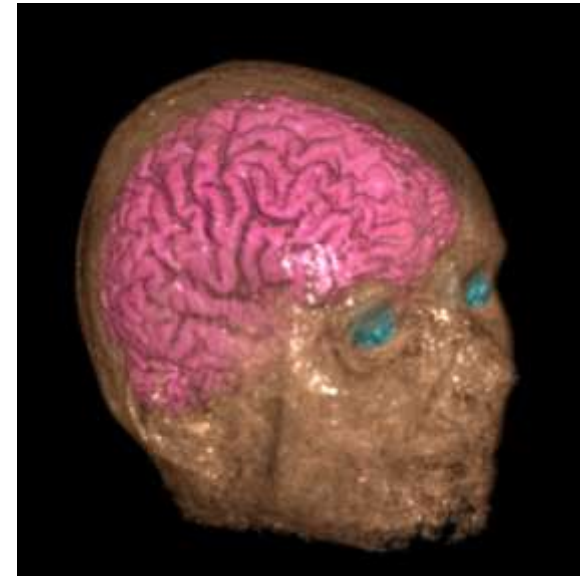
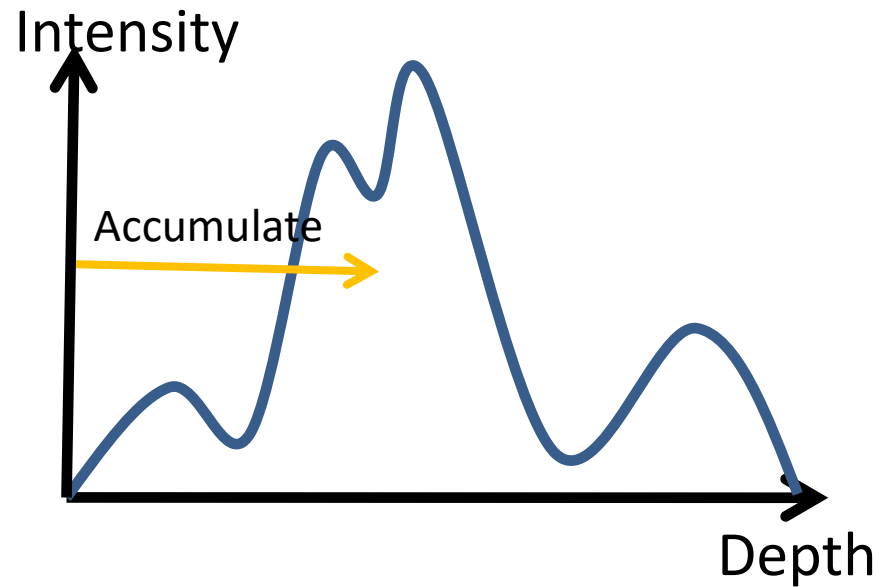
Implemented by rendering view-aligned proxy geometry in back-to-front order with alpha blending:



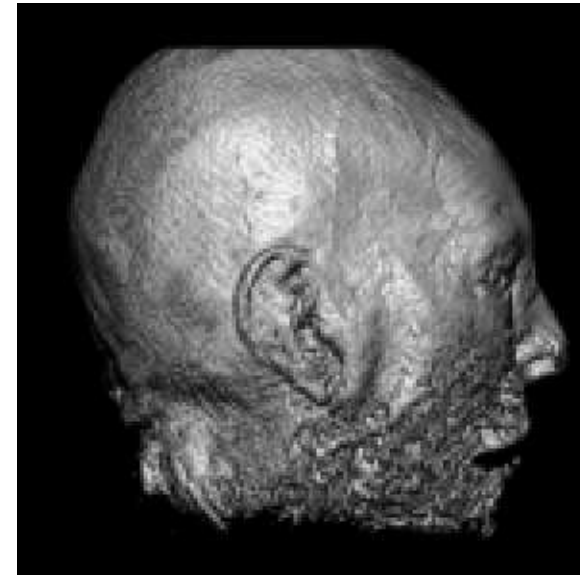
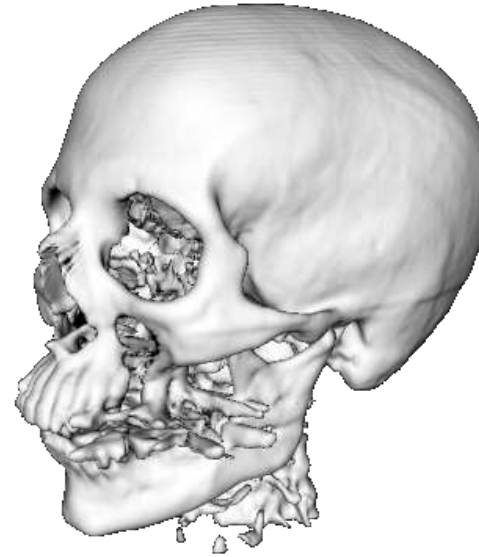
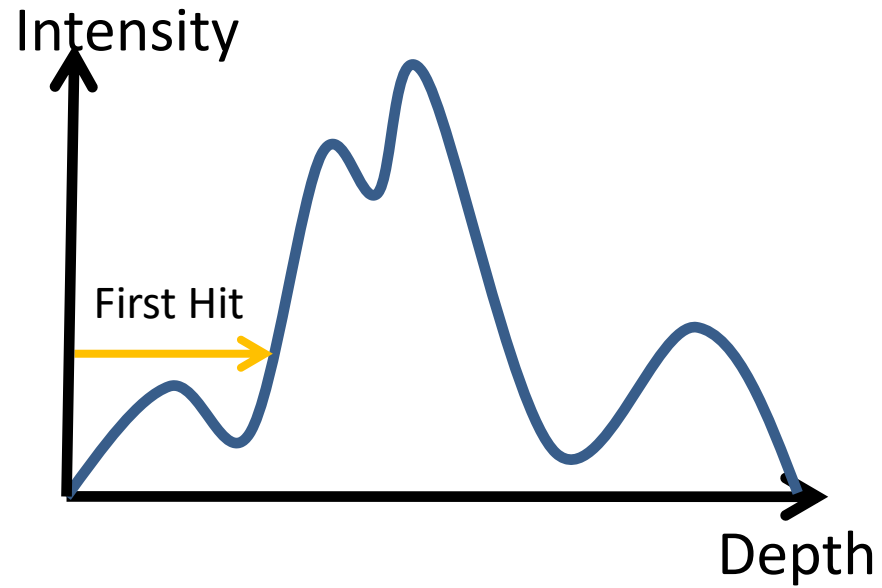
Other Compositing Schemes



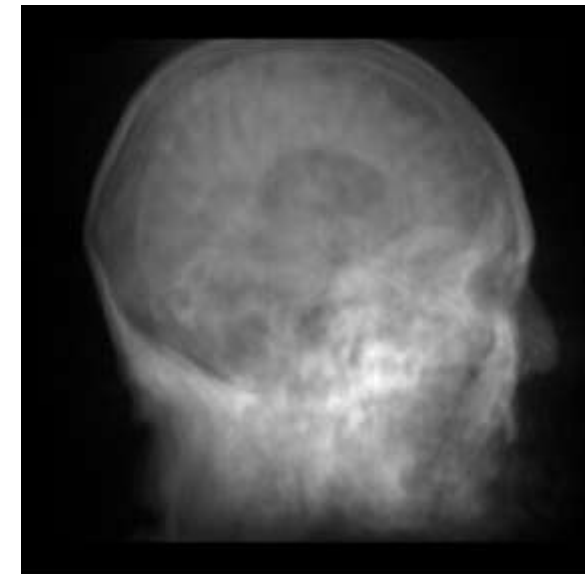
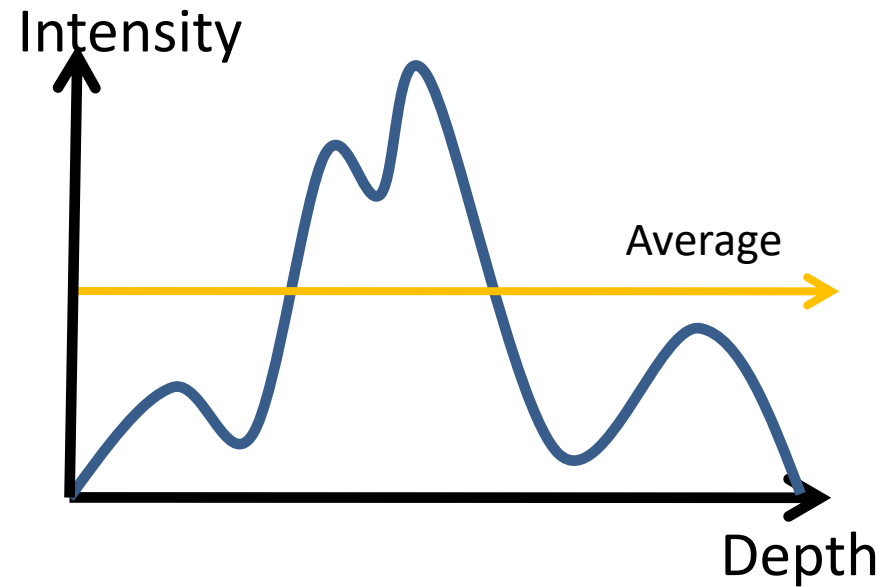
Accumulate: Standard Volume Rendering



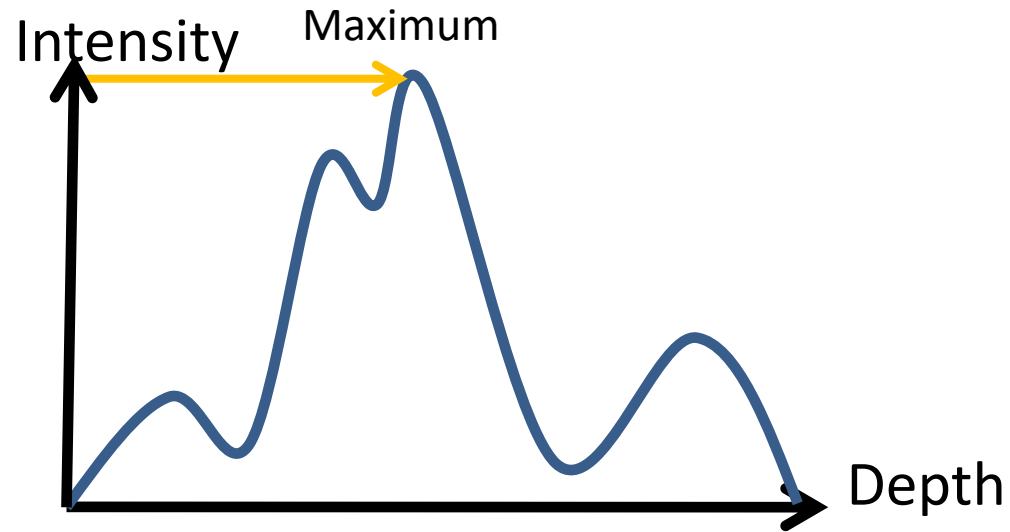
First Hit: Renders an Isosurface



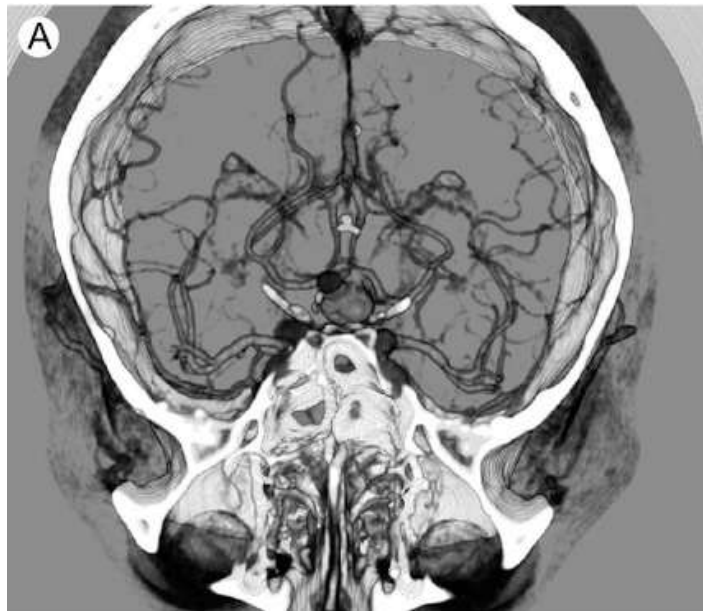
Averaging: Produces X-ray(-like) images



Maximum Intensity Projection (MIP)

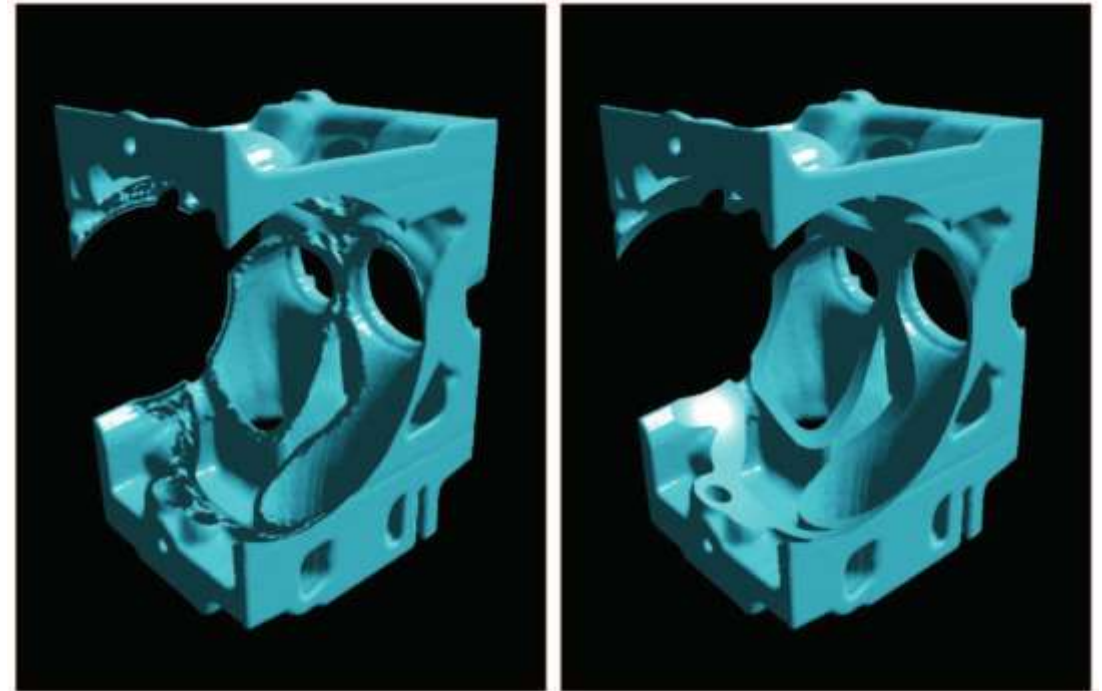


Often used to extract vessel structures in magnetic resonance angiograms



Volume Rendering with Clipping Geometry

- Parts of the volume can be made transparent by specifying clipping geometry.
- At its boundary, the surface normal of the clipping geometry, rather than the gradient of the volume, should be used for illumination:



(a)

(b)

Summary: Volume Rendering

- **Volume rendering** produces a 2D image of a 3D volume that contains colors and opacities
- Computes the light that would arrive at the image plane given an **optical model**
 - Assumes emission and absorption of light
- **Volume rendering integral**
 - Usually solved by ray casting with front-to-back compositing
 - *Alternative*: Textured slice based back-to-front compositing can be achieved with a traditional fixed-function graphics pipeline
- Alternative **compositing schemes**
 - MIP, average, first hit

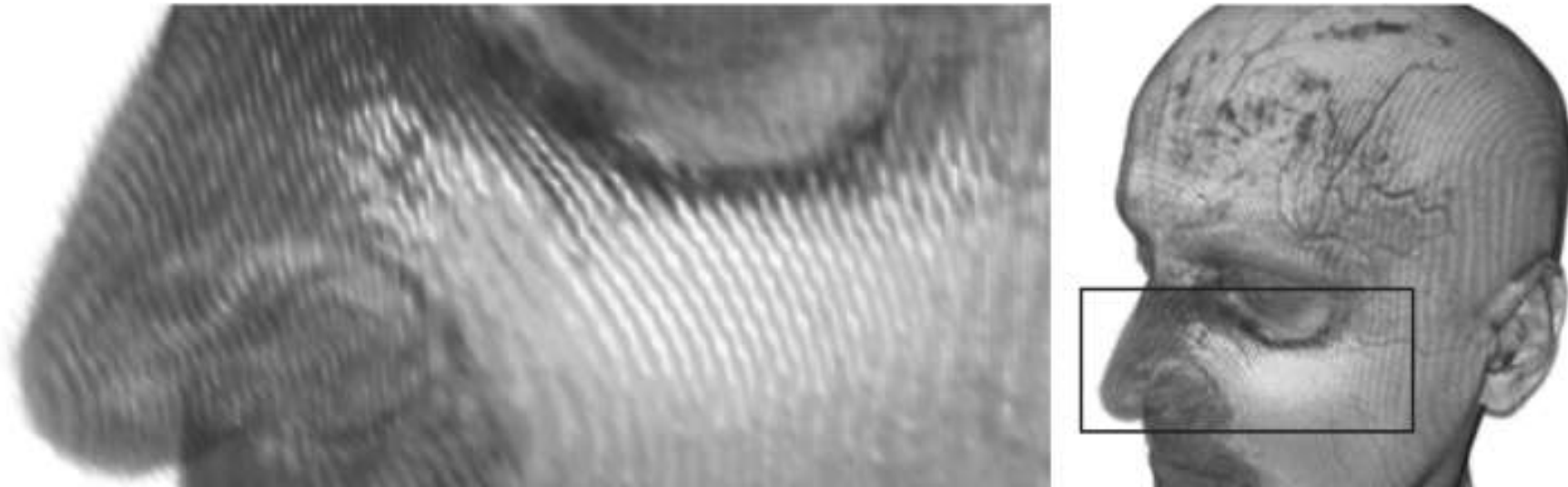
Section 10.4:

Accelerated Volume Rendering

Increase Stepsize during Interaction

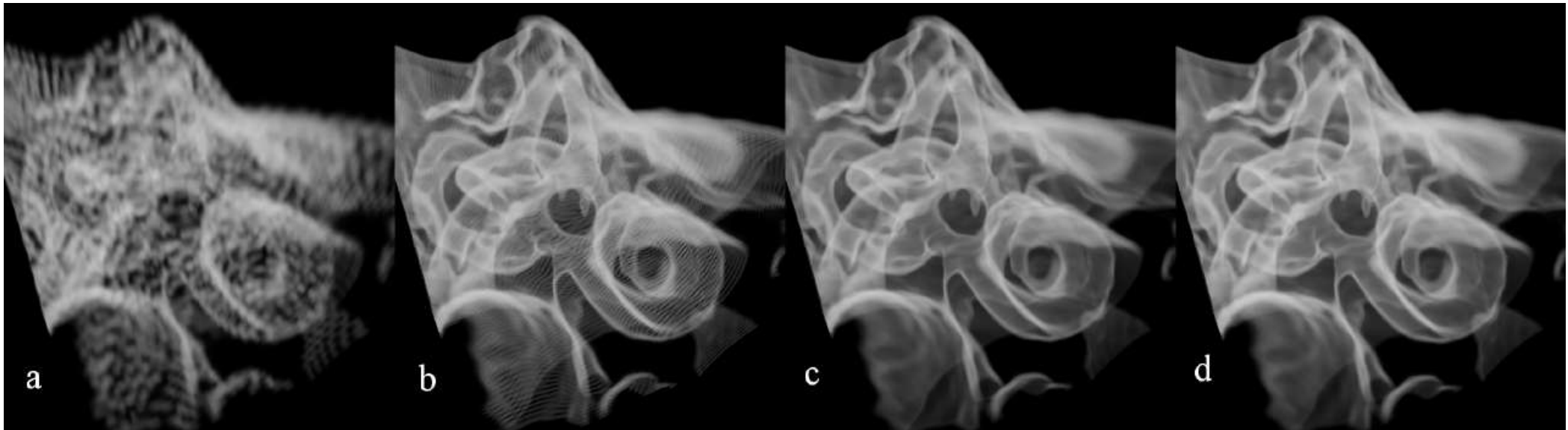
Increasing stepsize in ray casting, or reducing the number of slices reduces the computational effort, but can lead to artifacts in the visualization.

Common trick: Increase stepsize to allow for a more fluent interaction (e.g., changing the relative camera position), produce a high-quality still image afterwards.



Motivation: Pre-Integration

- **Pre-integration** [Engel et al. 2001] greatly improves rendering quality with limited sampling rates
 - Replaces assumption of constant data values in each step with piecewise linear assumption



Pre-classified

Post-classified
(low rate)

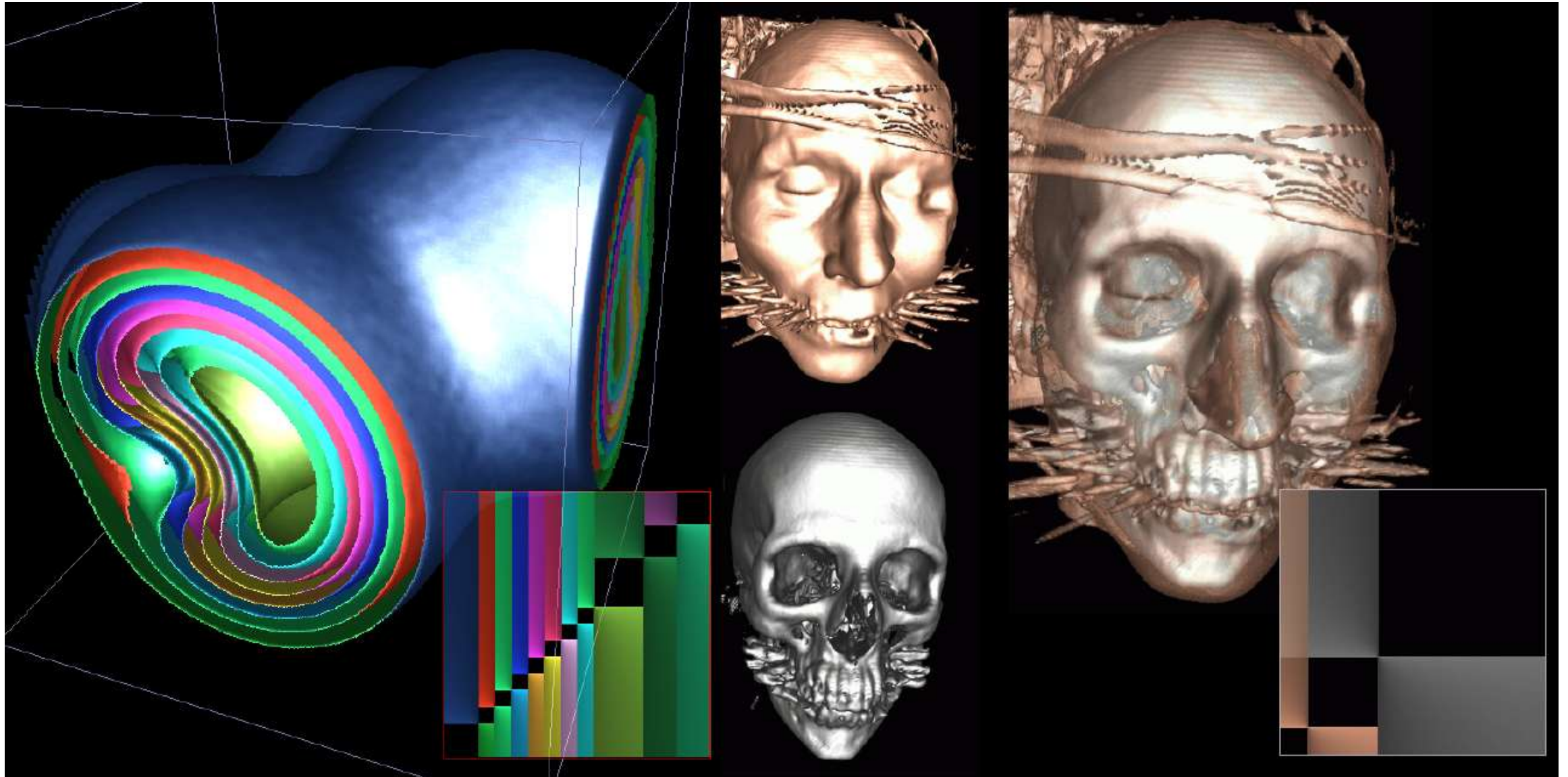
Post-classified
(high rate)

Pre-integrated
(low rate)

Pre-Integration

- Color and opacity vary **nonlinearly** in the data, and possibly much more rapidly, requiring very small step sizes
- Exact color and opacity within a step of size Δs are:
 - $A_i = 1 - e^{-\int_0^1 \kappa((i-1+\omega) \cdot \Delta s) \Delta s d\omega}$
 - $\tilde{C}_i = \int_0^1 C((i-1+\omega) \cdot \Delta s) (1 - e^{-\int_0^\omega \kappa((i-1+\omega') \cdot \Delta s) \Delta s d\omega'}) d\omega$
- Assuming data varies linearly within a step, these integrals can be pre-computed and stored in a 2D **lookup table**
 - Dimensions: Front/back value
 - Lookup table has to be (partly) recomputed when changing the transfer function

Isosurface Rendering with Pre-Integration



Empty Space Skipping

Space leaping

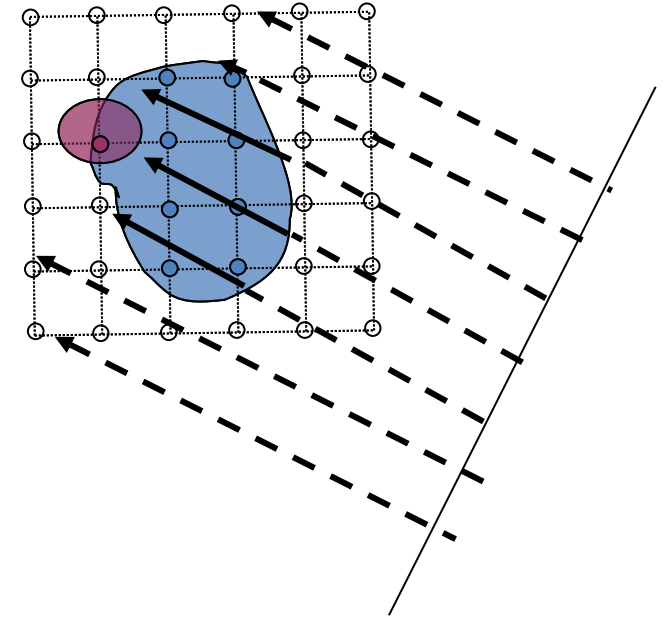
- Skip empty cells

More generally: Homogeneity acceleration

- Approximate homogeneous regions with fewer sample points

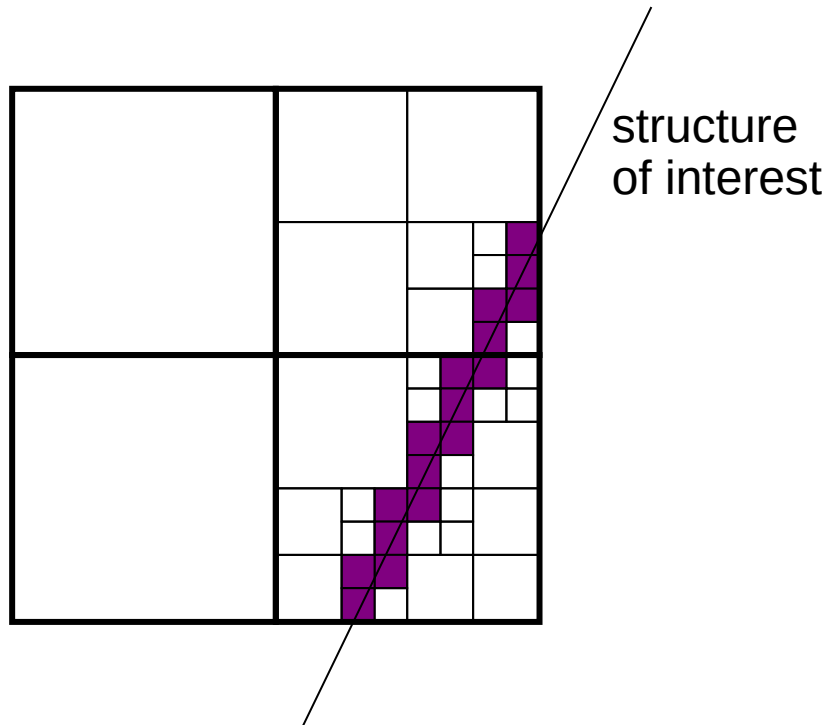
Approaches:

- Min-max Octrees
- Bounding geometry around objects
- Proximity clouds

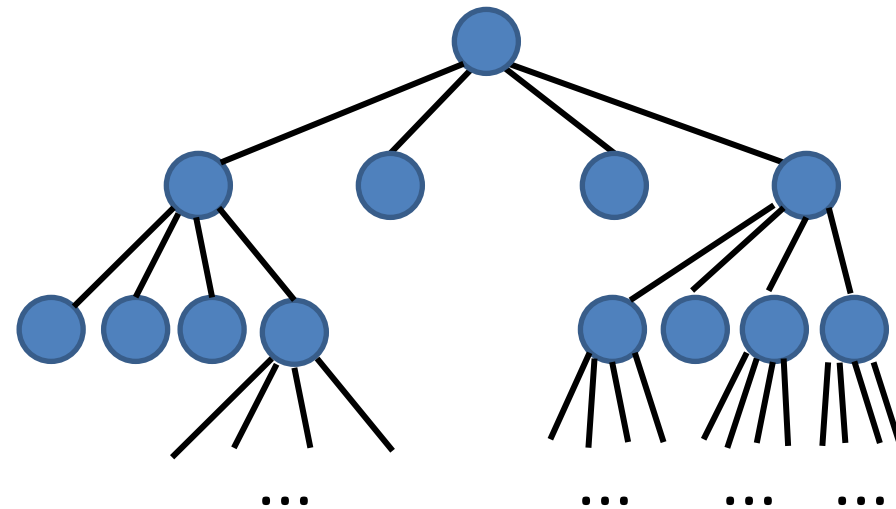


Background: Quadtrees / Octrees

- **Quadtrees** are a popular type of 2D unstructured grid that permit adaptively refining the resolution around structures of interest
 - Frequently used to save memory compared to high-res structured grid
- **Octrees** are the 3D counterpart



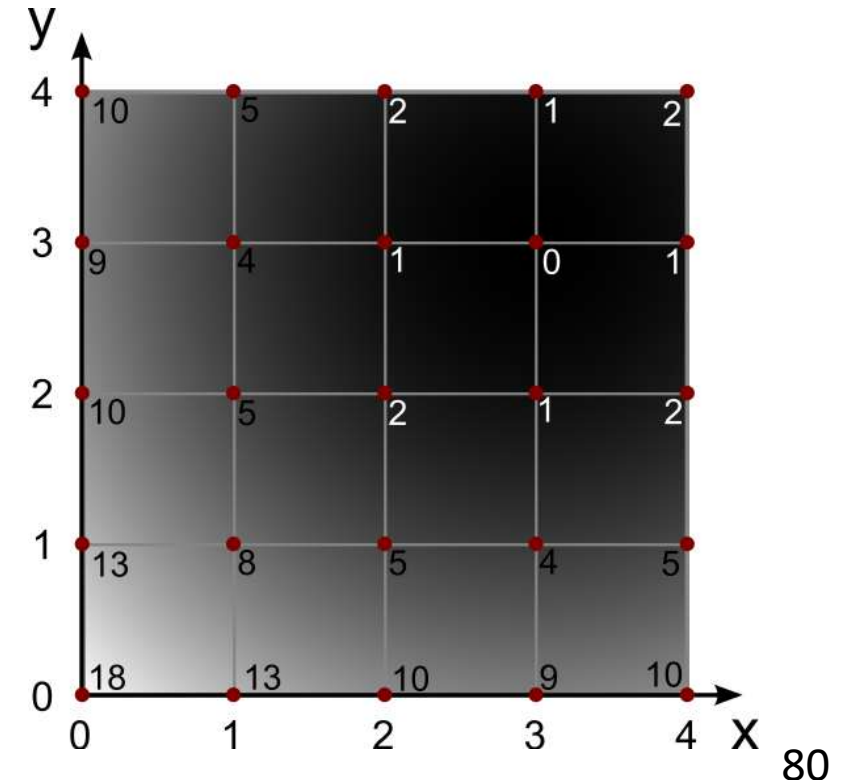
Node-and-Link Diagram:



Min-Max Octree

Min-Max Octree [Wilhelms, van Gelder 1992]

- Full octree, built on top of a complete high-resolution volume
 - Need *additional* memory
 - Goal is to save time, not memory
- Represents a hierarchical subdivision of the domain
 - Each node stores minimum and maximum scalar values within the respective domain
 - While traversing the octree, we can
 - skip parts which receive zero opacity
 - take larger integration steps in homogeneous regions

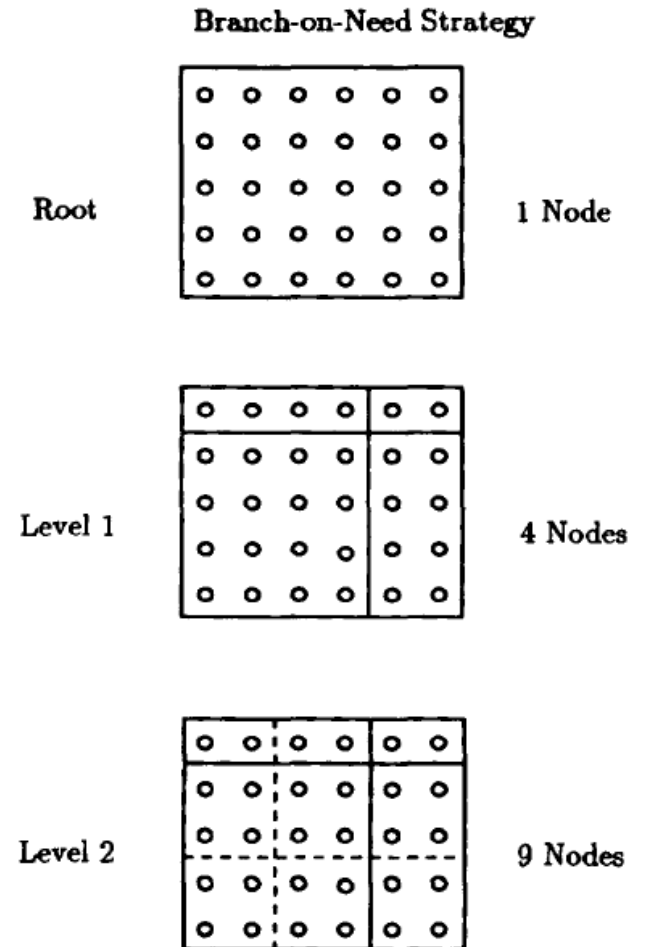


Problem with Full Octree

- **Problem** with memory consumption of complete octree:
 - Ideal: grid size of $2^n \times 2^n \times 2^n$
 - However: In practice, often have different resolutions that are not powers of two
- *Example:*
 - Data set: $320 \times 320 \times 40 \approx 4\text{M}$ data points
 - Full octree: $1 + 2^3 + 4^3 + \dots + 256^3 \approx 20\text{M}$ inner nodes
 - 2 values per element: minimum and maximum values

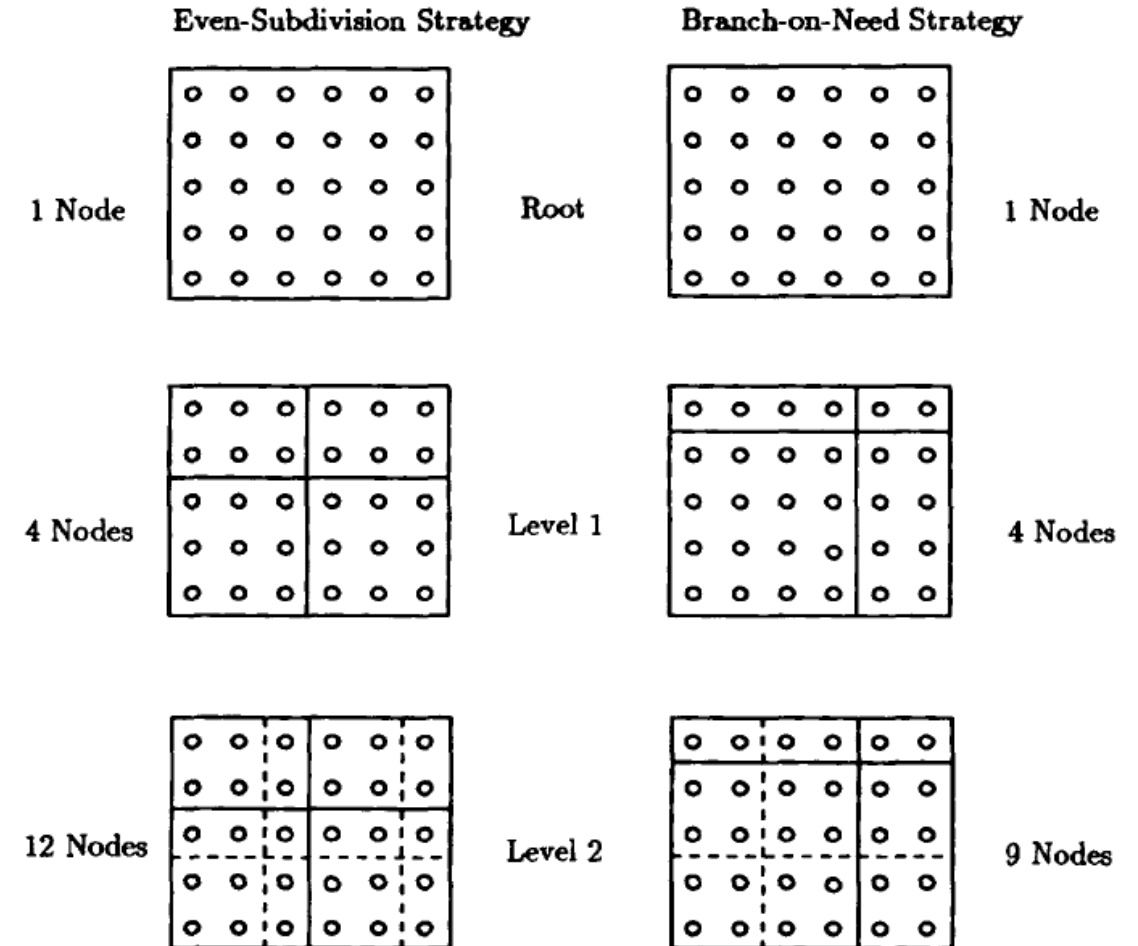
Branch on Need Octree (BONO)

- Branch on Need Octrees (BONO) can be interpreted as
 - still padding the given volume to the next power of two: $2^n \times 2^n \times 2^n$
 - only subdividing inner nodes that actually overlap with the original volume
- For example on previous slide: $\approx 585k$ nodes (opposed to 20M nodes)



Branch on Need Octree (BONO): Intuition

- Alternatively, BONO can be thought of as a bottom-up way of constructing an Octree
 - *Intuition*: Try to be as efficient as possible at the finest level (where most voxels are)
- Results are the same in ideal case $2^n \times 2^n \times 2^n$



Optimal Ratio of Inner Octree Nodes

- The number of *inner* nodes in a full octree on top of a uniform $s \times s \times s$ grid is

$$n_{\text{nodes}} = \sum_{i=0}^{(\log_2 s)-1} 8^i = \frac{s^3 - 1}{7} \approx 0.14 n_{\text{data points}}$$

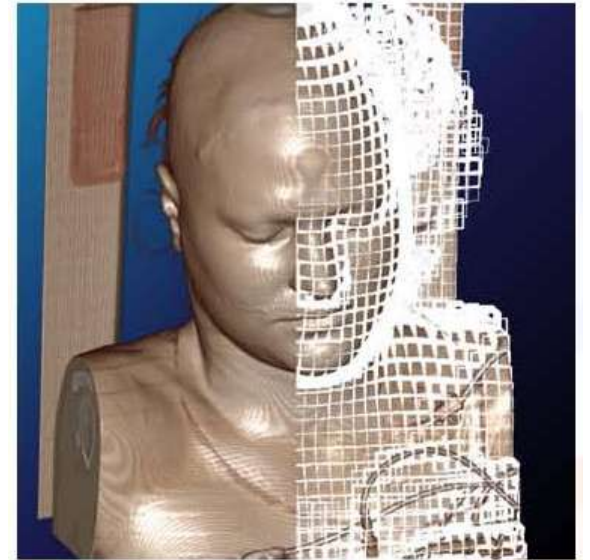
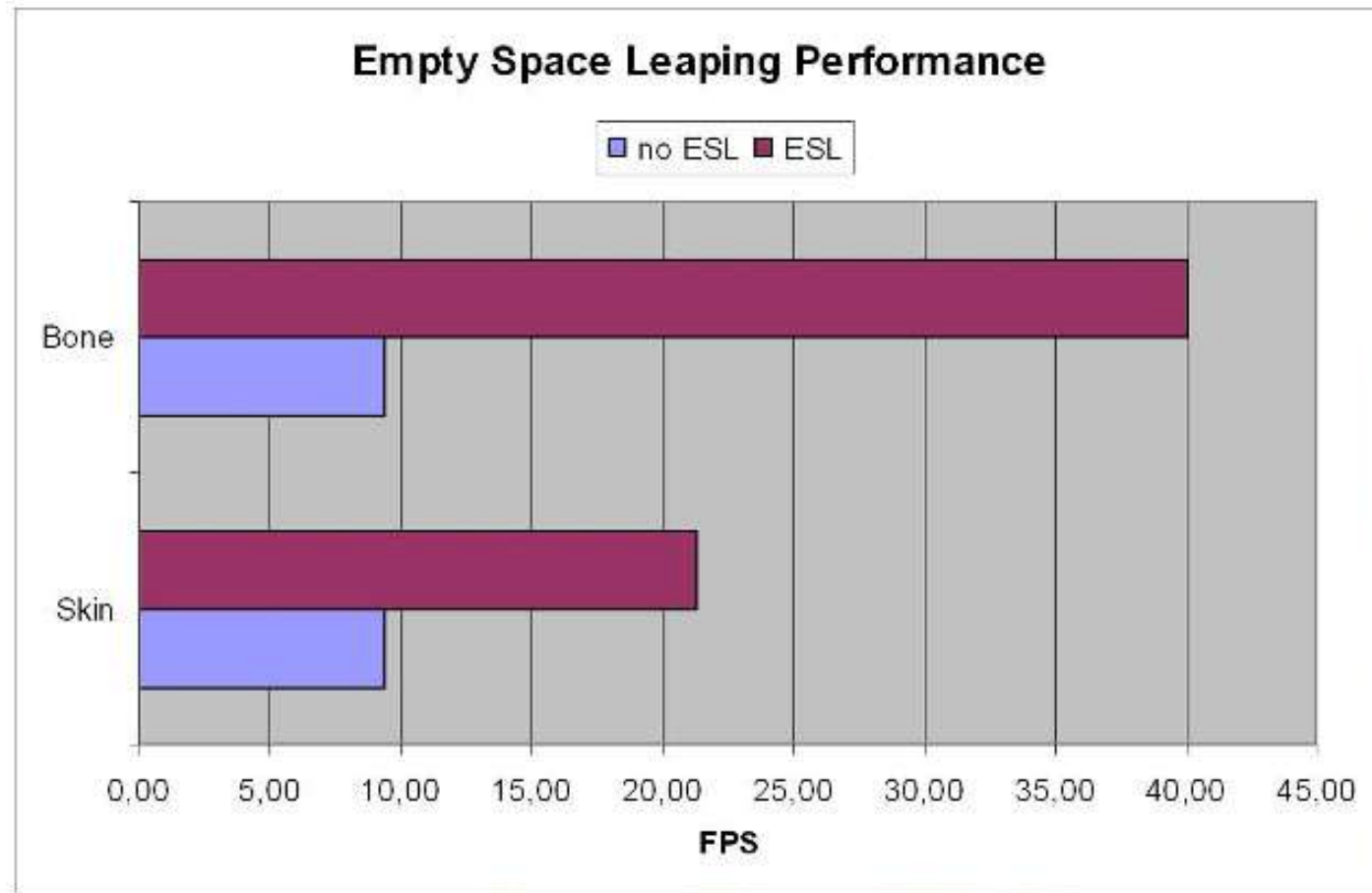
- Therefore, the optimal ratio is

$$n_{\text{nodes}} / n_{\text{data points}} \approx 0.14$$

- If all axes have at least 32 voxels, BONO has been shown to achieve a ratio that never exceeds 0.162
 - In our example above, the ratio is 0.1428

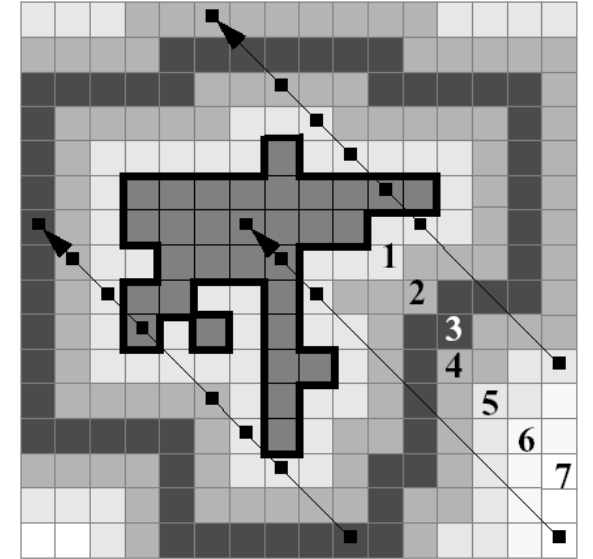
Polygon Assisted Ray Casting (PARC)

- Polygon assisted ray casting (PARC) makes use of bounding boxes around objects



Proximity Clouds for Empty Space Skipping

- Proximity clouds
 - Extension of vicinity voxels
 - Store distance to closest „object“ in voxels
 - Allows large integration steps



Precomputing Gradients

- *Reminder: Image gradient* needed for shading
 - Computing finite differences requires additional texture lookups at each integration step
 - High-quality gradient estimation (e.g., Sobel operator) can be too costly for real-time use
- **Common solution:** Pre-compute gradient and store it along with intensity information
 - Store direction and magnitude separately to avoid quantization artifacts for weak gradients
 - Often: Four extra bytes (x/y/z/magnitude);
 - Typical quantization for intensity: Two bytes (16 bit)

Summary: Faster Volume Rendering

Direct Volume Rendering can be accelerated by

- Adaptively changing **stepsize**
- Reducing required sampling rate via **pre-integration**
- **Leaving out** part of the space
 - Early ray termination
 - Empty / homogeneous space skipping
 - Min-Max Octrees
 - Bounding Polygons
 - Proximity Clouds
- **Pre-computing gradients**

Section 10.5:

Applications of Large-Scale Volume Rendering

Virtual Autopsies

- In Sweden, forensic autopsies are now preceded by a **virtual autopsy** on a routine basis
 - Collaboration between Swedish National Board of Forensic Medicine and Center for Medical Image Science and Visualization at Linköpings Universitet
 - Based on high-resolution full-body CT scan



Image from [Ljung et al. 2006]

Virtual Autopsies: Advantages and Challenges

- Different **transfer functions** provide a quick overview of
 - Skeleton and potential fractures
 - Foreign (especially metallic) objects
 - Distribution of air
- Can guide the actual autopsy and make it more efficient
 - Nondestructive, can return to volume rendering at any stage
- **Challenge:** High-quality multi-scale rendering of multi-GB data sets

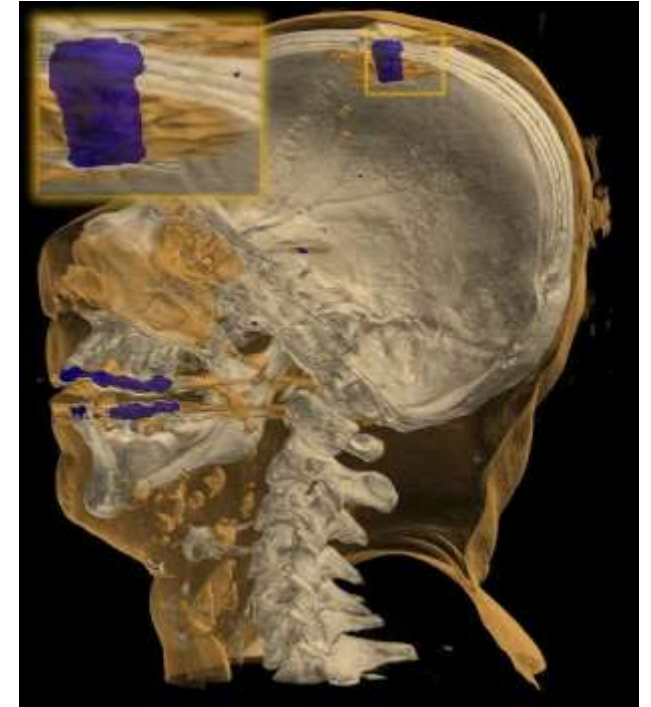


Image from [Ljung et al. 2006]

Volume Visualization in Connectomics

- **Connectome:** Entirety of all connections between neurons in an organism
 - C. elegans: 302 neurons, ≈ 7600 connections; mapped manually in 12 years
- **Large-scale scientific project:** Use serial section electron microscopy to map connectome in (small parts of) mammalian brains
 - Estimate in human brain: $\approx 10^{14}$ synapses

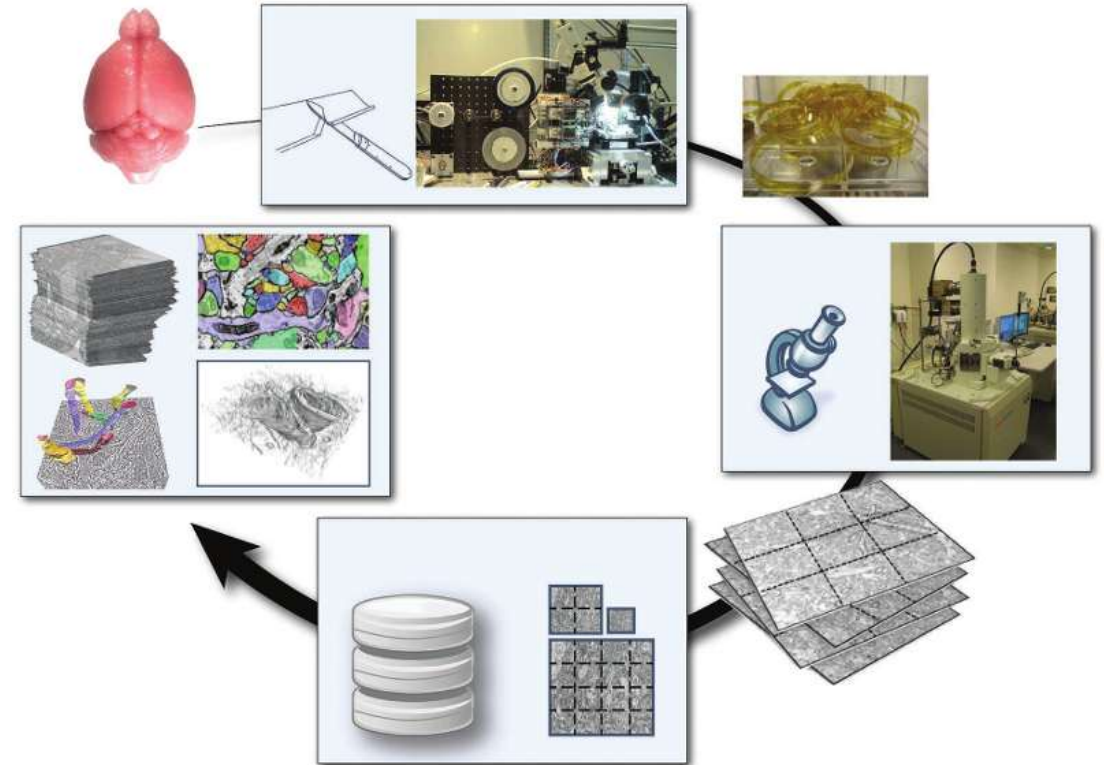
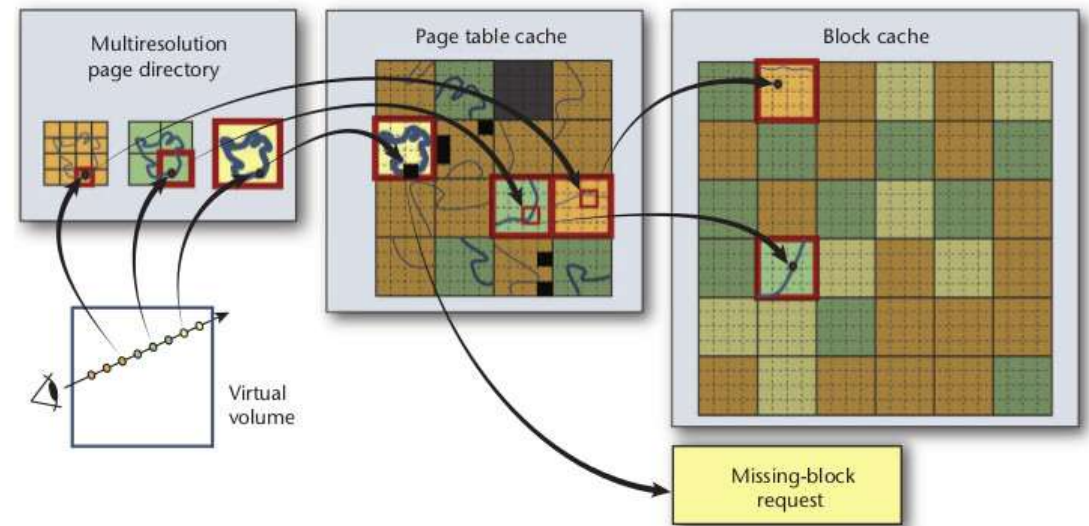


Image from [Beyer et al. 2013]

Challenges: Visualization in Connectomics

- **Large amount of anisotropic and incomplete data**

- 1mm³ of brain tissue amounts to 20,000 slices à 40 gigapixel, total of 800 TB of image data
- In-plane resolution up to 10 times larger than slice thickness
- Scanning can take months or even years, would like to start visualization before it is complete



- **Solution:**

- Dedicated GPU memory management that supports swapping, multiple resolutions (axis-specific downsampling), missing blocks

Examples: Visualization in Connectomics

- *Use case 1:* In-plane viewing with small amount of transparency

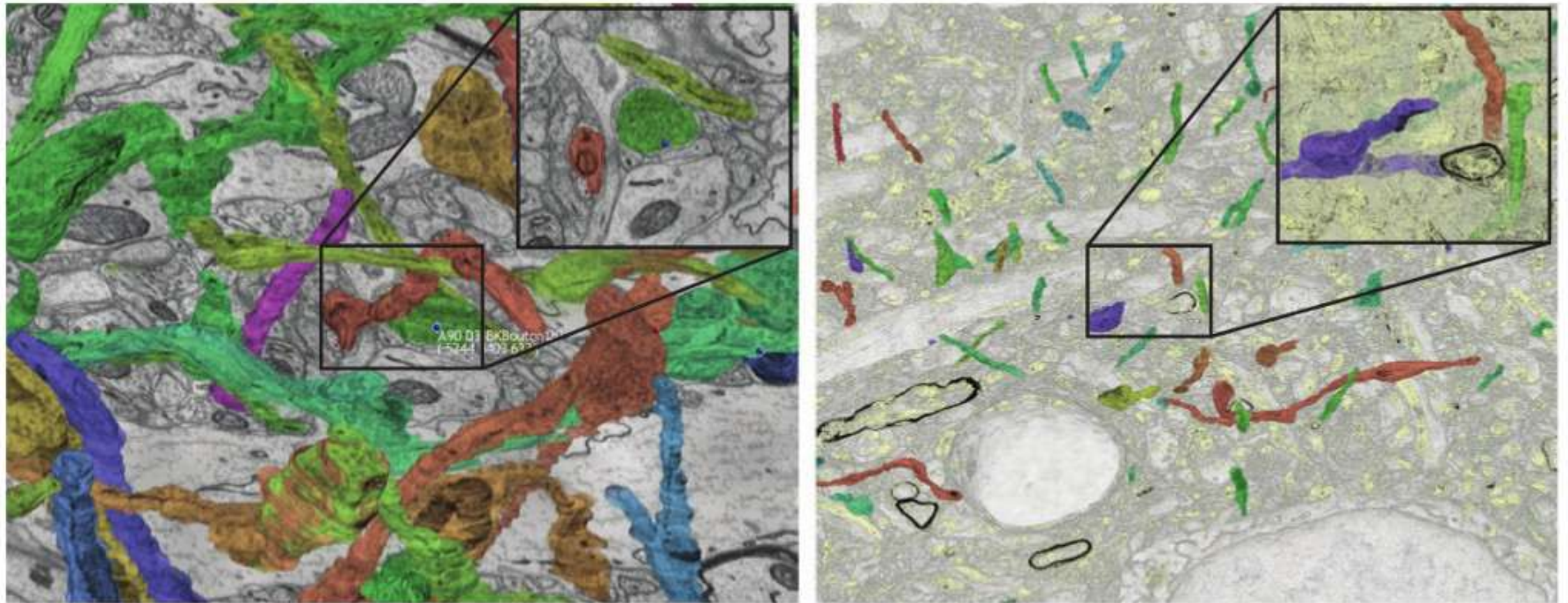
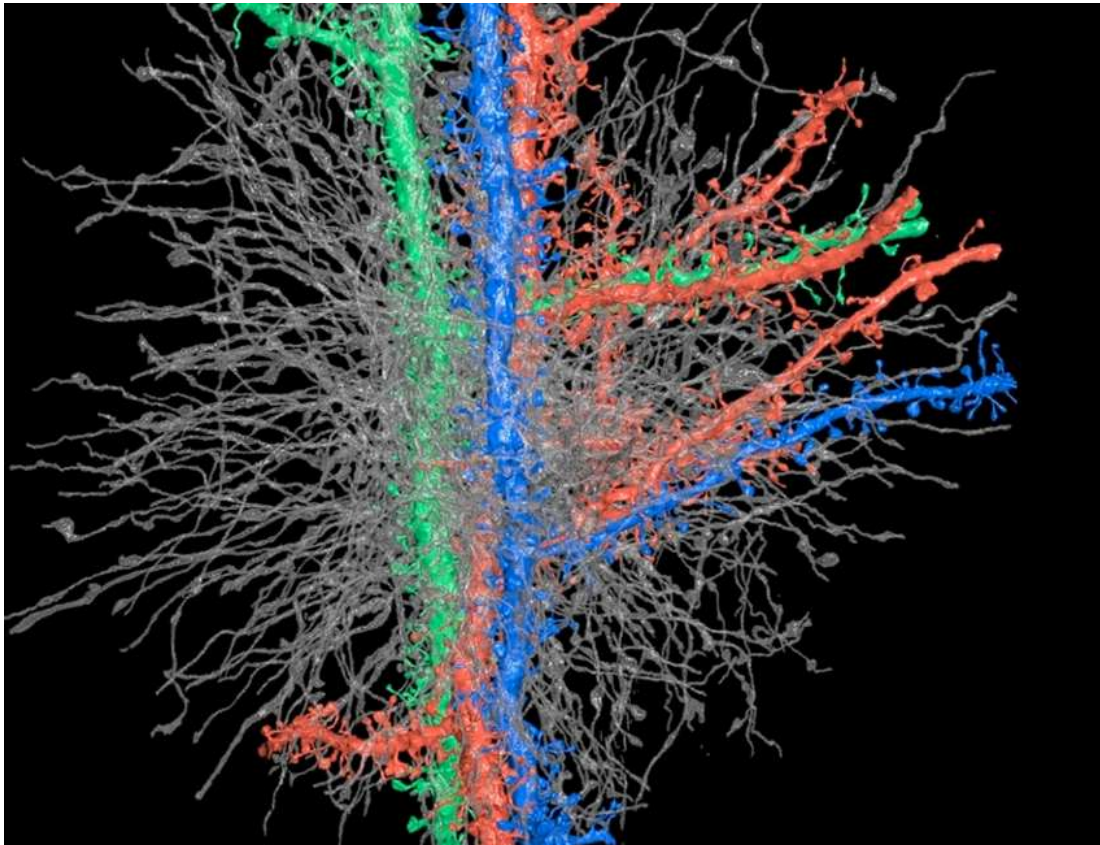


Image from [Beyer et al. 2013]

Examples: Visualization in Connectomics

- *Use case 2:* Viewing segmented dendritic trees or fly-throughs in 3D



Summary: Direct Volume Rendering

- From this chapter, you should remember:
 - The role of transfer functions in volume rendering
 - How 2D/3D transfer functions work
 - Strategies for transfer function design
 - Optical model behind direct volume rendering
 - How to solve the volume rendering integral
 - Strategies for accelerating volume rendering

Further Reading

- K. Engel, M. Hadwiger, J.M. Kniss, C. Rezk-Salama, D. Weiskopf: *Real-Time Volume Graphics*. A. K. Peters, 2006
- B. Preim, C. Botha: *Visual Computing for Medicine: Theory, Algorithms, and Applications*. Morgan Kaufmann, 2014
- C.D. Hansen, C. Johnson: *Visualization Handbook*, Academic Press, 2004

Exam-like Questions

1. What are the two dimensions of a two-dimensional transfer function? What is the main benefit of adding the second dimension compared to a traditional 1D transfer function?
2. What are associated colors and what is their benefit in direct volume rendering?
3. State a benefit of computing the direct volume rendering integral with front-to-back instead of back-to-front compositing.
4. Explain a mechanism that can be used to speed up direct volume rendering.